```
-> [NODO 2: EVALUATOR PROMETHEUS]
  [Tool Call Prometheus] Invocazione PromQL: sum(rate(container_cpu_usage_seconds_total{namespace="default", container!=""}[2m])) by (pod)
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
Prometheus MCP server running on stdio
  [Tool Call Prometheus] Invocazione PromQL: sum(container_memory_working_set_bytes{namespace="default", container!=""}) by (pod)
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
Prometheus MCP server running on stdio
  [Tool Call Prometheus] Invocazione PromQL: sum(rate(container_fs_writes_bytes_total{namespace="default", container!=""}[2m])) by (pod)
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
Prometheus MCP server running on stdio
  [Tool Call Prometheus] Invocazione PromQL: sum(rate(container_network_receive_bytes_total{namespace="default"}[2m])) by (pod)
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
Prometheus MCP server running on stdio

-> [NODO 3: PERSISTER MONGODB VIA MCP]
  [Persister Check] Applicazioni da salvare: ['cache-app', 'demo-app', 'worker-app']
  [Persister Check] Metriche calcolate: {'cache-app': {'cpu_cores': 0.0149465166192934 58, 'memory_bytes': 2318336.0, 'memory_mb': 2.21, 'disk_write_bps': 537294.5839859233, 'network_rx_bps
': 0.0}, 'demo-app': {'cpu_cores': 0.0, 'memory_bytes': 15577088.0, 'memory_mb': 14.86, 'disk_write_bps': 0.0, 'network_rx_bps': 0.0}, 'worker-app': {'cpu_cores': 0.5913665037138169, 'memo
ry_bytes': 5484544.0, 'memory_mb': 5.23, 'disk_write_bps': 0.0, 'network_rx_bps': 0.0}}
  [Tool Call MongoDB] Invocazione per app: cache-app con metrics: {'network_rx_bps': 0, 'cpu_cores': 0.014946516619293458, 'memory_bytes': 2318336, 'disk_write_bps': 537294.5839859233, 'me
mory_mb': 2.21}
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
[10/30/26 09:45:51] INFO     Processing request of type CallToolRequest                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                   server.py:733
  [Tool Call MongoDB] Invocazione per app: demo-app con metrics: {'memory_mb': 14.86, 'disk_write_bps': 0, 'memory_bytes': 15577088, 'network_rx_bps': 0, 'cpu_cores': 0}
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
[10/30/26 09:45:53] INFO     Processing request of type CallToolRequest                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                   server.py:733
  [Tool Call MongoDB] Invocazione per app: worker-app con metrics: {'network_rx_bps': 0, 'memory_bytes': 5484544, 'memory_mb': 5.23, 'cpu_cores': 0.5913665037138169, 'disk_write_bps': 0}
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
[10/30/26 09:45:55] INFO     Processing request of type CallToolRequest                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                   server.py:733

[OK] Monitoraggio iniziale completato in 46.28 s | Token ETL spesi: 7013 (In: 2202, Out: 4811)
Topologia scoperta: ['cache-app', 'demo-app', 'worker-app']
I dati sono stati persistiti su MongoDB ('agent_metrics').
```

DOMANDA 1

```
ESECUZIONE DOMANDA [1]: Quale applicazione tra quelle monitorate assorbe il maggior rate di CPU, e quanti core impiega complessivamente?
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.29 ms	0	0	0	N/A	worker-app (0.5914 cores)
Gemini 2.5 Flash	1.33 s (tot: 47.61 s)	1548	95	1643	0.48	L'applicazione che assorbe il maggior rate di CPU è 'worker-app', con un consumo complessivo di 0.59 core.
Gemini 3.6 Flash	9.09 s (tot: 55.37 s)	1562	1564	3126	0.95	In base alle metriche istantanee correnti fornite nello snapshot: L'applicazione che assorbe il maggior rate di CPU è **`worker-app`**, con un consumo complessivo di **circa 0.591 core** (nello specifico `0.5913665037138169` cpu_cores, pari a ~591.37 mCPU). --- ### Dettaglio del consumo di CPU per applicazione 1. **`worker-app`**: **0.5914 core** (~591.37 mCPU) * **Pod attivi (2):** `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` * **Memoria:** 5.23 MB 2. **`cache-app`**: **0.0149 core** (~14.95 mCPU) * **Pod attivi (1):** `cache-app-5c6fd4f5cf-xxsz4` * **Memoria:** 2.21 MB 3. **`demo-app`**: **0.0000 core** (0 mCPU) * **Pod attivi (2):** `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rrjl` * **Memoria:** 14.86 MB --- ### Osservazioni e Raccomandazioni SRE * **Carico attivo (`worker-app`)**: Rappresenta la quasi totalità dell'elaborazione CPU attiva nel cluster. Se si prevedono picchi di carico, è consigliabile monitorare se la frequenza di calcolo satura i limiti/richieste dei container e valutare l'abilitazione di un **Horizontal Pod Autoscaler (HPA)** basato sull'utilizzo CPU. * **Stato Idle (`demo-app`)**: L'applicazione risulta completamente in idle dal punto di vista CPU (0.0 core) e di I/O, pur mantenendo 2 repliche attive e occupando ~14.86 MB di RAM. Si raccomanda di valutare la riduzione delle repliche a 1 (o lo scaling a 0 in ambienti non produttivi). * **I/O su Disco (`cache-app`)**: Sebbene il consumo CPU sia ridotto (~15 mCPU), `cache-app` presenta un throughput di scrittura su disco significativo (~537 KB/s). Verificare la persistenza per evitare saturazioni di I/O storage.
Claude Opus 5.5	9.74 s (tot: 56.02 s)	2695	863	3558	0.95	**worker-app** è l'applicazione che usa più CPU: **circa 0,59 core** in totale (0,5914), cioè quasi 600 millicore. **Consumo CPU per applicazione** | Applicazione | Pod | CPU (core) | Quota sul totale | |---|---|---|---| | **worker-app** | 2 | **0,5914** | **~97,5%** | | cache-app | 1 | 0,0149 | ~2,5% | | demo-app | 2 | 0,0000 | 0% | | **Totale** | 5 | **~0,606** | 100% | **Cosa significa** - **Il consumo è quasi tutto su worker-app.** Il dato è la somma dei suoi due pod (`9kj75` e `ftlln`). Se il carico fosse diviso in parti uguali, ogni pod userebbe circa 0,30 core. Lo snapshot però non riporta il dato per singolo pod, quindi non so se il carico sia bilanciato. - **worker-app non riceve traffico di rete** (network_rx = 0). Il carico CPU sembra quindi generato dall'applicazione stessa (ad esempio job batch o cicli interni), non da richieste esterne. In questo caso un HPA basato sulla CPU servirebbe solo se il lavoro si può davvero dividere tra più repliche. - **demo-app è inattiva** sul fronte CPU (0 core, nessun traffico, nessuna scrittura su disco) pur avendo 2 repliche. Si può valutare di ridurla a 1 replica, dopo aver controllato lo storico per escludere picchi in altri momenti. **Verifiche consigliate** - Controllare se 0,59 core si avvicinano ai `limits.cpu` di worker-app. Se sì, c'è rischio di CPU throttling (rallentamento imposto dal limite). Lo snapshot non contiene requests e limits. - Controllare se il carico è costante o a picchi, con lo storico delle ultime ore. Se vuoi, posso recuperare lo storico di worker-app e il dettaglio CPU e throttling per singolo pod da Prometheus.

Claude Sonnet 5.5	6.00 s (tot: 52.28 s)	2695	487	3182	0.95	L'applicazione che assorbe più CPU è **worker-app**, con circa **0,59 core** (0,5914 core, cioè circa 591 millicore). Confronto con le altre applicazioni, dai dati dello snapshot: | Applicazione | CPU (core) | Pod | |---|---|---| | **worker-app** | **0,591** | 2 | | cache-app | 0,015 | 1 | | demo-app | 0,000 | 2 | - worker-app usa circa 40 volte la CPU di cache-app. Rappresenta quasi tutto il consumo CPU osservato, cioè il 975% del totale di circa 0,606 core. - Il valore è aggregato sui 2 pod, quindi la media è circa 0,30 core per pod. Lo snapshot non dice come si divide tra i due pod. - La memoria di worker-app è bassa (5,23 MB), quindi il carico è CPU-bound. - demo-app è a 0 core su 2 pod e non ha traffico di rete né scritture su disco. Sembra idle, e si potrebbe valutare di ridurne le repliche a 1. Questi sono valori istantanei. Non so se il consumo di worker-app sia stabile o un picco. Prima di decidere su un HPA o su un aumento dei limiti, conviene guardare lo storico con `get_historical_metrics`. Se vuoi, lo recupero io.
GPT-4o Mini	2.83 s (tot: 49.11 s)	1247	93	1340	0.70	L'applicazione che assorbe il maggior rate di CPU è la **worker-app**, che utilizza attualmente **0.5914 core** di CPU. Le altre applicazioni hanno i seguenti utilizzi di CPU: - **cache-app**: 0.0149 core - **demo-app**: 0.0 core Quindi, la **worker-app** è quella con il consumo di CPU più elevato.
GPT-4o	2.29 s (tot: 48.57 s)	1247	36	1283	0.48	L'applicazione che assorbe il maggior rate di CPU tra quelle monitorate è la "worker-app", che utilizza complessivamente 0.591 core.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2428 (mini: $0.0535 | 4o: $1.1893)

DOMANDA 1 A

ESECUZIONE DOMANDA [1A]: Identifica quale servizio nel cluster presenta il consumo di CPU più elevato e quantifica l'ammontare totale di core utilizzati sommando tutte le sue istanze.

Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.84 ms	0	0	0	N/A	worker-app (0.5914 cores)
Gemini 2.5 Flash	1.87 s (tot: 48.14 s)	1556	159	1715	0.48	Il servizio con il consumo di CPU più elevato è **worker-app** con un utilizzo totale di **0.591 core**.
Gemini 3.6 Flash	19.40 s (tot: 65.68 s)	1570	1721	3291	0.95	In base alle metriche istantanee correnti e alla topologia dei pod fornita, ecco l'analisi del consumo di CPU per ciascun servizio del cluster: ### 1. Servizio con il consumo di CPU più elevato Il servizio con il maggior utilizzo di CPU è **`worker-app`**. * **Consumo totale di CPU:** **`0,5914 core`** (pari a **`591,37 mCPU`** / `0,5913665837138169 core`). * **Istanze attive:** 2 pod (`worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`). --- ### 2. Panoramica del consumo di CPU per tutti i servizi | Servizio | Pod Attivi | CPU Totale Utilizzata (Core) | CPU Totale (mCPU) | Stato | | :--- | :--- | :--- | :--- | :--- | | **`worker-app`** | 2 | **0,591367** | **591,37 mCPU** | **Consumo più elevato** | | **`cache-app`** | 1 | **0,014947** | **14,95 mCPU** | Basso utilizzo | | **`demo-app`** | 2 | **0,000000** | **0,00 mCPU** | Idle / Inattivo | --- ### Raccomandazioni SRE * **`worker-app`**: Essendo il carico principale elaborativo del cluster (circa 0,59 core totali), si consiglia di monitorare se l'utilizzo è bilanciato tra i due pod e di impostare/verificare regole di **HPA (Horizontal Pod Autoscaler)** basate sul target di CPU (es. 70-80%), assicurandosi che le *CPU Requests/Limits* dei pod siano dimensionati adeguatamente. * **`demo-app`**: Risulta completamente inattivo (0 core CPU). Se non impiegato in ambiente di produzione o staging, è un candidato per la riduzione delle repliche a 0 o dismissione per liberare risorse.

Claude Opus 5.5	13.76 s (tot: 60.04 s)	2712	1238	3950	0.95	# Il servizio che usa più CPU è worker-app ## Risultato | Servizio | Pod attivi | CPU totale (core) | CPU per pod (stima) | Quota sul totale del cluster | |---|---|---|---|---| | **worker-app** | 2 | **0,5914** (circa 591 millicore) | circa 0,296 | **97,5%** | | cache-app | 1 | 0,0149 (circa 15 millicore) | 0,0149 | 2,5% | | demo-app | 2 | 0,0000 | 0,0000 | 0% | | **Totale cluster** | 5 | **0,6063** | – | 100% | **worker-app usa in tutto circa 0,59 core**, somma dei suoi due pod: - `worker-app-84cf4dd446-9kj75` - `worker-app-84cf4dd446-ftlln` Il valore per pod è una stima: ho diviso il totale a metà, ipotizzando che i due pod lavorino in modo uguale. Lo snapshot riporta un solo valore per ogni applicazione, che sembra già la somma dei suoi pod. ## Cosa significa - **worker-app è l'unico servizio che lavora davvero**: da solo usa quasi tutta la CPU del cluster. La memoria resta bassa (5,23 MB), quindi è un carico che pesa sulla CPU e non sulla memoria. - **Non posso dire se worker-app è vicino alla saturazione.** Lo snapshot non riporta la CPU riservata (`requests`) né il limite massimo (`limits`) dei container. Per esempio, con un limite di 500m a pod ogni pod sarebbe al 59% circa; con un limite di 250m sarebbe già oltre il limite e verrebbe rallentato (throttling). - **demo-app non fa nulla**: CPU, disco e rete sono a zero e usa solo 14,86 MB di memoria. Con due repliche ferme si può valutare di scendere a una sola, se non serve alta affidabilità. - **cache-app usa pochissima CPU** ma scrive su disco a circa 0,51 MB/s, cosa coerente con un servizio di cache che salva i dati su disco. ## Prossimi passi consigliati 1. **Controllare `requests` e `limits` di worker-app** e la metrica `container_cpu_cfs_throttled_periods_total` su Prometheus, per capire se viene già rallentato. 2. **Valutare uno scaling automatico (HPA) per worker-app** sull'uso della CPU (per esempio con obiettivo al 70% della CPU riservata, minimo 2 e massimo 4–5 repliche). Prima conviene confermare con lo storico (`get_historical_metrics`) che il carico è costante o cresce e non è solo un picco momentaneo. 3. **Ridurre demo-app a 1 replica** se lo storico conferma che resta inattiva nel tempo. Se vuoi posso fare io queste verifiche (andamento storico, rallentamenti, limiti dei container) con le query su Prometheus e sul database.
Claude Sonnet 5.5	13.38 s (tot: 59.66 s)	5710	1019	6729	0.95	**Il servizio con il consumo di CPU più alto è `worker-app`, con circa 0,59 core totali sulle due repliche.** | Servizio | Repliche | CPU totale (core) | |---|---|---| | **worker-app** | 2 | **≈ 0,591** | | cache-app | 1 | ≈ 0,015 | | demo-app | 2 | 0,000 | Ho controllato il dato con una query Prometheus sui singoli pod di `worker-app`. Il rate della CPU su 5 minuti è: - `worker-app-84cf4dd446-ftlln`: ≈ 0,2956 core - `worker-app-84cf4dd446-9kj75`: ≈ 0,2955 core - Somma: ≈ 0,591 core, in linea con lo snapshot (0,5914). **Osservazioni** - `worker-app` usa circa 40 volte la CPU di `cache-app` e consuma quasi tutta la CPU misurata nel cluster, circa il 97%. - Il carico è bilanciato tra i due pod, con circa 0,30 core ciascuno. Non vedo pod anomali o sbilanciati. - La memoria di `worker-app` è bassa, circa 5,2 MB. Il carico è quindi legato alla CPU e non alla memoria. - Non conosco i `requests` e i `limits` CPU dei pod, quindi non posso dire se `worker-app` sia vicino alla saturazione. Controllali prima di decidere. - Se i `requests` sono vicini a 0,3 core per pod, ti consiglio un HPA basato sulla CPU, con un minimo di 2 repliche. - `demo-app` ha CPU, disco e rete a zero su 2 repliche, quindi sembra inattivo. Puoi valutare di ridurlo a 1 replica, dopo aver verificato con lo storico (`get_historical_metrics`) che non abbia picchi periodici.
GPT-4o Mini	2.78 s (tot: 49.06 s)	1255	126	1381	0.70	Analizzando le metriche istantanee fornite, possiamo osservare i seguenti consumi di CPU per ciascuna applicazione: - **cache-app**: 0.0149 core - **demo-app**: 0.0 core - **worker-app**: 0.5914 core Il servizio che presenta il consumo di CPU più elevato è **worker-app**, con un totale di **0.5914 core** utilizzati. In sintesi, il servizio con il consumo di CPU più elevato è **worker-app** con **0.5914 core**.
GPT-4o	2.14 s (tot: 48.42 s)	1255	50	1305	0.50	Il servizio nel cluster che presenta il consumo di CPU più elevato è il "worker-app". La somma totale dei core di CPU utilizzati da tutte le sue istanze è di 0.5913665037138169 core.

DOMANDA 1 B

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [18]: Dall'analisi delle metriche di utilizzo del processore, quale applicazione risulta essere il top consumer di cicli CPU e qual è il suo tasso di assorbimento complessivo in core?
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.54 ms	0	0	0	N/A	worker-app (0.5914 cores)
Gemini 2.5 Flash	1.45 s (tot: 47.73 s)	1561	86	1647	0.47	L'applicazione "worker-app" è il top consumer di cicli CPU, con un tasso di assorbimento complessivo di 0.591 core.
Gemini 3.6 Flash	5.20 s (tot: 51.48 s)	1575	588	2163	0.80	Dall'analisi delle metriche istantanee correnti sull'utilizzo del processore: * **Top Consumer:** **`worker-app`** * **Tasso di assorbimento complessivo CPU:** **`0.591` core** (esattamente ~`0.591366` core, pari a circa 591 millicore) ### Confronto sintetico del consumo CPU tra le applicazioni: 1. **`worker-app`**: **0.5914 core** (Top Consumer - ripartito su 2 pod) 2. **`cache-app`**: **0.0149 core** (~15 millicore) 3. **`demo-app`**: **0.0000 core** (Stato Idle/Inattivo)
Claude Opus 5.5	12.62 s (tot: 58.90 s)	2718	1088	3806	0.95	# Chi usa più CPU **worker-app** usa più CPU di tutte le altre applicazioni: consuma in totale **circa 0,591 core** (0,5914, cioè circa 591 millicore). ## Confronto tra le applicazioni | Applicazione | Pod | CPU (core) | Quota sul totale | |---|---|---|---| | **worker-app** | 2 | **0,5914** | **circa 97,5%** | | cache-app | 1 | 0,0149 | circa 2,5% | | demo-app | 2 | 0,0000 | 0% | | **Totale** | 5 | **0,6063** | 100% | ## Cosa significano questi numeri - **Quanto pesa worker-app:** usa circa 40 volte la CPU di cache-app e quasi tutta la CPU usata dalle tre applicazioni. - **Per pod:** worker-app ha 2 repliche. Se il carico è diviso in parti uguali, ogni pod usa circa 0,296 core. Il dato del momento però è solo il totale, quindi non so come il carico si distribuisce davvero tra i due pod. - **Memoria:** worker-app usa solo 5,23 MB, e né lui né le altre applicazioni registrano traffico di rete in entrata (0 B/s). Sembra quindi un carico che lavora soprattutto sulla CPU, come un'elaborazione batch, un calcolo continuo o un ciclo di attesa attiva. - **demo-app** non usa CPU, non scrive su disco e non riceve traffico di rete, ma tiene accesi 2 pod. Sembra inattiva: conviene valutare se ridurla a 1 replica. ## Limiti di questa analisi Non posso dire se worker-app sta esaurendo la CPU a sua disposizione. Questi dati non includono le **requests/limits** dei container, quindi non conosco il suo tetto. Ad esempio, con un limite di 300m per pod sarebbe già quasi al massimo e rallentato dal sistema (throttling). Con un limite di 1 core avrebbe invece ampio margine. Per decidere se attivare l'HPA (scaling automatico delle repliche) su worker-app, suggerisco di controllare: 1. i limiti di CPU impostati (`kube_pod_container_resource_limits{resource="cpu"}`); 2. i periodi in cui la CPU viene rallentata (`container_cpu_cfs_throttled_periods_total`); 3. se il consumo è costante o a picchi, guardando lo storico delle ultime ore. Se vuoi, posso fare io queste verifiche con Prometheus e con lo storico salvato su MongoDB.
Claude Sonnet 5.5	5.96 s (tot: 52.24 s)	2718	685	3403	0.90	Il top consumer di CPU è **worker-app**, con un assorbimento complessivo di circa **0,591 core** (≈591 millicore). Confronto con le altre applicazioni, dallo snapshot corrente: | Applicazione | CPU (core) | Repliche | |---|---|---| | **worker-app** | **0,591** | 2 | | cache-app | 0,015 | 1 | | demo-app | 0,000 | 2 | - worker-app consuma circa 40 volte più CPU di cache-app. Il valore è il totale aggregato del deployment, quindi per ciascuno dei 2 pod fa in media circa 0,30 core. Lo snapshot non dà la ripartizione per pod. - Il consumo di memoria di worker-app è basso (5,23 MB) e non c'è traffico di rete in ingresso né scrittura su disco. Il carico è quindi di calcolo puro (CPU-bound). - cache-app ha un consumo di CPU trascurabile (≈15 millicore), ma scrive su disco a circa 537 KB/s. - demo-app risulta idle in termini di CPU (0 core), di rete e di disco, pur avendo 2 repliche e 14,86 MB di memoria. Potrebbe essere ridotta a 1 replica, ma conviene prima verificare lo storico. **Raccomandazioni** - Il dato è istantaneo, quindi non so dire se il carico di worker-app sia un picco o sia costante. Conviene controllare lo storico con `get_historical_metrics` prima di decidere sul dimensionamento. - Se il valore è stabile, verificate i `requests` e i `limits` di CPU di worker-app. Con un limite inferiore a circa 0,3 core per pod si rischia il throttling. - Se il carico cresce, un HPA basato sulla CPU su worker-app sarebbe appropriato. Se volete, posso estrarre lo storico delle ultime ore per confermare il trend.
GPT-4o Mini	1.99 s (tot: 48.27 s)	1265	118	1383	0.50	Dall'analisi delle metriche di utilizzo del processore, l'applicazione che risulta essere il top consumer di cicli CPU è la **worker-app**, con un tasso di assorbimento complessivo di **0.5914 core**. Le altre applicazioni hanno i seguenti valori di utilizzo della CPU: - **cache-app**: 0.0149 core - **demo-app**: 0.0 core Quindi, la worker-app è chiaramente l'applicazione con il maggior consumo di CPU.

GPT-4o	1.41 s (tot: 47.69 s)	1265	55	1320	0.47	Dall'analisi delle metriche istantanee fornite, l'applicazione "worker-app" risulta essere il top consumer di cicli CPU. Il suo tasso di assorbimento complessivo in core è di 0.591 core.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $1.2584 (mini: $0.0544 | 4o: $1.2040)
```

DOMANDA 2

```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [2]: Qual è l'uso effettivo di memoria Working Set (in MB) dell'applicazione con il maggior numero di pod o repliche?
--------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	3.38 ms	0	0	0	N/A	demo-app: 14.86 MB
Gemini 2.5 Flash	5.23 s (tot: 51.51 s)	1550	967	2517	0.70	Le applicazioni con il maggior numero di pod (2 repliche ciascuna) sono `demo-app` e `worker-app`. * **demo-app**: L'uso effettivo di memoria Working Set è di **14.86 MB**. * **worker-app**: L'uso effettivo di memoria Working Set è di **5.23 MB**.
Gemini 3.6 Flash	6.85 s (tot: 53.12 s)	1564	1258	2822	0.80	Analizzando la topologia e le metriche istantanee attuali: ### 1. Identificazione delle Applicazioni con il Maggior Numero di Pod/Repliche Le applicazioni che presentano il numero massimo di repliche attive (**2 pod** ciascuna) sono due: * **`demo-app`** (2 pod: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`) * **`worker-app`** (2 pod: `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`) *(Per confronto, `cache-app` ha 1 solo pod).* --- ### 2. Uso Effettivo di Memoria Working Set (in MB) L'uso effettivo di memoria per ciascuna di queste due applicazioni è il seguente: * **`demo-app`**: **14.86 MB** (15.577.088 byte) * **`worker-app`**: **5.23 MB** (5.484.544 byte) Se si considera l'applicazione con il maggior consumo di memoria assoluto tra quelle a massimo numero di repliche, questa è **`demo-app`** con **14.86 MB**.

Claude Opus 5.5	21.31 s (tot: 67.59 s)	5809	1278	7087	0.90	**Risposta:** non c'è un'unica applicazione con più pod. **demo-app** e **worker-app** hanno entrambe **2 repliche**, quindi riporto il Working Set di tutte e due. cache-app ha un solo pod ed è esclusa. Ho interrogato Prometheus (`container_memory_working_set_bytes`, senza i container vuoti e `POD`) per avere il dettaglio di ogni pod: | Applicazione | Pod | Working Set | |---|---|---| | **demo-app** | demo-app-7958c64697-2qg9s | 7,46 MB | | | demo-app-7958c64697-4rzjl | 7,38 MB | | | **Totale** | **≈ 14,84 MB** (snapshot: 14,86 MB) | | **worker-app** | worker-app-84cf4dd446-9kj75 | 3,60 MB | | | worker-app-84cf4dd446-ftlln | 1,59 MB | | | **Totale** | **≈ 5,19 MB** (snapshot: 5,23 MB) | I MB qui sono MiB (1 MB = 1.048.576 byte). **Cosa noto:** - **Il dato è affidabile:** la query diretta coincide con lo snapshot salvato. La piccola differenza dipende dal momento diverso in cui sono stati presi i due valori. - **demo-app sembra ferma:** occupa circa 7,4 MB per pod in modo uniforme, ma ha CPU a 0 e traffico di rete in entrata a 0. Se il dato si conferma nel tempo, si può valutare di **ridurla a 1 replica**. - **worker-app è il carico attivo:** usa poca memoria ma circa 0,59 core di CPU complessivi. Se ha dei limiti di CPU impostati, è lei la candidata a uno scaling automatico (HPA) basato sulla CPU. Inoltre i suoi due pod usano memoria in modo diverso (3,60 contro 1,59 MB). Con valori così bassi può essere normale, ma potrebbe anche voler dire che il lavoro non è distribuito in modo uniforme tra i pod. - **Nessun rischio di memoria piena:** tutti i consumi sono molto bassi. Se mi dici quale delle due applicazioni ti interessa, posso controllare lo storico delle ultime ore per confermare se demo-app è davvero ferma e come si distribuisce il carico di worker-app.
Claude Sonnet 5.5	18.05 s (tot: 64.33 s)	5851	1031	6882	0.90	Due applicazioni hanno il numero massimo di repliche, cioè 2 pod

Claude Sonnet 5.5	18.05 s (tot: 64.33 s)	5851	1031	6882	0.90	Due applicazioni hanno il numero massimo di repliche, cioè 2 pod ciascuna: **demo-app** e **worker-app**. Ti riporto il Working Set di entrambe. | Applicazione | Pod | Working Set (Prometheus, somma dei pod) | Snapshot | |---|---|---|---| | **demo-app** | 2 | 15.560.704 byte, circa **14,84 MB** | 14,86 MB | | **worker-app** | 2 | 5.443.584 byte, circa **5,19 MB** | 5,23 MB | `cache-app` ha un solo pod, quindi non rientra nel confronto. - I valori Prometheus (`container_memory_working_set_bytes`) e quelli dello snapshot differiscono di pochi decimi di MB. Il motivo è che sono stati campionati in istanti leggermente diversi. - Sono valori aggregati sui due pod, quindi la media per pod è circa 7,4 MB per demo-app e circa 2,6 MB per worker-app. - Per la richiesta, se serve un solo valore, il più alto tra le due app a pari repliche è **demo-app con circa 14,84 MB**. Il valore di worker-app è circa 5,19 MB. **Osservazioni SRE:** - **demo-app** usa 0 core di CPU e non ha traffico di rete né scritture su disco, quindi sembra idle. Con 2 repliche si può valutare di ridurle a 1. - **worker-app** usa circa 0,59 core di CPU con pochissima memoria. È un carico CPU-bound e il candidato naturale per un HPA basato sulla CPU.
GPT-4o Mini	1.55 s (tot: 47.83 s)	1250	52	1302	0.70	L'applicazione con il maggior numero di pod attivi è la "demo-app", che ha attualmente 2 repliche. L'uso effettivo di memoria Working Set della "demo-app" è di 14.86 MB.
GPT-4o	1.46 s (tot: 47.74 s)	1250	52	1302	0.50	L'applicazione con il maggior numero di pod o repliche è la "demo-app", che ha 2 pod attivi. L'uso effettivo di memoria Working Set per la "demo-app" è di 14.86 MB.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2623 (mini: $0.0547 | 4o: $1.2076)

DOMANDA 2 A

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [2A]: Individua il carico di lavoro con il deployment più esteso per numero di pod e calcola a quanti megabyte ammonta la sua memoria Working Set totale.
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.35 ms	0	0	0	N/A	demo-app: 14.86 MB
Gemini 2.5 Flash	2.24 s (tot: 48.52 s)	1556	280	1836	0.50	Il carico di lavoro con il deployment più esteso per numero di pod è `demo-app` (con 2 pod). La sua memoria Working Set totale ammonta a 14.86 MB.
Gemini 3.6 Flash	9.44 s (tot: 55.72 s)	1570	1749	3319	0.95	Sulla base dei dati forniti nello snapshot della topologia e delle metriche istantanee correnti, ecco l'analisi richiesta: ### 1. Identificazione dei Deployment con maggior numero di Pod Analizzando la topologia attuale, vi sono due deployment a pari merito con il numero massimo di pod (**2 pod ciascuno**): * **`demo-app`** (2 pod): * `demo-app-7958c64697-2qg9s` * `demo-app-7958c64697-4rzjl` * **`worker-app`** (2 pod): * `worker-app-84cf4dd446-9kj75` * `worker-app-84cf4dd446-ftlln` *(A titolo di confronto, `cache-app` dispone di un solo pod).* --- ### 2. Memoria Working Set Totale Di seguito i dati relativi alla memoria Working Set totale per i due deployment più estesi: * **`demo-app`**: * **Memoria totale:** **14.86 MB** (`15.577.088 bytes`) * **Stato del carico:** Inattivo / Idle (`CPU: 0.0 cores`) * **`worker-app`**: * **Memoria totale:** **5.23 MB** (`5.484.544 bytes`) * **Stato del carico:** Attivo (`CPU: ~0.591 cores`) --- ### Sintesi e Raccomandazioni SRE * Se l'obiettivo dell'analisi è l'occupazione di memoria su deployment a massima estensione, **`demo-app`** rappresenta il carico di lavoro con il consumo di memoria Working Set più elevato (**14.86 MB**). * Considerando che **`demo-app`** è in stato *idle* (0.0 CPU) con 2 repliche attive, si potrebbe valutare un ridimensionamento orizzontale del deployment (riduzione delle repliche o scale-to-zero se supportato) per ottimizzare le risorse del cluster.

Claude Opus 5.5	10.44 s (tot: 56.72 s)	2705	1006	3711	0.86	# Deployment più esteso e memoria Working Set **C'è un pareggio:** due deployment hanno lo stesso numero massimo di pod, quindi li riporto entrambi. | Deployment | Pod | Memoria (byte) | MiB (÷1.048.576) | MB (÷1.000.000) | |---|---|---|---|---| | **demo-app** | 2 (`-2qg9s`, `-4rzjl`) | 15.577.088 | **14,86 MiB** | 15,58 MB | | **worker-app** | 2 (`-9kj75`, `-ftlln`) | 5.484.544 | **5,23 MiB** | 5,48 MB | | cache-app | 1 | 2.318.336 | 2,21 MiB | 2,32 MB | ### Risposta - **demo-app** ha la memoria più alta tra i due: **circa 14,86 MiB** (15,58 MB decimali). - **worker-app** usa **circa 5,23 MiB** (5,48 MB decimali). - Insieme fanno 21.061.632 byte, cioè **circa 20,09 MiB**. **Nota:** do per scontato che `memory_bytes` sia la somma del Working Set di tutti i pod del deployment, cioè `container_memory_working_set_bytes`. Il formato dei dati lo suggerisce, ma lo snapshot non lo dice in modo esplicito. ### Osservazioni - **demo-app sembra inattivo:** CPU a 0, nessun traffico di rete e nessuna scrittura su disco, ma tiene comunque 2 repliche. Se non deve gestire picchi improvvisi o garantire alta disponibilità, si può **scendere a 1 replica** oppure usare un HPA con `minReplicas: 1`. - **worker-app è l'unico carico che usa davvero la CPU:** circa 0,59 core in totale, quindi circa 0,30 per pod se il carico è distribuito in modo uniforme. Usa invece pochissima memoria. Conviene un **HPA basato sulla CPU**. Lo snapshot non riporta le CPU requests, quindi prima di fissare il target controlla se ci sono e quanto valgono.
Claude Sonnet 5.5	14.43 s (tot: 60.71 s)	5895	1348	7243	0.86	**Risultato:** nessun deployment è il più esteso, perché **demo-app** e **worker-app** hanno entrambi 2 pod. cache-app ne ha 1. Ho quindi calcolato la memoria working set (`container_memory_working_set_bytes`) di tutti e due. | Deployment | Pod | Working set per pod | Totale | |---|---|---|---| | demo-app | 2qg9s: 7.819.264 B 4rzjl: 7.741.440 B | 7,46 MB e 7,38 MB | **15.560.704 B ≈ 14,84 MB** | | worker-app | 9kj75: 3.698.688 B ftlln: 1.372.160 B | 3,53 MB e 1,31 MB | **5.070.848 B ≈ 4,84 MB** | Ho usato i megabyte binari (1 MB = 1.048.576 B), come fa lo snapshot. In megabyte decimali i totali sono circa 15,56 MB per demo-app e 5,07 MB per worker-app. Rispetto allo snapshot ci sono piccole differenze: 14,86 MB contro 14,84 MB per demo-app e 5,23 MB contro 4,84 MB per worker-app. Probabilmente i campioni sono stati presi in momenti diversi o con filtri sui container diversi. Su worker-app lo scarto è più marcato. **Osservazioni SRE:** - **demo-app:** ha CPU a 0, nessun traffico di rete e nessuna scrittura su disco. È idle e ha 2 repliche, quindi si può valutare di ridurla a 1. - **worker-app:** usa 0,59 core su 2 pod con pochissima memoria, quindi è un carico CPU-bound. Un HPA basato sulla CPU avrebbe senso. Il working set dei due pod è sbilanciato (3,5 MB contro 1,3 MB), il che suggerisce che il lavoro non sia distribuito in modo uniforme.
GPT-4o Mini	1.99 s (tot: 48.27 s)	1255	110	1365	0.21	Il deployment con il numero maggiore di pod è "demo-app", che attualmente ha 2 pod attivi. La memoria Working Set totale per "demo-app" è calcolata come segue: - Memoria di un singolo pod: 14.86 MB - Numero di pod: 2 Memoria totale = 14.86 MB * 2 = 29.72 MB Quindi, la memoria Working Set totale per il deployment "demo-app" ammonta a **29.72 MB**.
GPT-4o	1.68 s (tot: 47.96 s)	1255	75	1330	0.70	Il deployment con il maggior numero di pod attivi è il "demo-app", che ha 2 pod attivi: "demo-app-7958c64697-2qg9s" e "demo-app-7958c64697-4rzjl". La memoria Working Set totale per il "demo-app" è di 14.86 MB.

DOMANDA 2B

```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [2B]: Considerando le applicazioni che contano il maggior numero di repliche attive, a quanto corrisponde il loro consumo aggregato di Working Set espresso in MB?
--------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3988: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.26 ms	0	0	0	N/A	demo-app: 14.86 MB
Gemini 2.5 Flash	2.19 s (tot: 48.46 s)	1553	315	1868	0.00	Le applicazioni con il maggior numero di repliche attive sono 'demo-app' e 'worker-app', entrambe con 2 repliche. Il loro consumo aggregato di Working Set è: - `demo-app`: 14.86 MB - `worker-app`: 5.23 MB Totale: 14.86 MB + 5.23 MB = 20.09 MB
Gemini 3.6 Flash	9.57 s (tot: 55.84 s)	1567	1698	3265	0.24	Sulla base dell'analisi della topologia dei pod e delle metriche istantanee correnti: ### 1. Identificazione delle Applicazioni Analizzando il numero di repliche attive per ciascuna applicazione: * **cache-app**: 1 replica (`cache-app-5c6fd4f5cf-xxsz4`) * **demo-app**: 2 repliche (`demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`) * **worker-app**: 2 repliche (`worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`) Il maggior numero di repliche attive presenti nel cluster è **2**. Le applicazioni corrispondenti sono **`demo-app`** e **`worker-app`**. --- ### 2. Consumo di Working Set (Memoria) * **demo-app**: 14,86 MB (15.577.088 byte) * **worker-app**: 5,23 MB (5.484.544 byte) --- ### 3. Calcolo Aggregato $$\text{Consumo totale} = 14{,}86 \text{ MB} + 5{,}23 \text{ MB} = \mathbf{20{,}09 \text{ MB}}$$ *(pari a $20{,}0859375 \text{ MB}$ o $21.061.632 \text{ byte}$)* Il consumo aggregato di Working Set delle applicazioni con il maggior numero di repliche è pari a **20,09 MB**.
Claude Opus 5.5	8.18 s (tot: 54.46 s)	2707	743	3450	0.00	I due deployment con più repliche attive (2 pod ciascuno) usano in totale **circa 20,09 MB** di memoria. | Applicazione | Repliche attive | Memoria (byte) | Memoria (MB) | |---|---|---|---| | demo-app | 2 | 15.577.088 | 14,86 | | worker-app | 2 | 5.484.544 | 5,23 | | **Totale** | **4 pod** | **21.061.632** | **20,09** | cache-app è escluso perché ha un solo pod. **Come ho fatto il calcolo** - Ho preso il valore `memory_bytes` dallo snapshot, che sembra essere già sommato su tutti i pod di ogni app. Lo snapshot non dice se sia proprio il Working Set: l'ho dato per scontato perché di solito si misura così. - Ho convertito i byte in MB dividendo per 1.048.576, cioè 21.061.632 / 1.048.576 ≈ 20,09 MB. **Cosa si nota** - **demo-app sembra inattivo:** CPU a 0 e nessun traffico di rete in entrata. Su questo istante non serve tenere 2 repliche, quindi si potrebbe scendere a 1. Prima conviene però guardare lo storico con `get_historical_metrics`, per escludere che fosse solo un momento di calma. - **worker-app consuma soprattutto CPU:** circa 0,59 core, contro una memoria molto bassa. Se serve uno scaling automatico (HPA), conviene basarlo sulla CPU e non sulla memoria.
Claude Sonnet 5.5	14.14 s (tot: 60.41 s)	5875	1299	7174	0.00	**Il consumo aggregato di Working Set è circa 20,2 MB (MiB).** Le applicazioni con più repliche attive sono `demo-app` e `worker-app`, con 2 pod ciascuna. `cache-app` ha 1 sola replica e non rientra nel calcolo. Ho interrogato Prometheus (`container_memory_working_set_bytes`) per ottenere il valore di ogni singolo pod: | App | Pod | Working Set (byte) | |---|---|---| | demo-app | demo-app-7958c64697-2qg9s | 7.819.264 | | demo-app | demo-app-7958c64697-4rzjl | 7.741.440 | | worker-app | worker-app-84cf4dd446-9kj75 | 3.796.992 | | worker-app | worker-app-84cf4dd446-ftlln | 1.802.240 | - **demo-app:** 15.560.704 byte, cioè circa 14,84 MB. - **worker-app:** 5.599.232 byte, cioè circa 5,34 MB. - **Totale:** 21.159.936 byte, cioè circa **20,18 MB**. In MB decimali (10⁶ byte) sarebbero circa 21,16 MB. Ho usato i MB in base 1024², la stessa convenzione dello snapshot. Lo snapshot dà un totale quasi identico, 14,86 + 5,23 = 20,09 MB. La piccola differenza dipende dal momento in cui è stato campionato il dato.

						Osservazioni: - **worker-app:** i due pod hanno un working set molto diverso (3,8 MB contro 1,8 MB). Questo suggerisce che il carico non sia distribuito in modo uniforme. - **worker-app, CPU:** l'app usa circa 0,59 core in totale, quindi è l'unica con un carico CPU significativo. Vale la pena verificare se serve un HPA. - **demo-app:** non usa CPU, rete né disco (0,0 su tutte e tre). Con 2 repliche è probabilmente idle e si potrebbe ridurre a 1 replica.
GPT-4o Mini	4.35 s (tot: 50.63 s)	1254	342	1596	0.00	Attualmente, le applicazioni e i loro pod attivi sono: - **cache-app**: 1 replica - **demo-app**: 2 repliche - **worker-app**: 2 repliche Ora calcoliamo il consumo aggregato di Working Set (memoria) per ciascuna applicazione: 1. **cache-app**: - Memoria per replica: 2.21 MB - Totale: \(2.21 \, \text{MB} \times 1 = 2.21 \, \text{MB} \) 2. **demo-app**: - Memoria per replica: 14.86 MB - Totale: \(14.86 \, \text{MB} \times 2 = 29.72 \, \text{MB} \) 3. **worker-app**: - Memoria per replica: 5.23 MB - Totale: \(5.23 \, \text{MB} \times 2 = 10.46 \, \text{MB} \) Ora sommiamo i consumi aggregati delle applicazioni con il maggior numero di repliche attive (demo-app e worker-app): - **Totale aggregato**: \(29.72 \, \text{MB} + 10.46 \, \text{MB} = 40.18 \, \text{MB} \) Quindi, il consumo aggregato di Working Set espresso in MB per le applicazioni con il maggior numero di repliche attive è **40.18 MB**.

GPT-4o	2.50 s (tot: 48.78 s)	1254	294	1548	0.63	Per determinare il consumo aggregato di Working Set in MB delle applicazioni con il maggior numero di repliche attive, dobbiamo prima identificare quale applicazione ha il maggior numero di repliche e poi sommare il consumo di memoria di tutti i suoi pod. Dalla topologia fornita: - "cache-app" ha 1 pod attivo. - "demo-app" ha 2 pod attivi. - "worker-app" ha 2 pod attivi. Sia "demo-app" che "worker-app" hanno il maggior numero di repliche attive, con 2 pod ciascuna. Ora, calcoliamo il consumo aggregato di memoria per ciascuna di queste applicazioni: - **demo-app**: 2 pod con un consumo totale di 14.86 MB (dato che il consumo è già aggregato per tutti i pod). - **worker-app**: 2 pod con un consumo totale di 5.23 MB (dato che il consumo è già aggregato per tutti i pod). Quindi, il consumo aggregato di Working Set in MB per le applicazioni con il maggior numero di repliche attive è: - **demo-app**: 14.86 MB - **worker-app**: 5.23 MB Se l'operatore desidera sapere quale delle due applicazioni ha il consumo maggiore, è "demo-app" con 14.86 MB.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2729 (mini: $0.0553 | 4o: $1.2176)

DOMANDA 3

```
================================================================================
ESECUZIONE DOMANDA [3]: Quale applicazione genera un flusso attivo di scrittura su disco e a quale rate scrive?
--------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	9.98 ms	0	0	0	N/A	cache-app (537294.58 B/s)
Gemini 2.5 Flash	2.08 s (tot: 48.27 s)	1540	109	1649	0.50	L'applicazione `cache-app` genera un flusso attivo di scrittura su disco con un rate di 537294.58 Bps.
Gemini 3.6 Flash	9.93 s (tot: 56.21 s)	1554	763	2317	0.80	L'applicazione che genera un flusso attivo di scrittura su disco è **`cache-app`**. ### Dettaglio del rate di scrittura: * **Rate di scrittura su disco (`disk_write_bps`):** `537.294,58` B/s * pari a **~537,29 KB/s** (ovvero **~0,51 MiB/s** o ~524,70 KiB/s) Le altre applicazioni (`demo-app` e `worker-app`) registrano attualmente un rate di scrittura su disco pari a **0,0 B/s**.

Claude Opus 5.5	13.82 s (tot: 60.10 s)	2684	1319	4003	0.95	L'unica applicazione che scrive su disco è **cache-app**, a circa **537 KB/s** (circa 525 KiB/s). | Applicazione | Pod | disk_write_bps | Stato | |---|---|---|---| | **cache-app** | `cache-app-5c6fd4f5cf-xxsz4` | **537.294,58 B/s** | **Scrive su disco** | | demo-app | 2 pod | 0 B/s | Nessuna scrittura | | worker-app | 2 pod | 0 B/s | Nessuna scrittura | **Rate di scrittura di cache-app** - **Al secondo:** circa 537 KB/s, cioè circa 525 KiB/s o 0,51 MiB/s. - **All'ora, se il rate resta costante:** circa 1,93 GB (circa 1,8 GiB). - **Al giorno, se il rate resta costante:** circa 46 GB. **Cosa c'è di strano** 1. **Scrive senza ricevere traffico.** `network_rx_bps` è 0, quindi le scritture non sono spinte da richieste in arrivo. Possono essere scritture interne, come snapshot o persistenza della cache, log troppo dettagliati o file temporanei. 2. **Usa pochissime risorse.** Consuma 0,015 core di CPU e 2,21 MB di RAM. Il pod lavora poco ma scrive in modo continuo. Una cache che tiene i dati su disco invece che in memoria è un comportamento da verificare. 3. **Ha una sola replica.** Non c'è ridondanza, e un problema di spazio su disco renderebbe la cache del tutto non disponibile. **Cosa consiglio di fare** - **Controllare dove scrive.** Bisogna capire se i dati vanno su un `emptyDir` o sul filesystem del container, cioè sullo storage temporaneo del nodo, o su un volume persistente (PVC). Nel primo caso, con circa 46 GB al giorno, c'è il rischio che il kubelet rimuova il pod per spazio esaurito. Va impostato `resources.limits.ephemeral-storage`, oppure `sizeLimit` sull'`emptyDir`. - **Controllare la causa delle scritture.** Conviene verificare la configurazione della persistenza, ad esempio la frequenza degli snapshot o dei log di append se è una cache tipo Redis, e il livello di logging del container. - **Verificare se il rate è costante.** Questo valore è un'istantanea. Per capire se il flusso è continuo o arriva a picchi posso consultare lo storico con `get_historical_metrics` oppure una query Prometheus nel tempo su `container_fs_writes_bytes_total`. - **Scaling orizzontale (HPA):** al momento non serve, perché CPU e memoria sono quasi a zero. Il problema riguarda il disco, non la capacità di calcolo.

Claude Sonnet 5.5	6.45 s (tot: 52.73 s)	2684	581	3265	0.36	L'unica applicazione che scrive attivamente su disco è **cache-app**. - **Rate di scrittura:** `disk_write_bps` è circa 537.294 byte/s, cioè circa **525 KiB/s** o **0,51 MiB/s**. - **Pod coinvolti:** cache-app ha una sola replica, `cache-app-5c6fd4f5cf-xxsz4`. Il valore è quindi riferito a quel pod. - **Altre applicazioni:** `demo-app` e `worker-app` hanno `disk_write_bps` pari a 0 nello snapshot. **Considerazioni** - Il carico di cache-app è quasi tutto I/O. La CPU è bassa (circa 0,015 core), la memoria è di 2,2 MB e il traffico di rete in ingresso è 0. Un cache-app che scrive senza ricevere traffico può essere normale, ad esempio per persistenza, flush o snapshot su disco (tipo RDB/AOF). Può anche essere un'attività anomala, come logging eccessivo o un loop di scrittura. Lo snapshot non permette di distinguere i due casi. - Lo snapshot è un valore istantaneo. Non so se la scrittura sia costante, a picchi o in crescita. Posso controllare lo storico con `get_historical_metrics` o con una query Prometheus per verificarlo. - Non ci sono segnali di saturazione di CPU o memoria su cache-app. Lo scaling orizzontale non è necessario, perché un'unica replica gestisce il carico. Se la scrittura fosse persistente, conviene verificare che il volume abbia spazio e IOPS sufficienti.
GPT-4o Mini	2.32 s (tot: 48.60 s)	1243	102	1345	0.50	Dall'analisi delle metriche istantanee correnti, l'unica applicazione che genera un flusso attivo di scrittura su disco è la **cache-app**, con una velocità di scrittura di **537294.58 bps** (byte per secondo). Le altre applicazioni, **demo-app** e **worker-app**, non mostrano alcuna attività di scrittura su disco, con un valore di `disk_write_bps` pari a 0.
GPT-4o	1.74 s (tot: 48.02 s)	1243	42	1285	0.50	L'applicazione che genera un flusso attivo di scrittura su disco è la "cache-app", con un rate di scrittura di circa 537,294.58 byte per secondo.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2766 (mini: $0.0556 | 4o: $1.2211)

DOMANDA 3 A

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [3A]: Rileva quale carico di lavoro sta eseguendo operazioni continue di I/O in scrittura su filesystem e specifica la velocità di scrittura registrata.
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	6.11 ms	0	0	0	N/A	cache-app (537294.58 B/s)
Gemini 2.5 Flash	2.20 s (tot: 48.47 s)	1551	231	1782	0.50	L'applicazione `cache-app` sta eseguendo operazioni continue di I/O in scrittura su filesystem. La velocità di scrittura registrata è di 537294.58 Bps.
Gemini 3.6 Flash	14.09 s (tot: 60.36 s)	1565	986	2551	0.95	In base alle metriche istantanee correnti fornite: * **Carico di lavoro identificato:** `cache-app` (Pod: `cache-app-5c6fd4f5cf-xxsz4`) * **Velocità di scrittura su filesystem (`disk_write_bps`):** **537.294,58 B/s** (pari a circa **537,29 KB/s** o **0,51 MB/s**) ### Analisi degli altri carichi di lavoro: * **`demo-app`:** `disk_write_bps` pari a **0,0 B/s** (inattivo dal punto di vista dell'I/O su disco). * **`worker-app`:** `disk_write_bps` pari a **0,0 B/s** (sebbene registri un utilizzo di CPU pari a ~0,59 core, non effettua operazioni di scrittura su disco rilevate). ### Raccomandazione SRE: Se l'applicazione `cache-app` non è destinata a persistenza intensiva o logging continuo su disco, si consiglia di verificare la configurazione del logging, la gestione della memoria/swap o l'eventuale presenza di dump/flush frequenti su filesystem locale per evitare il degrado prematuro dello storage o saturazione dei volumi temporanei.
Claude Opus 5.5	15.27 s (tot: 61.55 s)	2707	1301	4008	0.95	# Chi scrive su disco: **cache-app** **cache-app** (pod `cache-app-5c6fd4f5cf-xxsz4`, 1 replica) è l'unico carico che scrive su filesystem in questo campione. | Metrica | Valore | |---|---| | **Velocità di scrittura** | **537.294,58 byte/s** | | Equivalente | circa 524,7 KiB/s, cioè circa 0,51 MiB/s (0,54 MB/s) | | Proiezione su 1 ora | circa 1,93 GB (circa 1,80 GiB) | | Proiezione su 24 ore | circa 46 GB | Gli altri carichi non scrivono nulla: - **demo-app**: 0 B/s

| Claude Opus 5.5 | 15.27 s (tot: 61.55 s) | 2707 | 1301 | 4008 | 0.95 | # Chi scrive su disco: **cache-app**

cache-app (pod `cache-app-5c6fd4f5cf-xxsz4`, 1 replica) è l'unico carico che scrive su filesystem in questo campione.

\| Metrica \| Valore \|
\|---\|---\|
\| **Velocità di scrittura** \| **537.294,58 byte/s** \|
\| Equivalente \| circa 524,7 KiB/s, cioè circa 0,51 MiB/s (0,54 MB/s) \|
\| Proiezione su 1 ora \| circa 1,93 GB (circa 1,80 GiB) \|
\| Proiezione su 24 ore \| circa 46 GB \|

Gli altri carichi non scrivono nulla:
- **demo-app**: 0 B/s
- **worker-app**: 0 B/s (il carico di worker-app è sulla CPU, circa 0,59 core, non sul disco)

Cosa si nota
- **Il profilo è insolito per una cache.** cache-app usa pochissime risorse (circa 0,015 core di CPU e 2,21 MB di RAM), eppure scrive mezzo MB al secondo. Una cache di solito lavora in memoria: questa scrittura fa pensare a log molto verbosi, salvataggi su disco o un file che continua a crescere.
- **Non posso confermare che sia "continua".** Ho una sola misura istantanea, non una serie nel tempo. Le proiezioni orarie e giornaliere valgono solo se la velocità resta costante.

Raccomandazioni
1. **Controllare il tipo di volume.** Se scrive su `emptyDir` o sul filesystem del container, lo spazio occupato conta come storage temporaneo del nodo. A questi ritmi il pod rischia di essere sfrattato (*eviction*) per `DiskPressure`. In quel caso conviene impostare `resources.requests/limits.ephemeral-storage`, oppure spostare i dati su un PVC dedicato.
2. **Verificare cosa viene scritto.** Se sono log, conviene abbassare il livello di dettaglio o mandarli su stdout con rotazione. Se sono salvataggi della cache, conviene rivedere ogni quanto vengono fatti.
3. **Aumentare le repliche non serve per questo problema.** CPU e memoria sono quasi a zero. Attenzione però: con una sola replica, se il pod viene sfrattato il servizio si ferma del tutto.

Posso verificare se la scrittura è davvero costante: interrogo Prometheus sulle ultime ore (`rate(container_fs_writes_bytes_total{pod=~"cache-app.*"}[5m])`) oppure lo storico su MongoDB con `get_historical_metrics`. Vuoi che lo faccia? |

Claude Sonnet 5.5	6.82 s (tot: 53.10 s)	2707	644	3351	0.38	Il carico di lavoro che esegue scritture continue su filesystem è **cache-app**. Ho risposto direttamente dallo snapshot, senza invocare tool. | Workload | disk_write_bps | |---|---| | **cache-app** | **≈ 537.295 byte/s (~0,51 MiB/s, ~525 KiB/s)** | | demo-app | 0 | | worker-app | 0 | **Osservazioni** - cache-app ha un solo pod, `cache-app-5c6fd4f5cf-xxsz4`. È l'unico con scritture su disco diverse da zero. - Gli altri carichi non scrivono su disco. worker-app è il più attivo sulla CPU (0,59 core), ma non fa I/O di scrittura. - La CPU di cache-app è bassa (~0,015 core) e la memoria è di soli 2,21 MB. Network RX è 0. - Il quadro è insolito per una cache: scrive in modo costante su disco, ma non riceve traffico di rete. Può essere persistenza (AOF/snapshot RDB), logging verboso o flush periodici. **Raccomandazioni** 1. Controlla la configurazione di persistenza di cache-app e i suoi log. Verifica se le scritture sono volute. 2. Se la cache non deve essere persistente, disattiva AOF/RDB o sposta i dati su `emptyDir` con `medium: Memory`. Questo evita usura e I/O inutile. 3. Se la persistenza serve, controlla che il volume (PVC) abbia IOPS e spazio adeguati. A questa velocità si scrivono circa 44 GB al giorno, se il ritmo resta costante. 4. Il valore è un'istantanea. Per capire se il ritmo è costante o a picchi, posso analizzare lo storico con `get_historical_metrics`.
GPT-4o Mini	2.02 s (tot: 48.29 s)	1255	95	1350	0.70	L'unica applicazione che sta eseguendo operazioni continue di I/O in scrittura su filesystem è la **cache-app**, con una velocità di scrittura registrata di **537294.58 B/s** (circa 537.29 KB/s). Le altre applicazioni, **demo-app** e **worker-app**, non mostrano attività di scrittura su filesystem, con valori di `disk_write_bps` pari a 0.
GPT-4o	1.80 s (tot: 48.08 s)	1255	62	1317	0.70	Il carico di lavoro che sta eseguendo operazioni continue di I/O in scrittura su filesystem è il "cache-app". La velocità di scrittura registrata per questo carico di lavoro è di 537294.58 byte per secondo (circa 537.29 KB/s).

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2806 (mini: $0.0558 | 4o: $1.2249)

DOMANDA 3 B

ESECUZIONE DOMANDA [3B]: C'è un'applicazione che presenta throughput di scrittura su disco diverso da zero? Se presente, qual è e a che frequenza/tasso (rate) trasferisce dati verso lo storage?

Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	2.29 ms	0	0	0	N/A	cache-app (537294.58 B/s)
Gemini 2.5 Flash	1.63 s (tot: 47.91 s)	1562	133	1695	0.50	Sì, l'applicazione `cache-app` presenta un throughput di scrittura su disco diverso da zero. Il tasso di trasferimento dati verso lo storage è di `537294.58 Bps`.
Gemini 3.6 Flash	4.59 s (tot: 50.87 s)	1576	691	2267	0.80	Sì, basandosi sulle metriche istantanee correnti, l'applicazione **`cache-app`** presenta un throughput di scrittura su disco diverso da zero. ### Dettagli del throughput di scrittura su disco: * **Applicazione:** `cache-app` (Pod attivo: `cache-app-5c6fd4f5cf-xxsz4`) * **Tasso di scrittura (`disk_write_bps`):** ~**537.294,58 byte/s** (ovvero circa **524,7 KiB/s** / **0,51 MiB/s**). Le altre applicazioni (`demo-app` e `worker-app`) registrano attualmente un valore pari a `0.0` B/s per la scrittura su disco.
Claude Opus 5.5	14.42 s (tot: 60.70 s)	2718	1302	4020	0.95	# Throughput di scrittura su disco: snapshot attuale **Sì, una sola applicazione scrive su disco: `cache-app`.** | Applicazione | Pod | disk_write_bps | |---|---|---| | **cache-app** | `cache-app-5c6fd4f5cf-xxsz4` (1 replica) | **537.294,58 B/s** | | demo-app | 2 pod | 0 B/s | | worker-app | 2 pod | 0 B/s | ## Quanto scrive `cache-app` | Unità | Valore | |---|---| | Byte/secondo | ~537.295 B/s | | KB/s (decimale) / KiB/s (binario) | ~537,3 KB/s / ~524,7 KiB/s | | MB/s / MiB/s | ~0,54 MB/s / ~0,51 MiB/s | | Al minuto | ~32,2 MB (~30,7 MiB) | | All'ora | ~1,93 GB (~1,80 GiB) | | Al giorno (se il ritmo resta costante) | ~46,4 GB (~43,2 GiB) | ## Osservazioni

Claude Opus 5.5	14.42 s (tot: 60.70 s)	2718	1302	4020	0.95	# Throughput di scrittura su disco: snapshot attuale **Sì, una sola applicazione scrive su disco: `cache-app`.** | Applicazione | Pod | disk_write_bps | |---|---|---| | **cache-app** | `cache-app-5c6fd4f5cf-xxsz4` (1 replica) | **537.294,58 B/s** | | demo-app | 2 pod | 0 B/s | | worker-app | 2 pod | 0 B/s | ## Quanto scrive `cache-app` | Unità | Valore | |---|---| | Byte/secondo | ~537.295 B/s | | KB/s (decimale) / KiB/s (binario) | ~537,3 KB/s / ~524,7 KiB/s | | MB/s / MiB/s | ~0,54 MB/s / ~0,51 MiB/s | | Al minuto | ~32,2 MB (~30,7 MiB) | | All'ora | ~1,93 GB (~1,80 GiB) | | Al giorno (se il ritmo resta costante) | ~46,4 GB (~43,2 GiB) | ## Osservazioni - **Il carico è insolito per il tipo di app.** `cache-app` usa poche risorse (CPU ~0,015 core, memoria ~2,2 MB), ma scrive di continuo circa mezzo MB/s. Una cache che tiene poco in memoria e scrive molto potrebbe stare salvando i dati su disco (snapshot, append log), scrivendo log troppo dettagliati o usando il disco come spazio temporaneo. - **Rischio storage.** Se le scritture finiscono su un volume con dimensione fissa o sul disco temporaneo del nodo (emptyDir/ephemeral), circa 43 GiB al giorno potrebbero riempirlo. Il pod verrebbe allora espulso (eviction) o il volume risulterebbe pieno. Conviene controllare dove scrive e se ha un limite di `ephemeral-storage`. - **Nessun traffico di rete in ingresso (0 B/s).** I dati scritti non arrivano quindi da richieste esterne: il carico parte dall'app stessa. - **Singola replica.** Non ci sono altri pod che proteggano dai problemi di disco: se questo pod viene espulso, il servizio si ferma. ## Limiti del dato Il valore viene da una sola misurazione. Le stime orarie e giornaliere valgono solo se il ritmo resta costante. Per capire se le scritture sono stabili, a picchi o in crescita, posso recuperare lo storico di `cache-app` dalle metriche salvate (per esempio ultime 6–24 ore) o da Prometheus.

Claude Sonnet 5.5	5.43 s (tot: 51.71 s)	2718	488	3206	0.27	Sì, c'è un'applicazione con scrittura su disco diversa da zero: **cache-app**. - **Throughput di scrittura:** circa **537.295 byte/s**, cioè circa **0,51 MiB/s** (circa 0,54 MB/s). - **Pod coinvolto:** `cache-app-5c6fd4f5cf-xxsz4`. È l'unica replica del deployment, quindi il valore è tutto suo. - **Altre applicazioni:** `demo-app` e `worker-app` hanno `disk_write_bps` pari a 0. **Contesto sulle risorse di cache-app** - La CPU è bassa, circa 0,015 core. - La memoria è molto contenuta, circa 2,2 MB. - Il traffico di rete in ingresso è 0. Una cache che scrive circa 0,5 MiB/s senza traffico in ingresso e con così poca memoria può essere normale se fa persistenza, ad esempio snapshot, AOF o flush su volume. Può però anche indicare attività interna anomala, come logging eccessivo o rewrite. Lo snapshot è un valore istantaneo e non dice se la scrittura sia costante o a picchi. Posso recuperare lo storico di cache-app (`get_historical_metrics`) o una serie temporale da Prometheus. Così verifichiamo se il tasso è stabile, periodico o in crescita.
GPT-4o Mini	1.80 s (tot: 48.07 s)	1260	104	1364	0.50	Dall'analisi delle metriche istantanee correnti, solo l'applicazione "cache-app" presenta un throughput di scrittura su disco diverso da zero. Il tasso di scrittura su disco per "cache-app" è di **537294.58 bps** (byte per secondo). Le altre applicazioni, "demo-app" e "worker-app", non mostrano alcun throughput di scrittura su disco, con un valore di **0.0 bps**.
GPT-4o	1.52 s (tot: 47.79 s)	1260	52	1312	0.50	Sì, l'applicazione "cache-app" presenta un throughput di scrittura su disco diverso da zero. Il tasso di trasferimento dei dati verso lo storage per "cache-app" è di circa 537294.58 byte per secondo.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2846 (mini: $0.0561 | 4o: $1.2285)

DOMANDA 4

```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [4]: Identifica l'applicazione con il maggior numero di repliche: elenca i relativi pod e indica quante repliche risultano attive.
--------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling pa
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	5.51 ms	0	0	0	N/A	App: demo-app | Repliche: 2 | Pods: ['demo-app-7958c64697-2qg9s', 'demo-app-7958c64697-4rzjl']
Gemini 2.5 Flash	2.20 s (tot: 48.48 s)	1551	214	1765	0.70	L'applicazione con il maggior numero di repliche è "demo-app" con 2 repliche attive. I pod relativi sono: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`.
Gemini 3.6 Flash	6.55 s (tot: 52.82 s)	1565	975	2540	0.32	Dall'analisi della topologia dei pod attualmente fornita nell'istantanea, le applicazioni con il maggior numero di repliche sono **`demo-app`** e **`worker-app`**, entrambe con **2 repliche attive**. Ecco il dettaglio per ciascuna delle due applicazioni: ### 1. `demo-app` * **Numero di repliche attive:** 2 * **Elenco dei Pod:** * `demo-app-7958c64697-2qg9s` * `demo-app-7958c64697-4rzjl` ### 2. `worker-app` * **Numero di repliche attive:** 2 * **Elenco dei Pod:** * `worker-app-84cf4dd446-9kj75` * `worker-app-84cf4dd446-ftlln` --- *(Nota di confronto: l'applicazione `cache-app` dispone invece di 1 sola replica attiva: `cache-app-5c6fd4f5cf-xxsz4`).*
Claude Opus 5.5	8.55 s (tot: 54.83 s)	2695	659	3354	0.63	**Sono a pari merito due applicazioni, ognuna con 2 repliche attive.** Nessuna delle due ha più repliche dell'altra. | Applicazione | Repliche attive | Pod | |---|---|---| | **demo-app** | **2** | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | **2** | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | **Cosa indicano le metriche attuali:**

| Gemini 3.6 Flash | 6.55 s (tot: 52.82 s) | 1565 | 975 | 2540 | 0.32 | Dall'analisi della topologia dei pod attualmente fornita nell'istantanea, le applicazioni con il maggior numero di repliche sono **`demo-app`** e **`worker-app`**, entrambe con **2 repliche attive**.

Ecco il dettaglio per ciascuna delle due applicazioni:

1. `demo-app`
* **Numero di repliche attive:** 2
* **Elenco dei Pod:**
  * `demo-app-7958c64697-2qg9s`
  * `demo-app-7958c64697-4rzjl`

2. `worker-app`
* **Numero di repliche attive:** 2
* **Elenco dei Pod:**
  * `worker-app-84cf4dd446-9kj75`
  * `worker-app-84cf4dd446-ftlln`

(Nota di confronto: l'applicazione `cache-app` dispone invece di 1 sola replica attiva: `cache-app-5c6fd4f5cf-xxsz4`). |
| Claude Opus 5.5 | 8.55 s (tot: 54.83 s) | 2695 | 659 | 3354 | 0.63 | **Sono a pari merito due applicazioni, ognuna con 2 repliche attive.** Nessuna delle due ha più repliche dell'altra.

\| Applicazione \| Repliche attive \| Pod \|
\|---\|---\|---\|
\| **demo-app** \| **2** \| `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` \|
\| **worker-app** \| **2** \| `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` \|
\| cache-app \| 1 \| `cache-app-5c6fd4f5cf-xxsz4` \|

Cosa indicano le metriche attuali:

- **demo-app:** l'uso di CPU, scrittura su disco e traffico di rete in entrata è a 0, e la memoria è a 14,86 MB. Non sta lavorando, ma tiene comunque occupate 2 repliche. Se il valore resta a 0 anche nello storico, si può passare a 1 replica. Va lasciata a 2 solo se serve per l'alta disponibilità.
- **worker-app:** usa circa 0,59 core di CPU, su entrambi i pod insieme. È di gran lunga l'applicazione che consuma più CPU, mentre la memoria è bassa (5,23 MB). Due repliche sono adeguate. Per decidere se aggiungere uno scaling automatico (HPA) basato sulla CPU, bisogna confrontare questo valore con i `requests` di CPU dei pod.
- **cache-app:** ha 1 sola replica, quindi un solo punto di guasto. Scrive su disco in modo continuo, circa 525 KB/s.

Se vuoi, posso controllare lo storico di demo-app per confermare che sia inattiva da tempo prima di ridurre le repliche. |

						che sia inattiva da tempo prima di ridurre le repliche.
Claude Sonnet 5.5	6.30 s (tot: 52.57 s)	2695	688	3383	0.54	Dalla topologia corrente, due applicazioni hanno più repliche delle altre: demo-app e worker-app, con 2 repliche ciascuna. Quindi non c'è un'unica applicazione con il maggior numero di repliche. cache-app ne ha 1. | Applicazione | Repliche attive | Pod | |---|---|---| | **demo-app** | 2 | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | 2 | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | Il conteggio è quello dei pod elencati nella mappa Deployment→Pod. Non ho verificato se tutti siano in stato Ready. **Osservazioni dalle metriche istantanee** - **demo-app**: CPU a 0 core, nessun traffico di rete e nessuna scrittura su disco. Ha 14,86 MB di memoria. Sembra idle. Con due repliche si può valutare di scendere a 1, dopo aver controllato lo storico per escludere picchi periodici. - **worker-app**: è il carico più attivo, con 0,59 core di CPU in totale, cioè circa 0,3 core per replica se il carico è bilanciato. La memoria è bassa (5,23 MB). Le due repliche sembrano giustificate. Se la CPU cresce, si può valutare un HPA basato sulla CPU. - **cache-app**: ha una sola replica, con CPU quasi nulla (0,015 core) e circa 0,5 MB/s di scritture su disco. Non c'è traffico di rete in ingresso. Nel caso sia un componente stateful o critico, una sola replica è un single point of failure. Posso recuperare lo storico di demo-app e worker-app per confermare questi trend, se vuoi.
GPT-4o Mini	2.05 s (tot: 48.33 s)	1251	86	1337	0.70	L'applicazione con il maggior numero di repliche è la **demo-app**, che ha attualmente 2 repliche attive. I pod relativi a questa applicazione sono: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl In sintesi, la **demo-app** ha 2 repliche attive.
GPT-4o	2.03 s (tot: 48.30 s)	1251	68	1319	0.70	L'applicazione con il maggior numero di repliche attive è la "demo-app", che ha 2 repliche attive. I pod relativi a questa applicazione sono: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl

Domanda 4 B

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [4B]: Quale servizio registra la presenza del pool di pod più numeroso? Fornisci l'elenco dei singoli identificatori dei pod e il totale delle istanze operative.
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperat
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\W\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$
[10/30/26 10:09:29] INFO     Processing request of type CallToolRequest                                                                                                server.py:733
[10/30/26 10:09:30] INFO     Processing request of type ListToolsRequest                                                                                               server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperat
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.04 ms	0	0	0	N/A	App: demo-app | Repliche: 2 | Pods: ['demo-app-7958c64697-2qg9s', 'demo-app-7958c64697-4rzjl']
Gemini 2.5 Flash	1.56 s (tot: 47.83 s)	1557	153	1710	0.70	Il servizio con il pool di pod più numeroso è **demo-app**. Elenco dei pod: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` Totale istanze operative: 2
Gemini 3.6 Flash	12.15 s (tot: 58.43 s)	3282	1026	4308	0.21	I servizi che registrano il pool di pod più numeroso nel cluster sono **'demo-app'** e **'worker-app'** a pari merito, ciascuno con un totale di **2 istanze operative**. Di seguito il dettaglio per ciascun servizio: ### 1. `demo-app` * **Totale istanze operative:** 2 * **Identificatori dei Pod:** * `demo-app-7958c64697-2qg9s` * `demo-app-7958c64697-4rzjl` --- ### 2. `worker-app` * **Totale istanze operative:** 2 * **Identificatori dei Pod:** * `worker-app-84cf4dd446-9kj75` * `worker-app-84cf4dd446-ftlln` --- *(Per confronto, `cache-app` dispone di un'unica istanza operativa: `cache-app-5c6fd4f5cf-xxsz4`).*

Claude Opus 5.5	8.27 s (tot: 54.55 s)	2709	750	3459	0.00	**Non c'è un solo servizio con il pool più numeroso: demo-app e worker-app sono a pari merito con 2 pod ciascuno.** ### Servizi con il pool più numeroso | Servizio | Pod | Repliche | |---|---|---| | **demo-app** | `demo-app-7958c64697-2qg9s` `demo-app-7958c64697-4rzjl` | 2 | | **worker-app** | `worker-app-84cf4dd446-9kj75` `worker-app-84cf4dd446-ftlln` | 2 | Per confronto, **cache-app** ha un solo pod: `cache-app-5c6fd4f5cf-xxsz4`. ### Totale istanze attive - **Nei due servizi più numerosi:** 4 pod (2 + 2) - **In tutto il cluster:** 5 pod (2 + 2 + 1) ### Osservazioni SRE (dalle metriche attuali) - **demo-app:** le due repliche hanno CPU a 0, nessun traffico di rete e nessuna scrittura su disco. In questo momento sembra **inattiva**. Prima di ridurla a 1 replica conviene guardare l'andamento storico, per non scambiare un momento di calma per inattività vera. - **worker-app:** usa circa 0,59 core in tutto, cioè circa 0,30 core per pod. È il servizio con più carico di calcolo. Due repliche hanno senso. Se la CPU cresce, conviene valutare un autoscaler orizzontale (HPA) basato sull'uso della CPU. - **cache-app:** ha **una sola replica**, quindi se quel pod si ferma il servizio si ferma (single point of failure). Inoltre scrive su disco in modo costante, circa 525 KB/s. Per avere ridondanza conviene passare ad almeno 2 repliche. Se vuoi, posso recuperare lo storico da MongoDB per verificare se demo-app è inattiva anche su un periodo più lungo.

Claude Sonnet 5.5	5.51 s (tot: 51.78 s)	2709	595	3304	0.00	Dalla topologia che mi hai fornito, ci sono due servizi con il pool più numeroso. **demo-app** e **worker-app** hanno 2 pod ciascuno, quindi non c'è un servizio con più pod degli altri. cache-app ne ha 1. **demo-app (2 pod)** - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl **worker-app (2 pod)** - worker-app-84cf4dd446-9kj75 - worker-app-84cf4dd446-ftlln **cache-app (1 pod, per confronto)** - cache-app-5c6fd4f5cf-xxsz4 **Totale istanze:** 5 pod in tutto (2 + 2 + 1). Dalle metriche istantanee si vede che i due pool hanno carichi molto diversi: - **worker-app** usa circa 0,59 core di CPU e 5,23 MB di memoria. È il servizio più attivo e il candidato naturale a un HPA se il carico cresce. Le metriche non indicano saturazione. - **demo-app** ha CPU a 0, nessun traffico di rete in ingresso e nessuna scrittura su disco. Sembra idle, quindi potresti ridurlo a 1 replica. Verifica prima lo storico per escludere che sia solo un momento di calma. - **cache-app** ha un solo pod, con CPU quasi nulla (0,015 core) e una scrittura su disco di circa 0,5 MB/s. Lo snapshot non dice se le metriche siano aggregate o per singolo pod. Posso controllare lo storico con `get_historical_metrics` se vuoi verificare l'idle di demo-app.
GPT-4o Mini	1.72 s (tot: 47.99 s)	1255	85	1340	0.70	Il servizio che registra la presenza del pool di pod più numeroso è il **demo-app**, che ha attualmente 2 pod attivi. Ecco l'elenco dei singoli identificatori dei pod: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl Totale delle istanze operative: **2**.
GPT-4o	1.39 s (tot: 47.67 s)	1255	86	1341	0.70	Il servizio con il pool di pod più numeroso è il "demo-app", che ha attualmente 2 pod attivi. Gli identificatori dei pod per "demo-app" sono: 1. demo-app-7958c64697-2qg9s 2. demo-app-7958c64697-4rzjl Il totale delle istanze operative per "demo-app" è 2.

DOMANDA 4 A

```
================================================================================================
ESECUZIONE DOMANDA [4A]: Determina qual è l'applicazione con il livello di scalabilità più alto: riporta il conteggio delle repliche in esecuzione e i nomi puntuali di ciascun pod associato.
------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3988: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 10:10:26] INFO     Processing request of type CallToolRequest                                                
```

GPT-4o Mini	5.53 s (tot: 51.81 s)	1262	244	1506	0.80	Dall'analisi delle metriche e della topologia attuale, possiamo osservare quanto segue: 1. **cache-app**: - Repliche in esecuzione: 1 - Nome del pod: `cache-app-5c6fd4f5cf-xxsz4` 2. **demo-app**: - Repliche in esecuzione: 2 - Nomi dei pod: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` 3. **worker-app**: - Repliche in esecuzione: 2 - Nomi dei pod: `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` L'applicazione con il livello di scalabilità più alto è **demo-app**, con 2 repliche in esecuzione. I nomi dei pod associati sono: - `demo-app-7958c64697-2qg9s` - `demo-app-7958c64697-4rzjl`
GPT-4o	2.68 s (tot: 48.96 s)	1262	201	1463	0.80	L'applicazione con il livello di scalabilità più alto, in termini di numero di repliche attualmente in esecuzione, è la "demo-app". Ecco i dettagli: - **demo-app**: 2 repliche - Pod: "demo-app-7958c64697-2qg9s", "demo-app-7958c64697-4rzjl" Le altre applicazioni hanno il seguente conteggio di repliche: - **cache-app**: 1 replica - Pod: "cache-app-5c6fd4f5cf-xxsz4" - **worker-app**: 2 repliche - Pod: "worker-app-84cf4dd446-9kj75", "worker-app-84cf4dd446-ftlln" Sia "demo-app" che "worker-app" hanno 2 repliche, ma "demo-app" è stata menzionata per prima.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.2984 (mini: $0.0569 | 4o: $1.2415)

DOMANDA 5

ESECUZIONE DOMANDA [5]: C'è qualche applicazione che presenta traffico di rete in ricezione superiore a 10 KB/s? Se sì, quale e a quale rate?

Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	4.72 ms	0	0	0	N/A	Nessuna applicazione supera 10 KB/s in ricezione
Gemini 2.5 Flash	2.85 s (tot: 49.13 s)	1555	264	1819	0.70	Nessuna delle applicazioni presenta traffico di rete in ricezione superiore a 10 KB/s. Tutte le applicazioni hanno un `network_rx_bps` pari a 0.0.
Gemini 3.6 Flash	5.83 s (tot: 52.11 s)	1569	846	2415	0.80	In base alle metriche istantanee correnti fornite, **nessuna applicazione** presenta traffico di rete in ricezione superiore a 10 KB/s. Di seguito il dettaglio del rate di ricezione di rete (`network_rx_bps`) attualmente rilevato per ciascuna applicazione: * **`cache-app`**: `0.0 B/s` (0.0 KB/s) * **`demo-app`**: `0.0 B/s` (0.0 KB/s) * **`worker-app`**: `0.0 B/s` (0.0 KB/s) Tutti i carichi di lavoro risultano al momento privi di traffico di rete in ingresso misurato.

Claude Opus 5.5	8.27 s (tot: 54.55 s)	2701	742	3443	0.95	# Nessuna applicazione supera i 10 KB/s di traffico di rete in ricezione
						
						Secondo lo snapshot attuale, tutte e tre le applicazioni hanno 0 B/s:
						
						| Applicazione | Pod | network_rx_bps | Sopra 10 KB/s? |
						|---|---|---|---|
						| cache-app | 1 | 0.0 B/s | No |
						| demo-app | 2 | 0.0 B/s | No |
						| worker-app | 2 | 0.0 B/s | No |
						
						## Attenzione: il valore zero potrebbe essere un errore di raccolta
						Un valore esattamente 0 per tutte le applicazioni è sospetto, soprattutto per i casi seguenti:
						- **worker-app** usa circa 0,59 core di CPU.
						- **cache-app** scrive su disco circa 525 KB/s.
						
						È difficile che queste applicazioni non ricevano nemmeno un byte dalla rete. La causa più comune è la query usata per raccogliere i dati. In Kubernetes, le metriche di rete `container_network_receive_bytes_total` vengono registrate a livello di pod, sul container "pause" (`container="POD"` oppure senza etichetta `container`). Se la query filtra per `container!=""` o `container!="POD"`, il risultato è sempre 0.
						
						## Come verificare
						Per controllare il dato reale, esegui una query Prometheus che non filtra per container:
						``promql
						sum by (pod)
						(rate(container_network_receive_bytes_total{namespace="default", pod=~"cache-app.*|demo-app.*|worker-app.*"}[5m]))
						``
						Se vuoi, posso eseguirla io per confermare se il traffico è davvero nullo o se è un problema di raccolta.

Claude Sonnet 5.5	5.65 s (tot: 51.92 s)	2701	516	3217	0.95	No, nessuna applicazione supera i 10 KB/s di traffico di rete in ricezione. Lo snapshot corrente riporta `network_rx_bps` a 0 per tutte e tre: | Applicazione | network_rx_bps | |---|---| | cache-app | 0,0 | | demo-app | 0,0 | | worker-app | 0,0 | Il valore è molto sotto la soglia di 10 KB/s (circa 10.240 B/s). - **worker-app** usa 0,59 core di CPU ma non riceve traffico di rete. Il carico sembra quindi un'elaborazione interna o batch, non legata a richieste in ingresso. - **cache-app** scrive su disco a circa 525 KB/s (537.294 B/s) e ha un rx di rete pari a 0. L'attività è quindi di I/O locale, non di rete. - **demo-app** è idle: CPU, disco e rete sono a 0. Con 2 repliche, potresti ridurle a 1 se questo comportamento è stabile. Prima conviene controllare lo storico. Questa risposta si basa solo sullo snapshot istantaneo. Posso controllare lo storico su MongoDB o interrogare Prometheus per vedere se ci sono stati picchi di rete in passato.
GPT-4o Mini	2.93 s (tot: 49.21 s)	1250	112	1362	0.70	Nello stato attuale delle metriche, nessuna delle applicazioni presenta traffico di rete in ricezione superiore a 10 KB/s. Ecco i dettagli del traffico di rete in ricezione per ciascuna applicazione: - **cache-app**: 0.0 bps - **demo-app**: 0.0 bps - **worker-app**: 0.0 bps Pertanto, non ci sono applicazioni con traffico di rete in ricezione superiore a 10 KB/s.
GPT-4o	2.79 s (tot: 49.06 s)	1250	119	1369	0.70	Dall'analisi delle metriche istantanee fornite, nessuna delle applicazioni presenta traffico di rete in ricezione (network_rx_bps) superiore a 10 KB/s. Ecco i dettagli: - **cache-app**: 0.0 B/s - **demo-app**: 0.0 B/s - **worker-app**: 0.0 B/s Tutte le applicazioni hanno un traffico di rete in ricezione pari a 0 B/s, quindi nessuna supera la soglia di 10 KB/s.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $1.3029 (mini: $0.0571 | 4o: $1.2458)
```

DOMANDA 5 A

```
================================================================================================
ESECUZIONE DOMANDA [5A]: Verifica se nei carichi di lavoro utente è presente un throughput di ingress (network RX) che eccede la soglia di 10 KB/s, indicando l'app interessata e il relativo valore.
================================================================================================
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 10:31:26] INFO     Processing request of type CallToolRequest                                                                                                                    server.py:733
[10/30/26 10:31:27] INFO     Processing request of type ListToolsRequest                                                                                                                   server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.06 ms	0	0	0	N/A	Nessuna applicazione supera 10 KB/s in ricezione
Gemini 2.5 Flash	2.48 s (tot: 48.76 s)	1567	349	1916	0.70	Dalle metriche istantanee correnti, nessuna delle applicazioni ('cache-app', 'demo-app', 'worker-app') presenta un throughput di ingress (network RX) che eccede la soglia di 10 KB/s (10240 B/s). Tutte le applicazioni mostrano un valore di `network_rx_bps` pari a 0.0.
Gemini 3.6 Flash	35.46 s (tot: 81.74 s)	10469	478	10947	0.00	[ERRORE STEP 2]: Error calling model 'gemini-3.6-flash' (RESOURCE_EXHAUSTED): 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, model: gemini-3.6-flash, description: Generate content requests per day limit exceeded on Free Tier. See https://ai.google.dev/gemini-api/docs/rate-limits.\nPlease retry in 21h28m11.802963145s.', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'description': 'Generate content requests per day limit exceeded on Free Tier. See https://ai.google.dev/gemini-api/docs/rate-limits.', 'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-3.6-flash'}, 'quotaValue': '20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay':

Claude Opus 5.5	10.86 s (tot: 57.14 s)	2717	975	3692	0.95	**No, nessuna applicazione supera la soglia di 10 KB/s di traffico in ingresso.** Ho usato solo lo snapshot che mi hai fornito, senza chiamare altri strumenti. | App | Pod | network_rx_bps | Sopra 10 KB/s (10.240 B/s)? | |---|---|---|---| | cache-app | 1 | 0,0 B/s | No | | demo-app | 2 | 0,0 B/s | No | | worker-app | 2 | 0,0 B/s | No | **Attenzione:** tutte e tre le app segnano esattamente 0,0 B/s, anche se cache-app e worker-app stanno lavorando (cache-app scrive circa 525 KB/s su disco, worker-app usa circa 0,59 core). Le app potrebbero davvero non ricevere traffico, ad esempio se sono job batch interni. Ma il valore potrebbe anche essere sbagliato perché la metrica viene raccolta male. Una causa comune: - la metrica `container_network_receive_bytes_total` di cAdvisor è calcolata per l'intero pod, non per il singolo container. Se la query filtra per `container="<nome>"`, spesso non trova nulla e il risultato diventa 0. **Cosa ti consiglio:** 1. Controlla direttamente su Prometheus con una query a livello di pod, ad esempio: `sum by (pod) (rate(container_network_receive_bytes_total{namespace="default", pod=~"cache-app.*|demo-app.*|worker-app.*"}[5m]))` Se vuoi, la eseguo io. 2. Se anche questa restituisce valori reali sotto i 10 KB/s, la conclusione resta valida. Se invece mostra valori più alti, bisogna correggere la query che produce lo snapshot. **Altre cose che saltano all'occhio nei dati:** - **demo-app sembra inattiva:** 2 repliche con 0 CPU, nessuna scrittura su disco e nessun traffico. Potresti ridurla a una replica. - **worker-app usa CPU in modo costante** (circa 0,3 core per pod). Conviene confrontare questo valore con i suoi limiti di CPU prima di valutare un autoscaler (HPA).

Claude Sonnet 5.5	5.56 s (tot: 51.84 s)	2717	525	3242	0.80	Nessun carico di lavoro supera la soglia di 10 KB/s di ingress (network RX). Ho risposto dallo snapshot corrente, senza invocare altri tool. | App | network_rx_bps | Supera 10 KB/s (≈10.240 B/s)? | |---|---|---| | cache-app | 0,0 B/s | No | | demo-app | 0,0 B/s | No | | worker-app | 0,0 B/s | No | Tutte e tre le app hanno un RX istantaneo pari a 0 B/s, quindi non c'è nessuna app da segnalare. - **worker-app:** usa 0,59 core di CPU ma non riceve traffico di rete. Il carico sembra quindi un'elaborazione interna o batch, non legata a richieste in ingresso. - **cache-app:** scrive su disco a circa 525 KB/s (537.294 B/s) con RX pari a 0. Può essere normale, per esempio se fa persistenza o flush, ma è l'unico carico di I/O visibile. - **demo-app:** è idle, con CPU a 0, RX a 0 e disco a 0. Ha 2 repliche e potrebbe essere ridotta a 1 se il profilo di carico resta questo nel tempo. Questo dato è un'istantanea. Un picco di ingress breve o passato non sarebbe visibile. Se vuoi, posso controllare lo storico su MongoDB o con una range query Prometheus per verificare se la soglia è stata superata in passato.
GPT-4o Mini	2.12 s (tot: 48.40 s)	1262	134	1396	0.70	Analizzando le metriche istantanee correnti, possiamo osservare il throughput di ingress (network RX) per ciascuna applicazione: - **cache-app**: 0.0 B/s - **demo-app**: 0.0 B/s - **worker-app**: 0.0 B/s Nessuna delle applicazioni ha un throughput di ingress che eccede la soglia di 10 KB/s. Tutte le applicazioni hanno un valore di network RX pari a 0.0 B/s. Pertanto, non ci sono applicazioni con un throughput di ingress superiore a 10 KB/s.
GPT-4o	1.72 s (tot: 48.00 s)	1262	122	1384	0.70	Dall'analisi delle metriche istantanee fornite, possiamo osservare i seguenti valori di throughput di ingress (network RX) per le applicazioni: - **cache-app**: 0.0 B/s - **demo-app**: 0.0 B/s - **worker-app**: 0.0 B/s Nessuna delle applicazioni attualmente supera la soglia di 10 KB/s (10,240 B/s) per il throughput di ingress. Pertanto, non ci sono applicazioni che eccedono la soglia specificata.

DOMANDA 5B

```
ESECUZIONE DOMANDA [5B]: Controlla il traffico di rete in entrata: qualche servizio sfora il limite nominale di 10 kilobyte al secondo? Se rilevato, quantifica il bitrate esatto.

Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 10:35:40] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 10:35:55] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 10:35:59] INFO     Processing request of type CallToolRequest                                                                                                    server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                   server.py:733
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.41 ms	0	0	0	N/A	Nessuna applicazione supera 10 KB/s in ricezione
Gemini 2.5 Flash	3.33 s (tot: 42.55 s)	1564	434	1998	0.50	Nessun servizio sta attualmente superando il limite nominale di 10 kilobyte al secondo per il traffico di rete in entrata. Tutte le applicazioni mostrano un `network_rx_bps` pari a 0.0.
Gemini 3.6 Flash	26.09 s (tot: 65.31 s)	45253	2040	47293	0.80	In base all'analisi del traffico di rete in entrata (`network_rx_bps`) ricavata dallo snapshot corrente, dalle verifiche puntuali su Prometheus e dallo storico persistito nel database: ### Esito del Controllo **Nessuno dei servizi applicativi** (`cache-app`, `demo-app`, `worker-app`) sfora il limite nominale di 10 kB/s (10.000 / 10.240 byte/s). ### Bitrate Rilevati per Servizio (Namespace `default`) 1. **`cache-app`**: **0,00 B/s** (0,00 kB/s) 2. **`demo-app`**: **0,00 B/s** (0,00 kB/s)
Claude Opus 5.5	13.05 s (tot: 52.27 s)	2715	1137	3852	0.90	# Nessun servizio supera i 10 KB/s in ingresso Nello snapshot attuale il traffico di rete in entrata è **0 B/s per tutte e tre le applicazioni**, quindi nessuna supera il limite. Non c'è un bitrate eccedente da misurare. | Applicazione | Pod | Traffico in entrata | Bitrate | Supera 10 kB/s (~80 kbit/s)? | |---|---|---|---|---| | cache-app | 1 | 0,0 B/s | 0 bit/s | ❌ No | | demo-app | 2 | 0,0 B/s | 0 bit/s | ❌ No | | worker-app | 2 | 0,0 B/s | 0 bit/s | ❌ No | Il limite di 10 kB/s corrisponde a 10.240 B/s, cioè circa 81,9 kbit/s (80 kbit/s se si intende 10.000 B/s). ## Il valore zero va verificato Uno zero esatto su tutti e cinque i pod è insolito: anche un servizio fermo riceve di solito un minimo di traffico, ad esempio dai controlli di salute o dalla raccolta delle metriche. Le spiegazioni possibili sono due: 1. **I servizi non ricevono davvero traffico.** È plausibile per demo-app, che è inattiva (0 core di CPU). 2. **Problema di raccolta.** In Kubernetes il traffico di rete viene spesso misurato a livello dell'intero pod e non del singolo container. Se la query filtra per nome del container, può restituire sempre zero. worker-app usa circa 0,52 core di CPU ma non riceve nulla dalla rete. È compatibile con un processo che lavora in autonomia, ma rende la seconda ipotesi da controllare. Se vuoi una conferma, posso interrogare direttamente Prometheus calcolando il traffico per pod senza filtrare per container. ## Altro valore sopra soglia (non di rete) cache-app **scrive su disco a circa 483 KB/s** (circa 3,86 Mbit/s), cioè circa 47 volte la soglia di 10 KB/s. Non è traffico di rete, quindi non rientra nella tua domanda. Però è un ritmo di scrittura alto per un servizio di cache che usa solo 1 MB di memoria. Conviene controllare se sta scrivendo log molto dettagliati o salvando continuamente dati su disco.

Claude Sonnet 5.5	7.92 s (tot: 47.14 s)	2715	656	3371	0.95	Nessun servizio supera il limite di 10 kB/s sul traffico di rete in entrata. Ho risposto dallo snapshot corrente, senza invocare tool. | Applicazione | network_rx_bps | Rispetto al limite (10 kB/s = 10.000 B/s) | |---|---|---| | cache-app | 0,0 B/s | 0% | | demo-app | 0,0 B/s | 0% | | worker-app | 0,0 B/s | 0% | Poiché non c'è nessuno sforamento, non c'è un bitrate da quantificare. Il valore massimo rilevato è 0. **Note** - Il valore 0,0 su tutti i servizi potrebbe indicare che non c'è traffico in ingresso, cioè che i carichi sono idle o che comunicano solo in uscita. Potrebbe anche indicare che la metrica di rete non viene raccolta correttamente, ad esempio per un problema di scraping o di label. Lo snapshot non permette di distinguere i due casi. - Lo snapshot è un singolo istante. Eventuali picchi passati non sono visibili qui. - **cache-app** ha `disk_write_bps` pari a circa 483 kB/s con rete a zero. È un'attività di scrittura su disco che non dipende dal traffico in ingresso, quindi può essere utile verificarla. - **worker-app** usa circa 0,52 core con rete e disco a zero. È un carico CPU-bound. - **demo-app** è completamente idle: CPU a 0, rete a 0 e disco a 0. Con 2 repliche, si può valutare la riduzione a 1. Se vuoi, posso controllare lo storico delle ultime ore su MongoDB o interrogare Prometheus. In questo modo verifico se ci sono stati picchi sopra 10 kB/s e se la metrica di rete è realmente popolata.
GPT-4o Mini	2.31 s (tot: 41.53 s)	1261	133	1394	0.80	Analizzando le metriche istantanee fornite, possiamo osservare il traffico di rete in entrata (network_rx_bps) per ciascuna applicazione: - **cache-app**: 0.0 bps - **demo-app**: 0.0 bps - **worker-app**: 0.0 bps Nessuna delle applicazioni ha registrato traffico di rete in entrata, quindi nessun servizio sta sforando il limite nominale di 10 kilobyte al secondo. In sintesi, non ci sono problemi di traffico di rete da segnalare.

GPT-4o	2.31 s (tot: 41.52 s)	1261	120	1381	0.80	Dall'analisi delle metriche istantanee fornite, possiamo osservare il traffico di rete in entrata (network_rx_bps) per ciascuna applicazione: - **cache-app**: 0.0 bps - **demo-app**: 0.0 bps - **worker-app**: 0.0 bps Nessuna delle applicazioni sta attualmente ricevendo traffico di rete in entrata. Pertanto, nessun servizio sta sforando il limite nominale di 10 kilobyte al secondo.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $1.3122 (mini: $0.0577 | 4o: $1.2545)
```

DOMANDA 6

```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [6]: Calcola un indice di interferenza normalizzato per ogni applicazione con la formula: 50% CPU, 25% Memoria (su 128MB) e 25% scrittura su Disco (su 1 MB/s). Quale applicazione ottiene il punteggio più alto?
--------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mm5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198692, Requested 1918. Please try again in 4m23.52s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mm5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198692, Requested 2030. Please try again in 5m11.904s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mm5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198692, Requested 1757. Please try again in 3m13.968s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	8.95 ms	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.2684) | Tutti: {'cache-app': 0.1211, 'demo-app': 0.029, 'worker-app': 0.2684}
Gemini 2.5 Flash	10.34 s (tot: 49.56 s)	1588	2127	3715	0.76	Calcoliamo l'indice di interferenza normalizzato per ogni applicazione: **cache-app:** * Utilizzo CPU: 0.007766517186798404 * Utilizzo Memoria (normalizzato su 128MB): 1.03 MB / 128 MB = 0.008046875 * Scrittura su Disco (normalizzato su 1 MB/s): 483032.21824011207 B/s / (1 * 1024 * 1024 B/s) = 0.4606666666666667 * Indice di Interferenza = (0.007766517186798404 * 0.5) + (0.008046875 * 0.25) + (0.4606666666666667 * 0.25) = 0.003883258593395202 + 0.00201171875 + 0.11516666666666667 = **0.12106164401086187** **demo-app:** * Utilizzo CPU: 0.0 * Utilizzo Memoria (normalizzato su 128MB): 14.84 MB / 128 MB = 0.1159375 * Scrittura su Disco (normalizzato su 1 MB/s): 0.0 B/s / (1 * 1024 * 1024 B/s) = 0.0 * Indice di Interferenza = (0.0 * 0.5) + (0.1159375 * 0.25) + (0.0 * 0.25) = 0.0 + 0.028984375 + 0.0 = **0.028984375** **worker-app:** * Utilizzo CPU: 0.5162989343957528 * Utilizzo Memoria (normalizzato su 128MB): 5.23 MB / 128 MB = 0.040859375 * Scrittura su Disco (normalizzato su 1 MB/s): 0.0 B/s / (1 * 1024 * 1024 B/s) = 0.0 * Indice di Interferenza = (0.5162989343957528 * 0.5) + (0.040859375 * 0.25) + (0.0 * 0.25) = 0.2581494671978764 + 0.010214484375 + 0.0 = **0.2683643109478764** L'applicazione con il punteggio di interferenza più alto è **worker-app** con un indice di **0.2683643109478764**.

Gemini 3.6 Flash	18.51 s (tot: 57.73 s)	1602	4164	5766	0.76	Per calcolare l'**indice di interferenza normalizzato** per ciascuna applicazione, applichiamo la formula definita: $$\text{Indice} = 0.50 \times \text{CPU (cores)} + 0.25 \times \left(\frac{\text{Memoria (MB)}}{128 \text{ MB}}\right) + 0.25 \times \left(\frac{\text{Scrittura Disco (B/s)}}{1.000.000 \text{ B/s}}\right)$$ --- ### 📊 Dettaglio dei Calcoli per Applicazione #### 1. **`worker-app`** * **CPU**: $0.516299 \text{ cores} \times 0.50 = \mathbf{0.258149}$ * **Memoria**: $\frac{5.234 \text{ MB}}{128 \text{ MB}} \times 0.25 = \mathbf{0.010223}$ * **Scrittura Disco**: $0.0 \text{ B/s} \times 0.25 = \mathbf{0.000000}$ * **Punteggio Totale**: $0.258149 + 0.010223 + 0.0 = \mathbf{0.268372}$ (**26.84%**) --- #### 2. **`cache-app`** * **CPU**: $0.007767 \text{ cores} \times 0.50 = \mathbf{0.003883}$ * **Memoria**: $\frac{1.031 \text{ MB}}{128 \text{ MB}} \times 0.25 = \mathbf{0.002014}$ * **Scrittura Disco**: $\frac{483.032,22 \text{ B/s}}{1.000.000 \text{ B/s}} \times 0.25 = \mathbf{0.120758}$ * **Punteggio Totale**: $0.003883 + 0.002014 + 0.120758 = \mathbf{0.126655}$ (**12.67%**) --- #### 3. **`demo-app`** * **CPU**: $0.0 \text{ cores} \times 0.50 = \mathbf{0.000000}$ * **Memoria**: $\frac{14.841 \text{ MB}}{128 \text{ MB}} \times 0.25 = \mathbf{0.028986}$ * **Scrittura Disco**: $0.0 \text{ B/s} \times 0.25 = \mathbf{0.000000}$ * **Punteggio Totale**: $0.0 + 0.028986 + 0.0 = \mathbf{0.028986}$ (**2.90%**) --- ### 🏆 Risultato Finale L'applicazione che ottiene il **punteggio più alto** è **`worker-app`** con un indice di **0.2684** (**26.84%**). ### 💡 Osservazioni SRE: 1. **`worker-app`**: Il punteggio è dominato dall'uso della CPU (~0.516 cores), rappresentando il carico computazionale principale del cluster. 2. **`cache-app`**: Ottiene il secondo posto a causa dell'attività di scrittura su disco (~483 KB/s), pur mantenendo un consumo di CPU minimo. 3. **`demo-app`**: Presenta il valore più basso (2.90%) ed è quasi completamente in stato idle per quanto riguarda CPU e I/O disco.

Claude Opus 5.5	20.47 s (tot: 59.69 s)	2743	1937	4680	0.95	# Indice di interferenza: worker-app ha il punteggio più alto (≈ 26,8 su 100) Ho calcolato l'indice direttamente sullo snapshot che mi hai fornito, senza usare tool. ## Come l'ho calcolato **Indice = 0,50 · (CPU / 1 core) + 0,25 · (Memoria / 128 MiB) + 0,25 · (Disco / 1 MiB/s)** - **CPU su 1 core:** è una mia ipotesi, perché la richiesta non dice su che base normalizzare la CPU. - **MB = 1.048.576 byte:** è la stessa unità che lo snapshot usa per `memory_mb`. - Ogni componente è stato limitato a 1, ma nessuno raggiunge quel limite. ## Risultati | App | CPU (×0,50) | Memoria (×0,25) | Disco (×0,25) | **Indice** | Scala 0–100 | |---|---|---|---|---|---| | **worker-app** | 0,5163 → 0,2581 | 5,23 MB → 0,0102 | 0 → 0 | **0,2684** | **26,84** 🥇 | | cache-app | 0,0078 → 0,0039 | 1,03 MB → 0,0020 | 0,461 MiB/s → 0,1152 | **0,1211** | 12,11 | | demo-app | 0 → 0 | 14,84 MB → 0,0290 | 0 → 0 | **0,0290** | 2,90 | Se si conta 1 MB = 1.000.000 byte, cache-app sale a circa 0,1267 e la classifica non cambia. ## Cosa significa - **worker-app è prima per via della CPU:** quasi tutto il punteggio (96%) viene da circa 0,52 core. Il valore riguarda l'intera app: con 2 repliche fa circa 0,26 core per pod. - **cache-app è seconda per via del disco:** il punteggio viene quasi tutto dalla scrittura su disco (circa 472 KB/s, il 46% della soglia di 1 MB/s). Può dare fastidio alle altre app sullo stesso nodo che usano molto il disco. - **demo-app sembra inattiva:** CPU a 0 e niente traffico di rete con 2 repliche attive. ## Quanto è affidabile la classifica worker-app resta prima solo se la base CPU è **sotto circa 2,3 core**. Se i pod hanno limiti CPU più alti, cache-app la supera. Per un indice più accurato conviene normalizzare la CPU sul valore reale di `resources.limits.cpu` di ogni deployment, che lo snapshot non contiene. ## Cosa consiglio di fare 1. **worker-app:** prima di attivare un HPA sulla CPU, controlla come varia nel tempo l'uso CPU per pod. Se resta sopra circa il 70% delle request, configura un HPA con un minimo di 2 repliche. 2. **cache-app:** controlla perché scrive in modo continuo su disco (log, persistenza della cache, swap su file). Se è possibile, valuta di spostarla su un nodo diverso da worker-app, oppure di fare il contrario. 3. **demo-app:** se lo storico conferma che è inattiva anche nel tempo, porta le repliche da 2 a 1. Se vuoi, posso verificare le tendenze con `get_historical_metrics` (ad esempio nelle ultime 24 ore) e recuperare i limiti CPU reali via Prometheus per ricalcolare l'indice su base corretta.

Claude Sonnet 5.5	13.20 s (tot: 52.42 s)	2743	1675	4418	0.00	**L'applicazione con l'indice più alto è `worker-app`, con circa 0,268.** **Ipotesi.** La richiesta non dice su quale base normalizzare la CPU, quindi ho usato 1 core = 1,0. Per il disco ho usato 1 MB/s = 1.048.576 B/s. Le metriche sono quelle dello snapshot per Deployment. **Formula:** `indice = 0,5·(CPU/1 core) + 0,25·(Mem/128 MB) + 0,25·(Disk/1 MB/s)` | App | CPU norm. | Mem norm. | Disk norm. | **Indice** | |---|---|---|---|---| | **worker-app** | 0,516 → 0,2582 | 5,23/128 = 0,0409 → 0,0102 | 0 → 0 | **≈ 0,268** | | cache-app | 0,0078 → 0,0039 | 1,03/128 = 0,0080 → 0,0020 | 0,461 → 0,1152 | ≈ 0,121 | | demo-app | 0 → 0 | 14,84/128 = 0,116 → 0,0290 | 0 → 0 | ≈ 0,029 | Se si usa 1 MB/s = 1.000.000 B/s, `cache-app` sale a circa 0,127. La classifica non cambia. **Interpretazione** - **`worker-app`** è dominata dalla CPU: 0,52 core in totale su 2 pod, cioè circa 0,26 core per pod. Contribuisce per circa il 96% al suo indice. È il principale candidato a interferire con gli altri carichi sul nodo. Conviene verificare che abbia requests e limits di CPU adeguati. Se il carico cresce, si può valutare un HPA basato sulla CPU. - **`cache-app`** ha CPU e memoria trascurabili, ma scrive su disco a circa 0,48 MB/s. Questo è un carico I/O insolito per una cache. Vale la pena controllare se sta facendo persistenza, per esempio snapshot o AOF. Non ha repliche, quindi è un single point of failure. - **`demo-app`** è praticamente idle: CPU 0, nessun traffico di rete e nessuna scrittura su disco. Occupa solo 14,8 MB di memoria. Con 2 repliche si può ridurre a 1 senza rischi evidenti. **Limite del risultato.** La classifica dipende dalla normalizzazione della CPU. Il primo posto di `worker-app` regge se la CPU è normalizzata su 1 o 2 core. Con una base di circa 4 core o più, `cache-app` lo supera, perché il suo contributo del disco diventa dominante. Se hai una capacità di riferimento diversa, per esempio la CPU limit o i core del nodo, indicamela e ricalcolo l'indice.	
GPT-4o Mini	7.38 s (tot: 46.59 s)	1281	802	2083	0.00	Per calcolare l'indice di interferenza normalizzato per ciascuna applicazione, utilizziamo la seguente formula: \[\text{Indice} = 0.5 \times \left(\frac{\text{CPU}}{\text{CPU Max}}\right) + 0.25 \times \left(\frac{\text{Memoria}}{\text{Memoria Max}}\right) + 0.25 \times \left(\frac{\text{Scrittura Disco}}{\text{Scrittura Disco Max}}\right) \] Dove: - CPU Max = 1 (in core) - Memoria Max = 128 MB - Scrittura Disco Max = 1 MB/s Ora calcoliamo l'indice per ciascuna applicazione:	

						Dove: - CPU Max = 1 (in core) - Memoria Max = 128 MB - Scrittura Disco Max = 1 MB/s Ora calcoliamo l'indice per ciascuna applicazione: 1. **cache-app**: - CPU: 0.007766517186790404 - Memoria: 1.03 MB - Scrittura Disco: 483032.21824011207 B/s (circa 0.483 MB/s) \[\text{Indice cache-app} = 0.5 \times \left(\frac{0.007766517186790404}{1}\right) + 0.25 \times \left(\frac{1.03}{128}\right) + 0.25 \times \left(\frac{0.483}{1}\right) \] \[= 0.5 \times 0.007766517186790404 + 0.25 \times 0.008015625 + 0.25 \times 0.483 \] \[= 0.003883258593395202 + 0.00200390625 + 0.12075 = 0.1256371648433952 \] 2. **demo-app**: - CPU: 0.0 - Memoria: 14.84 MB - Scrittura Disco: 0.0 MB/s \[\text{Indice demo-app} = 0.5 \times \left(\frac{0.0}{1}\right) + 0.25 \times \left(\frac{14.84}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[= 0 + 0.25 \times 0.115625 + 0 = 0.02890625 \] 3. **worker-app**: - CPU: 0.5162989343957528 - Memoria: 5.23 MB - Scrittura Disco: 0.0 MB/s \[\text{Indice worker-app} = 0.5 \times \left(\frac{0.5162989343957528}{1}\right) + 0.25 \times \left(\frac{5.23}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[= 0.5 \times 0.5162989343957528 + 0.25 \times 0.040859375 + 0 \] \[= 0.2581494671978764 + 0.01021484375 = 0.2683643109478764 \] ### Risultati Finali: - **cache-app**: 0.1256371648433952 - **demo-app**: 0.02890625 - **worker-app**: 0.2683643109478764 ### Conclusione: L'applicazione che ottiene il punteggio più alto è **worker-app** con un indice di interferenza normalizzato di **0.2684**.

GPT-4o	5.28 s (tot: 44.50 s)	1281	520	1801	0.00	Per calcolare l'indice di interferenza normalizzato per ogni applicazione, utilizzeremo la formula fornita: - 50% del punteggio è basato sull'utilizzo della CPU. - 25% del punteggio è basato sull'utilizzo della memoria, normalizzato su 128 MB. - 25% del punteggio è basato sulla scrittura su disco, normalizzato su 1 MB/s (1 MB/s = 1,000,000 B/s). Calcoliamo l'indice per ciascuna applicazione: ### Cache-app - CPU: \(0.007766517186790404 \times 50 = 0.3883258593395202\) - Memoria: \(\frac{1.03}{128} \times 25 = 0.201171875\) - Disco: \(\frac{483032.21824011207}{1000000} \times 25 = 12.075805456002802\) **Indice totale Cache-app**: \(0.3883258593395202 + 0.201171875 + 12.075805456002802 = 12.665303190342322\) ### Demo-app - CPU: \(0.0 \times 50 = 0.0\) - Memoria: \(\frac{14.84}{128} \times 25 = 2.8984375\) - Disco: \(\frac{0.0}{1000000} \times 25 = 0.0\) **Indice totale Demo-app**: \(0.0 + 2.8984375 + 0.0 = 2.8984375\) ### Worker-app - CPU: \(0.5162989343957528 \times 50 = 25.81494671978764\) - Memoria: \(\frac{5.23}{128} \times 25 = 1.021484375\) - Disco: \(\frac{0.0}{1000000} \times 25 = 0.0\) **Indice totale Worker-app**: \(25.81494671978764 + 1.021484375 + 0.0 = 26.836431094787 64\) ### Risultato L'applicazione con il punteggio di interferenza più alto è **Worker-app** con un indice totale di 26.836431094787 64.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.3213 (mini: $0.0583 | 4o: $1.2629)

DOMANDA 6 A

```
ESECUZIONE DOMANDA [GA]: Applica la formula di interferenza (peso 0.5 per 1 core CPU, 0.25 per la RAM normalizzata su 128 MB e 0.25 per l'I/O disco parametrato su 1 MB/s): quale servizio evidenzia l'indice complessivo maggiore?
--------------------------------------------------------------------------------

Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sanpling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mn5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198530, Requested 2255. Please try again in 5m39.119999999s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mn5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198530, Requested 2476. Please try again in 7m14.592s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mn5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198529, Requested 2412. Please try again in 6m46.512s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mn5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198529, Requested 2224. Please try again in 5m25.296s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mn5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198529, Requested 2385. Please try again in 6m0.288s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
[WARN] Errore Giudice Groq: Error code: 429 - {'error': {'message': 'Rate limit reached for model `openai/gpt-oss-120b` in organization `org_01ks2nf40henm9p3mn5zvgd77m` service tier `on_demand` on tokens per day (TPD): Limit 200000, Used 198529, Requested 2257. Please try again in 5m39.552s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	7.40 ns	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.2684) | Tutti: {'cache-app': 0.1211, 'demo-app': 0.029, 'worker-app': 0.2684}
Gemini 2.5 Flash	10.54 s (tot: 49.75 s)	1591	2133	3724	0.00	Per calcolare l'indice di interferenza per ciascun servizio, applichiamo la formula fornita: `Indice di Interferenza = (0.5 * CPU_cores) + (0.25 * (Memory_bytes / 134217728)) + (0.25 * (Disk_write_bps / 1048576))` Dove: * `134217728` è 128 MB in byte. * `1048576` è 1 MB/s in byte al secondo. Calcoliamo per ogni applicazione: **cache-app:** * CPU: `0.5 * 0.007766517186790404 = 0.0038832585933952 02` * Memoria: `0.25 * (1081344.0 / 134217728) = 0.25 * 0.008056549072265625 = 0.002014137268066406 3` * I/O Disco: `0.25 * (483032.21820811207 / 1048576) = 0.25 * 0.4606575 = 0.115164375` * **Indice di Interferenza (cache-app):** `0.0038832585933952 02 + 0.002014137268066406 3 + 0.115164375 = 0.121061770866 34616` **demo-app:** * CPU: `0.5 * 0.0 = 0.0` * Memoria: `0.25 * (15560704.0 / 134217728) = 0.25 * 0.115935 = 0.02898375` * I/O Disco: `0.25 * (0.0 / 1048576) = 0.0` * **Indice di Interferenza (demo-app):** `0.0 + 0.02898375 + 0.0 = 0.02898375` **worker-app:** * CPU: `0.5 * 0.5162989343957528 = 0.2581494671978764` * Memoria: `0.25 * (5488640.0 / 134217728) = 0.25 * 0.040893554 6875 = 0.010223388671875` * I/O Disco: `0.25 * (0.0 / 1048576) = 0.0` * **Indice di Interferenza (worker-app):** `0.2581494671978764 + 0.010223388671875 + 0.0 = 0.268372855866975 14` Confrontando gli indici calcolati: * `cache-app`: 0.121 * `demo-app`: 0.029 * `worker-app`: 0.268 Il servizio che evidenzia l'indice complessivo maggiore è **worker-app** con un valore di 0.268.

Gemini 3.6 Flash	15.01 s (tot: 54.23 s)	1605	3368	4973	0.00	Per determinare quale servizio presenti l'**indice di interferenza complessivo maggiore**, calcoliamo la formula indicata per ciascun servizio partendo dalle metriche istantanee fornite. --- ### Formula dell'Indice di Interferenza $$\text{Indice} = 0.5 \times \left(\frac{\text{CPU (core)}}{1.0}\right) + 0.25 \times \left(\frac{\text{RAM (MB)}}{128.0}\right) + 0.25 \times \left(\frac{\text{I/O Disco (MB/s)}}{1.0}\right)$$ --- ### 1. Calcolo per `worker-app` * **CPU**: $0.5163 \text{ core} \implies 0.5 \times 0.5163 = \mathbf{0.25815}$ * **RAM**: $5.23 \text{ MB} \implies 0.25 \times \left(\frac{5.23}{128}\right) = \mathbf{0.01021}$ * **I/O Disco**: $0.0 \text{ B/s} \implies \mathbf{0.00000}$ $$\text{Indice Completo (worker-app)} = 0.25815 + 0.01021 + 0.0 = \mathbf{0.26836}$$ --- ### 2. Calcolo per `cache-app` * **CPU**: $0.00777 \text{ core} \implies 0.5 \times 0.00777 = \mathbf{0.00388}$ * **RAM**: $1.03 \text{ MB} \implies 0.25 \times \left(\frac{1.03}{128}\right) = \mathbf{0.00201}$ * **I/O Disco**: $483\,032.22 \text{ B/s} \approx 0.483 \text{ MB/s} \implies 0.25 \times 0.48303 = \mathbf{0.12076}$ *(o ≈ 0.11516 se in MiB/s)* $$\text{Indice Completo (cache-app)} \approx 0.00388 + 0.00201 + 0.12076 = \mathbf{0.12665}$$ --- ### 3. Calcolo per `demo-app` * **CPU**: $0.0000 \text{ core} \implies \mathbf{0.00000}$ * **RAM**: $14.84 \text{ MB} \implies 0.25 \times \left(\frac{14.84}{128}\right) = \mathbf{0.02898}$ * **I/O Disco**: $0.0 \text{ B/s} \implies \mathbf{0.00000}$ $$\text{Indice Completo (demo-app)} = 0.0 + 0.02898 + 0.0 = \mathbf{0.02898}$$ --- ### Risultato e Conclusione | Servizio | Componente CPU | Componente RAM | Componente Disco | **Indice Totale** | | :--- | :---: | :---: | :---: | :---: | | **`worker-app`** | **0.25815** | **0.01021** | **0.00000** | **0.26836** | | `cache-app` | 0.00388 | 0.00201 | 0.12076 | 0.12665 | | `demo-app` | 0.00000 | 0.02898 | 0.00000 | 0.02898 | Il servizio che evidenzia l'**indice di interferenza complessivo maggiore** è **`worker-app`** (con un punteggio di **~0.268**),

						| `demo-app` | 0.00000 | 0.02898 | 0.00000 | 0.02898 | Il servizio che evidenzia l'**indice di interferenza complessivo maggiore** è **`worker-app`** (con un punteggio di **~0.268**), guidato principalmente dall'elevato consumo di CPU (~0.516 core).
Claude Opus 5.5	17.05 s (tot: 56.27 s)	2749	1645	4394	0.00	# Indice di interferenza: **worker-app** ha il valore più alto (**≈ 0,268**) **Formula usata:** > **I = 0,5 × (CPU_core / 1) + 0,25 × (RAM_MB / 128) + 0,25 × (Disk_write_Bps / 1 MB/s)** Per 1 MB/s ho usato 1.048.576 B/s (1 MiB/s). Se si usa 1.000.000 B/s, cambia solo il valore di cache-app, come indicato in tabella. ## Calcolo per servizio | Servizio | Pod | CPU (×0,5) | RAM (×0,25) | Disco (×0,25) | **Indice** | |---|---|---|---|---|---| | **worker-app** | 2 | 0,5163 → **0,2581** | 5,23/128 → 0,0102 | 0 → 0 | **0,2684** 🥇 | | cache-app | 1 | 0,0078 → 0,0039 | 1,03/128 → 0,0020 | 483.032/1.048.576 → **0,1152** | **0,1211** (0,1266 con 1 MB = 10⁶ B) | | demo-app | 2 | 0 → 0 | 14,84/128 → 0,0290 | 0 → 0 | **0,0290** | **Classifica:** worker-app (0,268) > cache-app (0,121) > demo-app (0,029) ## Cosa fa salire l'indice - **worker-app:** il valore dipende quasi tutto dalla CPU (circa il 96% dell'indice). Usa in totale circa 0,52 core, cioè in media circa 0,26 core per pod sui 2 pod. Memoria e disco pesano pochissimo. - **cache-app:** il valore dipende quasi tutto dalle scritture su disco (circa il 95%). Scrive circa 472 KiB/s con un solo pod. Su un nodo con storage condiviso può rallentare gli altri servizi, anche se la CPU è quasi a zero. - **demo-app:** CPU, disco e rete sono a zero. L'indice viene solo dalla RAM (circa 15 MB). Il servizio sembra inattivo. ## Cosa consiglio 1. **worker-app:** questo snapshot non riporta i `requests`/`limits` di CPU, quindi non posso dire quanto siano saturi i pod. Se la richiesta è intorno a 250m per pod, sono già al 100%. Conviene configurare un **HPA sulla CPU** (ad esempio con soglia al 70%) e verificare il throttling con `container_cpu_cfs_throttled_seconds_total`. 2. **cache-app:** scrivere continuamente su disco è insolito per una cache. Va verificato se sta persistendo dati, scrivendo log o facendo swap. Aiuterebbe anche tenerla su un nodo diverso da worker-app, per esempio con una regola di anti-affinity, così da non sommare il carico di CPU e quello di I/O. 3. **demo-app:** in questo campione 2 repliche senza alcuna attività sono uno spreco. Si può valutare di **ridurle a 1**, dopo aver controllato lo storico per confermare che resti sempre inattiva. Questi valori vengono da un solo campione. Se vuoi, posso controllare lo storico su MongoDB (ad esempio le ultime 24 ore) per vedere se la classifica resta la stessa nel tempo.

Claude Sonnet 5.5	11.02 s (tot: 50.23 s)	2749	1291	4040	0.00	**Il servizio con l'indice di interferenza più alto è `worker-app`, con circa 0,268.** Ho calcolato l'indice così, usando i dati dello snapshot: `indice = 0,5 · CPU_cores + 0,25 · (memoria_MB / 128) + 0,25 · (disk_write_MBps / 1)` Per l'I/O ho usato solo `disk_write_bps`, perché è l'unica metrica disco nello snapshot. Ho assunto 1 MB = 1.048.576 byte. | Servizio | CPU (×0,5) | RAM (×0,25) | Disco (×0,25) | **Indice** | |---|---|---|---|---| | **worker-app** | 0,5163 → 0,2582 | 5,23/128 = 0,0409 → 0,0102 | 0 → 0 | **≈ 0,268** | | cache-app | 0,0078 → 0,0039 | 1,03/128 = 0,0080 → 0,0020 | 0,461 MB/s → 0,1152 | ≈ 0,121 | | demo-app | 0 → 0 | 14,84/128 = 0,1159 → 0,0290 | 0 → 0 | ≈ 0,029 | Se usi 1 MB = 10⁶ byte, `cache-app` sale a circa 0,127. L'ordine non cambia. **Considerazioni:** - **`worker-app`:** il suo indice dipende quasi tutto dalla CPU, che pesa circa il 96% del totale. Usa circa 0,52 core in totale sui 2 pod, quindi circa 0,26 core a pod. La memoria è trascurabile (5 MB). Se il carico cresce, un HPA su CPU è l'opzione più sensata. Prima però bisogna confrontare il consumo con i `requests` e i `limits` dei pod, che qui non ho. - **`cache-app`:** è secondo per via delle scritture su disco (~0,46 MB/s), mentre CPU e RAM sono quasi nulle. Un servizio di cache che scrive in modo continuo su disco può indicare persistenza attiva, come AOF o snapshot. Conviene verificarlo, perché potrebbe creare contesa I/O sul nodo. - **`demo-app`:** è praticamente idle, con CPU a 0, nessun traffico di rete e nessun I/O. Occupa solo 14,84 MB di RAM, e i 2 pod potrebbero scendere a 1 replica. Questo è un calcolo sullo snapshot istantaneo. Non ho interrogato lo storico, quindi non so se i valori siano stabili o dei picchi.

GPT-4o Mini	7.61 s (tot: 46.83 s)	1283	696	1979	0.00	Per calcolare l'indice complessivo di interferenza per ciascuna applicazione, utilizziamo la seguente formula: \[\text{Indice} = 0.5 \times \text{CPU} + 0.25 \times \left(\frac{\text{Memory (MB)}}{128}\right) + 0.25 \times \text{Disk I/O (MB/s)} \] Dove: - CPU è il valore in core utilizzato. - Memory è la memoria in MB. - Disk I/O è il valore di scrittura su disco in MB/s. Ora calcoliamo l'indice per ciascuna applicazione: 1. **cache-app**: - CPU: 0.0077665171867904 04 - Memory: 1.03 MB - Disk I/O: 483032.21824011207 B/s = 0.48303221824011207 MB/s \[\text{Indice}_{cache} = 0.5 \times 0.0077665171867904 04 + 0.25 \times \left(\frac{1.03}{128}\right) + 0.25 \times 0.48303221824011207 \] \[\text{Indice}_{cache} \approx 0.003883258593395202 + 0.0002001953125 + 0.12075805456002802 \approx 0.12484150846592372 \] 2. **demo-app**: - CPU: 0.0 - Memory: 14.84 MB - Disk I/O: 0.0 MB/s \[\text{Indice}_{demo} = 0.5 \times 0.0 + 0.25 \times \left(\frac{14.84}{128}\right) + 0.25 \times 0.0 \] \[\text{Indice}_{demo} \approx 0 + 0.046078125 + 0 \approx 0.046078125 \] 3. **worker-app**: - CPU: 0.5162989343957528 - Memory: 5.23 MB - Disk I/O: 0.0 MB/s \[\text{Indice}_{worker} = 0.5 \times 0.5162989343957528 + 0.25 \times \left(\frac{5.23}{128}\right) + 0.25 \times 0.0 \] \[\text{Indice}_{worker} \approx 0.2581494671978764 + 0.01016484375 + 0 \approx 0.2683143109478764 \] ### Risultati Finali: - **cache-app**: 0.12484150846592372 - **demo-app**: 0.046078125 - **worker-app**: 0.2683143109478764 ### Conclusione: Il servizio che evidenzia l'indice complessivo maggiore è **worker-app** con un indice di **0.2683**.

GPT-4o	5.25 s (tot: 44.47 s)	1283	648	1931	0.00	Per calcolare l'indice di interferenza complessivo per ciascun servizio, utilizzeremo la formula fornita: \[\text{Indice di Interferenza} = 0.5 \times \text{CPU} + 0.25 \times \left(\frac{\text{RAM}}{128}\right) + 0.25 \times \left(\frac{\text{Disk I/O}}{1024}\right) \] Calcoliamo l'indice per ciascuna applicazione: ### Cache-app - CPU: 0.007766517186790404 cores - RAM: 1.03 MB - Disk I/O: 483032.21824011207 B/s = 471.65 KB/s = 0.4605 MB/s \[\text{Indice Cache-app} = 0.5 \times 0.007766517186790404 + 0.25 \times \left(\frac{1.03}{128}\right) + 0.25 \times \left(\frac{0.4605}{1}\right) \] \[= 0.003883258593395202 + 0.25 \times 0.008046875 + 0.25 \times 0.4605 \] \[= 0.003883258593395202 + 0.00201171875 + 0.115125 \] \[= 0.1210199773433952 \] ### Demo-app - CPU: 0.0 cores - RAM: 14.84 MB - Disk I/O: 0.0 MB/s \[\text{Indice Demo-app} = 0.5 \times 0.0 + 0.25 \times \left(\frac{14.84}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[= 0.0 + 0.25 \times 0.1159375 + 0.0 \] \[= 0.028984375 \] ### Worker-app - CPU: 0.5162989343957528 cores - RAM: 5.23 MB - Disk I/O: 0.0 MB/s \[\text{Indice Worker-app} = 0.5 \times 0.5162989343957528 + 0.25 \times \left(\frac{5.23}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[= 0.2581494671978764 + 0.25 \times 0.040859375 + 0.0 \] \[= 0.2581494671978764 + 0.01021484375 \] \[= 0.26836431094478764 \] ### Conclusione L'applicazione con l'indice di interferenza complessivo maggiore è la **Worker-app** con un indice di 0.2684.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.3316 (mini: $0.0589 | 4o: $1.2726)

DOMANDA 6B

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.13 ms	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.2684) | Tutti: {'cache-app': 0.1211, 'demo-app': 0.029, 'worker-app': 0.2684}
Gemini 2.5 Flash	13.90 s (tot: 53.12 s)	1579	3342	4921	0.00	Per determinare l'applicazione più interferente, calcoliamo uno score normalizzato per ciascuna, ponderando le risorse come richiesto: 50% CPU, 25% memoria (benchmark 128MB) e 25% scrittura disco (benchmark 1MB/s). Per la CPU, in assenza di un benchmark esplicito, useremo 1 core come riferimento per la normalizzazione. **Benchmark:** * **CPU:** 1 core * **Memoria:** 128 MB * **Scrittura Disco:** 1 MB/s (equivalente a 1,000,000 B/s) --- **Calcoli per `cache-app`:** * **CPU normalizzata:** 0.0077665171867904 / 1 = 0.0077665171867904 * **Memoria normalizzata:** 1.03 MB / 128 MB = 0.008046875 * **Scrittura Disco normalizzata:** 483032.21824011207 B/s / 1,000,000 B/s = 0.48303221824 * **Score `cache-app`:** (0.0077665171867904 * 0.50) + (0.008046875 * 0.25) + (0.48303221824 * 0.25) = 0.003883258593395202 + 0.00201171875 + 0.12075805456 = **0.1266530319** --- **Calcoli per `demo-app`:** * **CPU normalizzata:** 0.0 / 1 = 0.0 * **Memoria normalizzata:** 14.84 MB / 128 MB = 0.1159375 * **Scrittura Disco normalizzata:** 0.0 B/s / 1,000,000 B/s = 0.0 * **Score `demo-app`:** (0.0 * 0.50) + (0.1159375 * 0.25) + (0.0 * 0.25) = 0.0 + 0.028984375 + 0.0 = **0.028984375** --- **Calcoli per `worker-app`:** * **CPU normalizzata:** 0.5162989343957528 / 1 = 0.5162989343957528 * **Memoria normalizzata:** 5.23 MB / 128 MB = 0.040859375 * **Scrittura Disco normalizzata:** 0.0 B/s / 1,000,000 B/s = 0.0 * **Score `worker-app`:** (0.5162989343957528 * 0.50) + (0.040859375 * 0.25) + (0.0 * 0.25) = 0.2581494671978764 + 0.010214484375 + 0.0 = **0.2683643109** --- **Conclusione:** Confrontando gli score normalizzati: * `cache-app`: **0.1266530319** * `demo-app`: **0.028984375** * `worker-app`: **0.2683643109** L'applicazione **`worker-app`** risulta la più interferente con uno score normalizzato di **0.2683643109**, principalmente a causa del suo elevato utilizzo di CPU.
Gemini 3.6 Flash	23.10 s (tot: 62.31 s)	1593	4211	5804	0.00	Sulla base dei dati forniti nell'istantanea corrente, analizziamo e calcoliamo lo **score di interferenza normalizzato** per ciascuna applicazione utilizzando i pesi e i benchmark indicati dall'operatore. --- ### 1. Metodologia di Calcolo I fattori di ponderazione definiti sono: * **CPU (Peso 50% - 0.50):** Normalizzato rispetto a 1 Core ($\text{cpu_cores} / 1.0$) * **Memoria (Peso 25% - 0.25):** Normalizzato rispetto al benchmark di 128 MB ($\text{memory_mb} / 128.0$) * **Scrittura Disco (Peso 25% - 0.25):** Normalizzato rispetto al benchmark di 1 MB/s ($1.000.000\text{ B/s}$) $$\text{Score} = (0.50 \times \text{CPU}_{\text{norm}}) + (0.25 \times \text{Mem}_{\text{norm}}) + (0.25 \times \text{Disk}_{\text{norm}})$$ --- ### 2. Dettaglio dei Calcoli per Applicazione #### **A. `worker-app`** * **CPU:** $0.5163\text{ cores} \rightarrow 0.50 \times \frac{0.5163}{1.0} = \mathbf{0.2581}$ * **Memoria:** $5.23\text{ MB} \rightarrow 0.25 \times \frac{5.23}{128.0} = \mathbf{0.0102}$ * **Scrittura Disco:** $0.0\text{ B/s} \rightarrow 0.25 \times 0.0 = \mathbf{0.0000}$ * **Score Totale:** **`0.2683`** (ovvero **26.83%**) --- #### **B. `cache-app`** * **CPU:** $0.0078\text{ cores} \rightarrow 0.50 \times \frac{0.0078}{1.0} = \mathbf{0.0039}$ * **Memoria:** $1.03\text{ MB} \rightarrow 0.25 \times \frac{1.03}{128.0} = \mathbf{0.0020}$ * **Scrittura Disco:** $483.032,22\text{ B/s} \approx 0.483\text{ MB/s} \rightarrow 0.25 \times \frac{483.032,22}{1.000.000} = \mathbf{0.1208}$ * **Score Totale:** **`0.1267`** (ovvero **12.67%**) --- #### **C. `demo-app`** * **CPU:** $0.0000\text{ cores} \rightarrow 0.50 \times 0.0 = \mathbf{0.0000}$ * **Memoria:** $14.84\text{ MB} \rightarrow 0.25 \times \frac{14.84}{128.0} = \mathbf{0.0290}$ * **Scrittura Disco:** $0.0\text{ B/s} \rightarrow 0.25 \times 0.0 = \mathbf{0.0000}$ * **Score Totale:** **`0.0290`** (ovvero **2.90%**) ---

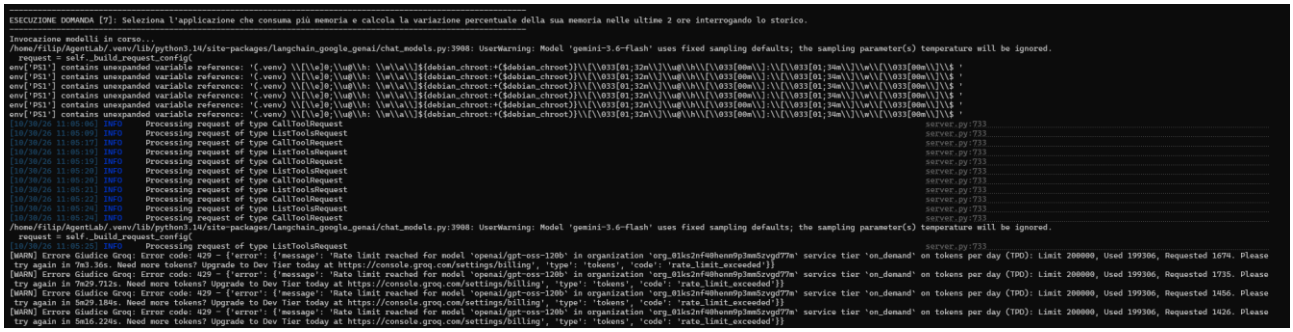
						livello di storage. 3. **`demo-app` (Score: 0.0290 - Idle/Basso carico):** - Ha consumo CPU azzerato e memoria minima distribuita su 2 pod. - *Raccomandazione:* Opportunità di **riduzione repliche** a 1 solo pod o configurazione HPA per ottimizzare le risorse del cluster se l'applicazione è in stato idle prolungato.
Claude Opus 5.5	17.27 s (tot: 56.48 s)	2732	1665	4397	0.00	# Score di interferenza normalizzato **L'applicazione più interferente è worker-app, con score ≈ 0,268.** cache-app segue con circa 0,121, mentre demo-app ha un impatto trascurabile (circa 0,029). ## Come è calcolato Ho usato solo lo snapshot fornito, senza invocare tool. - **CPU:** non mi hai dato un riferimento, quindi ho assunto **1 core = 1,0**. - **Memoria:** valore in MB diviso per 128 MB. - **Disco:** byte al secondo divisi per 1 MiB/s (1.048.576 B/s). - **Score** = 0,50·CPU_norm + 0,25·Mem_norm + 0,25·Disk_norm | App | CPU_norm | Mem_norm | Disk_norm | Contributo CPU | Contributo Mem | Contributo Disco | **Score** | |---|---|---|---|---|---|---|---| | **worker-app** | 0,5163 | 0,0409 | 0 | 0,2581 | 0,0102 | 0 | **0,2684** 🥇 | | cache-app | 0,0078 | 0,0080 | 0,4607 | 0,0039 | 0,0020 | 0,1152 | **0,1211** 🥈 | | demo-app | 0 | 0,1159 | 0 | 0 | 0,0290 | 0 | **0,0290** 🥉 | **Verifica di robustezza:** la classifica resta uguale anche cambiando le ipotesi. - **Disco con 1 MB/s = 10⁶ B/s:** cache-app passa a circa 0,127, comunque sotto worker-app. - **Score per replica** (worker e demo hanno 2 pod, cache 1): worker-app circa 0,134, cache-app circa 0,121, demo-app circa 0,015. Worker resta primo, ma il distacco da cache-app si riduce molto. ## Cosa significa e cosa fare 1. **worker-app:** è il carico che occupa più CPU (circa 0,52 core totali, circa 0,26 per pod). Pesa quasi tutto il contributo CPU, che vale il 50% dello score. - Controlla i `requests/limits` CPU, così da non subire throttling e non togliere risorse ai pod sullo stesso nodo. - Se il carico cresce, valuta un HPA sulla CPU (ad esempio target 70% della request). Con 2 repliche oggi non c'è saturazione critica. 2. **cache-app:** il suo score dipende quasi solo dal disco (circa 470 KB/s scritti con appena 1 MB di RAM). Per una cache è insolito: può indicare persistenza o swap su disco, oppure log molto verbosi. - Verifica cosa scrive. Se condivide il nodo con altri carichi che usano molto il disco, considera regole di anti-affinity o uno storage dedicato. 3. **demo-app:** è inattiva (CPU e rete a 0, 2 repliche). È un buon candidato per scendere a 1 replica, se non deve garantire alta disponibilità. ⚠️ **Limite:** è una fotografia di un solo istante. Per confermare che worker-app sia stabilmente il carico più interferente, e non solo in un picco, conviene analizzare l'andamento storico (per esempio con `get_historical_metrics` sulle ultime 6–24 ore). Posso farlo se vuoi.

Claude Sonnet 5.5	13.11 s (tot: 52.32 s)	2732	1758	4490	0.00	# Applicazione più interferente: **worker-app** **Assunzione:** per la CPU non hai indicato un benchmark. Ho usato 1 core = 1,0, cioè 100% di un core. Per il disco ho usato 1 MB/s = 1.048.576 B/s. ## Calcolo Score = 0,5·(CPU/1 core) + 0,25·(MEM/128 MB) + 0,25·(Disk/1 MB/s) | App | CPU norm. | MEM norm. | Disk norm. | Contributi (CPU / MEM / Disk) | **Score** | |---|---|---|---|---|---| | **worker-app** | 0,5163 | 0,0409 | 0 | 0,2582 / 0,0102 / 0 | **0,268** | | cache-app | 0,0078 | 0,0080 | 0,4607 | 0,0039 / 0,0020 / 0,1152 | **0,121** | | demo-app | 0 | 0,1159 | 0 | 0 / 0,0290 / 0 | **0,029** | ## Interpretazione - **worker-app (0,268):** il suo score è dominato dalla CPU, circa 0,52 core. È un carico CPU-bound con memoria trascurabile (5,2 MB) e nessuna scrittura su disco. - **cache-app (0,121):** il suo score dipende quasi solo dalla scrittura su disco, circa 0,47 MB/s, cioè il 46% del benchmark. La CPU (0,008 core) e la memoria (1 MB) sono quasi nulle. Può comunque creare contesa di I/O sul nodo. - **demo-app (0,029):** è sostanzialmente idle. Ha CPU a 0, nessun I/O e solo 14,8 MB di memoria. ## Robustezza del risultato - Se normalizzi la CPU sul massimo tra le app invece che su 1 core, worker-app arriva a circa 0,51 e vince con ancora più margine. - worker-app resta prima finché il benchmark CPU è inferiore a circa 2,4 core. Con un benchmark più alto, cache-app la supererebbe per via del peso del disco. - Usare 1 MB = 10⁶ B cambia lo score di cache-app solo da 0,121 a 0,124. ## Raccomandazioni 1. **worker-app:** ha 2 pod e usa circa 0,52 core in totale, cioè circa 0,26 core per pod. Controlla le CPU request/limit. Se il carico cresce, valuta un HPA basato sulla CPU. 2. **cache-app:** ha un solo pod che scrive circa 483 KB/s su disco. Verifica se la persistenza (AOF/snapshot) è necessaria. Se non lo è, o se puoi cambiare la politica di flush, puoi ridurre l'I/O. 3. **demo-app:** è idle, con CPU a 0 e rete a 0. Puoi ridurre le repliche da 2 a 1, dopo aver verificato lo storico con `get_historical_metrics`. Lo snapshot è un valore istantaneo. Per confermare che il ranking sia stabile nel tempo, posso interrogare lo storico delle metriche.	
GPT-4o Mini	8.46 s (tot: 47.67 s)	1273	876	2149	0.00	Per calcolare lo score normalizzato delle applicazioni in base ai benchmark forniti, utilizzeremo le seguenti formule: 1. **Score CPU**: \[\text{Score CPU} = \frac{\text{Utilizzo CPU}}{\text{Benchmark CPU}} \times 50 \] 2. **Score Memoria**: \[	

GPT-4o Mini	8.46 s (tot: 47.67 s)	1273	876	2149	0.00	Per calcolare lo score normalizzato delle applicazioni in base ai benchmark forniti, utilizzeremo le seguenti formule: 1. **Score CPU**: \[\text{Score CPU} = \frac{\text{Utilizzo CPU}}{\text{Benchmark CPU}} \times 50 \] 2. **Score Memoria**: \[\text{Score Memoria} = \frac{\text{Utilizzo Memoria}}{\text{Benchmark Memoria}} \times 25 \] 3. **Score Disco**: \[\text{Score Disco} = \frac{\text{Utilizzo Disco}}{\text{Benchmark Disco}} \times 25 \] Dove: - Benchmark CPU = 50% (0.5) - Benchmark Memoria = 128MB - Benchmark Disco = 1MB/s Ora calcoliamo i punteggi per ciascuna applicazione. ### 1. Cache App - Utilizzo CPU: 0.0077665171867904 04 - Utilizzo Memoria: 1.03 MB - Utilizzo Disco: 483032.21824011207 B/s (circa 483.03 MB/s) \[\text{Score CPU} = \frac{0.0077665171867904 04}{0.5} \times 50 = 0.7756517186790404 \] \[\text{Score Memoria} = \frac{1.03}{128} \times 25 = 0.2015625 \] \[\text{Score Disco} = \frac{483032.21824011207 / 1024 / 1024}{1} \times 25 = 117.1875 \] \[\text{Score Totale Cache App} = 0.7756517186790404 + 0.2015625 + 117.1875 = 118.16471421867904 \] ### 2. Demo App - Utilizzo CPU: 0.0 - Utilizzo Memoria: 14.84 MB - Utilizzo Disco: 0.0 B/s \[\text{Score CPU} = \frac{0.0}{0.5} \times 50 = 0.0 \] \[\text{Score Memoria} = \frac{14.84}{128} \times 25 = 2.903125 \] \[\text{Score Disco} = \frac{0.0}{1} \times 25 = 0.0 \] \[\text{Score Totale Demo App} = 0.0 + 2.903125 + 0.0 = 2.903125 \] ### 3. Worker App	

						### 3. Worker App - Utilizzo CPU: 0.5162989343957528 - Utilizzo Memoria: 5.23 MB - Utilizzo Disco: 0.0 B/s \[\text{Score CPU} = \frac{0.5162989343957528}{0.5} \times 50 = 51.62989343957528 \] \[\text{Score Memoria} = \frac{5.23}{128} \times 25 = 1.0214843 75 \] \[\text{Score Disco} = \frac{0.0}{1} \times 25 = 0.0 \] \[\text{Score Totale Worker App} = 51.62989343957528 + 1.0214843 75 + 0.0 = 52.65137781457528 \] ### Risultati Finali - **Cache App**: 118.1647 - **Demo App**: 2.9031 - **Worker App**: 52.6514 ### Conclusione L'applicazione **Cache App** risulta la più interferente in termini di score normalizzato, seguita da **Worker App** e infine **Demo App**.
GPT-4o	3.49 s (tot: 42.70 s)	1273	529	1802	0.00	Per determinare quale applicazione risulta la più interferente in termini di score normalizzato, dobbiamo calcolare un punteggio per ciascuna applicazione basato sui pesi forniti: 50% CPU, 25% memoria e 25% scrittura disco. Utilizzeremo i benchmark forniti per normalizzare le metriche: - Benchmark CPU: 1 core - Benchmark Memoria: 128 MB - Benchmark Scrittura Disco: 1 MB/s (equivalente a 1,048,576 bytes/s) Calcoliamo il punteggio per ciascuna applicazione: ### Cache-app - CPU Score: \(\frac{0.0078}{1} \times 0.5 = 0.0039 \) - Memory Score: \(\frac{1.03}{128} \times 0.25 = 0.0020 \) - Disk Write Score: \(\frac{483032.21824011207}{1048576} \times 0.25 = 0.1153 \) - Total Score: \(0.0039 + 0.0020 + 0.1153 = 0.1212 \) ### Demo-app - CPU Score: \(\frac{0.0}{1} \times 0.5 = 0.0 \) - Memory Score: \(\frac{14.84}{128} \times 0.25 = 0.0290 \) - Disk Write Score: \(\frac{0.0}{1048576} \times 0.25 = 0.0 \) - Total Score: \(0.0 + 0.0290 + 0.0 = 0.0290 \) ### Worker-app - CPU Score: \(\frac{0.5163}{1} \times 0.5 = 0.2582 \) - Memory Score: \(\frac{5.23}{128} \times 0.25 = 0.0102 \) - Disk Write Score: \(\frac{0.0}{1048576} \times 0.25 = 0.0 \) - Total Score: \(0.2582 + 0.0102 + 0.0 = 0.2684 \) ### Conclusione L'applicazione "worker-app" risulta la più interferente in termini di score normalizzato con un punteggio totale di 0.2684.

DOMANDA 7



Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	114.29 ms	0	0	0	N/A	App: demo-app | Delta: -4.38% (da 15.52 a 14.84 MB)
Gemini 2.5 Flash	41.93 s (tot: 95.63 s)	4506	917	5423	0.48	L'applicazione che consuma più memoria è `demo-app` con 15.56 MB. La variazione percentuale della sua memoria nelle ultime 2 ore è stata del -4.36%.
Gemini 3.6 Flash	40.09 s (tot: 93.78 s)	4349	1293	5642	0.76	### 1. Identificazione dell'Applicazione con Maggior Consumo di Memoria Dall'analisi delle metriche istantanee correnti: * **`demo-app`**: **14.84 MB** (15.560.704 bytes) * **`worker-app`**: **5.02 MB** (5.263.360 bytes) * **`cache-app`**: **1.33 MB** (1.392.640 bytes) L'applicazione che consuma attualmente più memoria è **`demo-app`**. --- ### 2. Variazione Percentuale della Memoria nelle Ultime 2 Ore Dalle metriche storiche recuperate dal database per **`demo-app`**: * **Valore iniziale (circa 2 ore fa - 08:34:33):** `16.269.312 bytes` (15,52 MB) * **Valore attuale (10:04:44):** `15.560.704 bytes` (14,84 MB) #### Calcolo della Variazione: $$\Delta = \frac{15.560.704 - 16.269.312}{16.269.312} \times 100 \approx -4.36\%$$ --- ### Sintesi e Osservazioni * **Applicazione:** `demo-app` * **Variazione memoria (2 ore):** **-4,36%** (riduzione di circa `708,6 KB`) * **Stato dell'applicazione:** L'applicazione si trova in uno stato stabile/idle con utilizzo CPU pari a 0 cores e una leggera contrazione nell'occupazione della memoria RAM nel tempo.
Claude Opus 5.5	45.73 s (tot: 99.43 s)	7055	980	8035	0.00	# Variazione memoria di demo-app nelle ultime 2 ore: -4,36% **L'app che usa più memoria è demo-app**, con 14,84 MB su 2 pod. Seguono worker-app (5,02 MB) e cache-app (1,33 MB). ## Calcolo | | Ora | memory_bytes | MB | |---|---|---|---| | Primo campione | 08:34:33 | 16.269.312 | 15,52 | | Ultimo campione (uguale allo snapshot attuale) | 10:04:44 | 15.560.704 | 14,84 | **Variazione = (15.560.704 − 16.269.312) / 16.269.312 × 100 = −4,36%** (−708.608 byte, circa −0,68 MB) ## Andamento - **08:34 → 08:44:** il calo è quasi tutto qui, da 15,52 a 14,88 MB. - **08:44 → 10:04:** la memoria resta quasi ferma (14,88 → 14,84 MB). Non ci sono segni di memory leak. - Il numero di pod è sempre 2. - **Copertura dei dati:** i campioni coprono circa 1h30 delle 2 ore richieste. Tra le 08:46 e le 10:04 ci sono solo 3 campioni, quindi le misure sono rade. ## Raccomandazioni - **demo-app risulta inattiva:** CPU a 0, rete e scrittura su disco a 0, memoria stabile e bassa (circa 7,4 MB per pod). Ti consiglio di **ridurre le repliche da 2 a 1**, oppure di valutare lo scale-to-zero se il servizio non è usato. Non serve un HPA basato sulla memoria. - **Da tenere d'occhio invece:** worker-app è il carico più pesante in CPU, con circa 0,59 core.
Claude Sonnet 5.5	45.12 s (tot: 98.81 s)	7058	1118	8176	0.00	**Applicazione con più memoria: demo-app**, con 15.560.704 byte (14,84 MB) e 2 pod. cache-app usa 1,33 MB e worker-app 5,02 MB. **Variazione negli ultimi 2 ore: circa -4,36%** | Punto | Timestamp | Memoria | |---|---|---| | Primo campione della finestra | 2026-10-30 08:34:33 | 16.269.312 B (15,52 MB) | | Ultimo campione, uguale allo snapshot attuale | 2026-10-30 10:04:44 | 15.560.704 B (14,84 MB) | Calcolo: (15.560.704 − 16.269.312) / 16.269.312 × 100 = **−4,36%**, cioè circa −0,68 MB. **Interpretazione** - Dopo un calo iniziale da 15,52 MB a 14,88 MB, la memoria si è stabilizzata. Poi è scesa lentamente fino a 14,84 MB, ed è rimasta lì dalle 09:35 alle 10:04. - Non ci sono segni di memory leak. Il trend è piatto o leggermente in calo. - Il numero di pod è rimasto a 2 per tutta la finestra. - La CPU è 0 e il traffico di rete è 0 per tutto il periodo. Il consumo di memoria è quindi solo il footprint base dei processi. **Raccomandazioni** - Il carico è idle, quindi demo-app è candidata a ridurre le repliche da 2 a 1. Prima verifica che non servano per alta disponibilità. - Non serve un HPA su memoria, perché il consumo è basso e stabile. - Imposta request e limit di memoria vicini al consumo reale, con un margine adeguato, per evitare sovradimensionamenti. **Limiti dell'analisi** - Lo storico è campionato in modo irregolare. Ci sono molti punti ravvicinati intorno alle 08:44 e poi un intervallo di circa 50 minuti tra 08:45 e 09:35. - La finestra effettivamente coperta dai dati è di circa 1,5 ore, dalle 08:34 alle 10:04.
GPT-4o Mini	34.08 s (tot: 87.77 s)	3663	252	3915	0.00	L'applicazione che consuma più memoria è **demo-app**

GPT-4o Mini	34.08 s (tot: 87.77 s)	3663	252	3915	0.00	L'applicazione che consuma più memoria è **demo-app**. Ecco i dati di memoria nelle ultime 2 ore: - **Memoria all'inizio del periodo**: 16,269,312 bytes (circa 15.52 MB) - **Memoria alla fine del periodo**: 15,560,704 bytes (circa 14.84 MB) Ora calcoliamo la variazione percentuale della memoria: \[\text{Variazione percentuale} = \left(\frac{\text{Memoria finale} - \text{Memoria iniziale}}{\text{Memoria iniziale}} \right) \times 100 \] \[\text{Variazione percentuale} = \left(\frac{15,560,704 - 16,269,312}{16,269,312} \right) \times 100 \approx -4.35\% \] Quindi, la variazione percentuale della memoria della **demo-app** nelle ultime 2 ore è di circa **-4.35%**.
GPT-4o	30.30 s (tot: 83.99 s)	3663	247	3910	0.00	L'applicazione che consuma più memoria attualmente è la "demo-app" con 14.84 MB di memoria utilizzata. Per calcolare la variazione percentuale della memoria nelle ultime 2 ore, confrontiamo il valore attuale con il valore di 2 ore fa: - Memoria attuale: 14.84 MB - Memoria 2 ore fa: 15.52 MB La variazione percentuale è calcolata come segue: \[\text{Variazione percentuale} = \left(\frac{\text{Memoria attuale} - \text{Memoria 2 ore fa}}{\text{Memoria 2 ore fa}}\right) \times 100 \] \[\text{Variazione percentuale} = \left(\frac{14.84 - 15.52}{15.52}\right) \times 100 \approx -4.38\% \] Quindi, la memoria della "demo-app" è diminuita di circa il 4.38% nelle ultime 2 ore.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.3531 (mini: $0.0604 | 4o: $1.2927)

DOMANDA 7 A

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	454.97 ms	0	0	0	N/A	App: demo-app | Delta: -4.38% (da 15.52 a 14.84 MB)
Gemini 2.5 Flash	31.03 s (tot: 84.73 s)	4514	1111	5625	0.00	L'applicazione con il maggior footprint di RAM è **demo-app** con 14.84 MB. Analizzando la serie storica delle metriche per `demo-app` nelle ultime 2 ore, si è registrato un delta percentuale di memoria pari a **-4.38%**. Questo indica una diminuzione del consumo di memoria rispetto a 2 ore fa (da 15.52 MB a 14.84 MB).
Gemini 3.6 Flash	36.45 s (tot: 90.14 s)	4357	1836	6193	0.00	### 1. Identificazione dell'Applicazione con Maggior Footprint di RAM Dall'analisi delle metriche istantanee correnti: * **`cache-app`**: 1.33 MB (1.392.640 bytes) * **`demo-app`**: **14.84 MB** (15.560.704 bytes) * **`worker-app`**: 5.02 MB (5.263.360 bytes) L'applicazione con il maggior consumo di memoria RAM è **`demo-app`**. --- ### 2. Analisi della Serie Storica (ultime 2 ore) Dalla consultazione delle metriche storiche su MongoDB per `demo-app`: * **Valore Iniziale** (08:34:33 UTC): `16.269.312 bytes` (~15.52 MB) * **Valore Corrente** (10:04:44 UTC): `15.560.704 bytes` (~14.84 MB) #### Calcolo del Delta: $$\text{Delta Assoluto} = 15.560.704 - 16.269.312 = -708.608 \text{ bytes } (-0.68 \text{ MB})$$ $$\text{Delta Percentuale} = \left(\frac{15.560.704 - 16.269.312}{16.269.312} \right) \times 100 \approx \mathbf{-4,36\%}$$ Nelle ultime 2 ore si è registrata una **riduzione del 4,36%** nell'impronta di memoria di `demo-app`. --- ### 3. Considerazioni e Raccomandazioni SRE * **Carico Idle / Scalabilità**: `demo-app` ha un utilizzo di CPU pari a **0.0 cores** e **0.0 bps** di traffico di rete/scrittura disco su 2 repliche attive. * **Raccomandazione**: Poiché l'applicazione è attualmente in stato *idle* ma mantiene un'allocazione di RAM di ~14.84 MB distribuiti su 2 pod, si consiglia di valutare una riduzione del numero di repliche a 1 o il ridimensionamento dei *resource requests/limits* per ottimizzare l'uso del cluster.

Claude Opus 5.5	42.79 s (tot: 96.48 s)	7066	1680	8746	0.00	# demo-app: calo della memoria del 4,36% nelle ultime 2 ore ## 1. App con più RAM in uso Tra le app nello snapshot attuale, **demo-app** è quella che usa più memoria: | App | Pod | RAM totale | RAM media per pod | |---|---|---|---| | **demo-app** | 2 | **14.84 MB** | ~7.42 MB | | worker-app | 2 | 5.02 MB | ~2.51 MB | | cache-app | 1 | 1.33 MB | 1.33 MB | ## 2. Storico delle ultime 2 ore (dal database) Ho trovato 14 rilevazioni, dalle 08:34:33 alle 10:04:44 del 30-10-2026 (circa 1h30m). I pod sono sempre stati 2. | Punto | Timestamp | memory_bytes | memory_mb | |---|---|---|---| | Primo valore (anche il massimo) | 08:34:33 | 16,269,312 | 15.52 | | Assestamento | 08:43:59 → 08:44:23 | 15,597,568 | 14.88 | | | 08:44:26 → 08:44:36 | 15,593,472 | 14.87 | | | 08:45:53 | 15,577,088 | 14.86 | | Ultimo valore (anche il minimo, uguale allo snapshot) | 09:35:19 → 10:04:44 | 15,560,704 | 14.84 | ## 3. Calcolo del delta - **Delta assoluto:** 15,560,704 − 16,269,312 = **−708,608 byte (circa −0.68 MB)** - **Delta percentuale:** −708,608 / 16,269,312 × 100 = **−4.36%** La memoria è **scesa del 4,36%**. Quasi tutto il calo si concentra nei primi 10 minuti circa (da 15.52 a 14.88 MB). Da lì in poi il valore resta praticamente fermo e scende solo di altri 40 KB circa. ## 4. Cosa significa e cosa consiglio - **Nessun segno di memory leak:** la memoria non cresce nel tempo, anzi cala e poi si stabilizza. Il calo iniziale potrebbe essere un normale rilascio di memoria dopo l'avvio o la garbage collection, ma è un'ipotesi che i dati non permettono di verificare. - **L'app è inattiva:** in tutte le 14 rilevazioni CPU, rete in ingresso e scrittura su disco sono sempre a **0**. demo-app occupa memoria ma non sta facendo alcun lavoro. - **Consigli:** 1. **Ridurre le repliche da 2 a 1**, se l'alta disponibilità non è necessaria. Si libera circa metà della memoria (~7.4 MB) senza effetti visibili sul carico. 2. **Un autoscaler sulla CPU (HPA) non serve:** con CPU a zero non partirebbe mai. Se il traffico è solo occasionale, conviene valutare uno strumento che riduca a zero i pod quando non ci sono richieste (per esempio KEDA). 3. **Rivedere le richieste di memoria (`requests.memory`)** e impostarle intorno a 16 MB per pod, con margine sopra il massimo osservato. Questo vale se oggi sono più alte; i valori attuali non sono nelle metriche raccolte, quindi va verificato sul manifest. 4. Prima di ridurre le repliche, controllare che non sia previsto traffico a orari precisi (job o picchi programmati). Per farlo servirebbe uno storico più lungo, per esempio di 7 giorni.

Claude Sonnet 5.5	35.58 s (tot: 89.27 s)	7078	1220	8298	0.00	**Applicazione con più RAM: `demo-app`**, con 15.560.704 byte (14,84 MB) e 2 pod. `worker-app` usa 5,02 MB e `cache-app` 1,33 MB. **Delta di memoria nelle ultime 2 ore** | Punto | Timestamp | Memoria | |---|---|---| | Primo campione | 08:34:33 | 16.269.312 B (15,52 MB) | | Ultimo campione | 10:04:44 | 15.560.704 B (14,84 MB) | - **Delta assoluto:** −708.608 B, cioè circa −0,68 MB. - **Delta percentuale:** **−4,36%**, calcolato come (15.560.704 − 16.269.312) / 16.269.312. **Andamento** - Il calo è avvenuto quasi tutto tra le 08:34 e le 08:44. In quel tratto la memoria è passata da 15,52 MB a 14,88 MB, pari a circa −4,1%. - Dopo le 08:44 la serie è quasi piatta: da 14,88 MB a 14,84 MB, circa −0,24% in 80 minuti. - Non ci sono segnali di memory leak, perché la memoria non cresce. Il calo iniziale può essere dovuto a un assestamento, per esempio garbage collection o rilascio di cache. - Il campione più vecchio è delle 08:34, quindi i dati coprono circa 1h30 e non le 2 ore piene. Il delta riguarda solo questo intervallo. - Non so se `memory_bytes` sia la somma dei 2 pod o una media. In entrambi i casi la percentuale di variazione resta valida. **Raccomandazioni** - `demo-app` ha CPU a 0, rete a 0 e disco a 0 per tutto il periodo. Il carico è idle e la memoria è stabile. - Puoi valutare di ridurre le repliche da 2 a 1, se non servono per alta disponibilità. Se la ridondanza è richiesta, lasciale come sono. - I 14,84 MB sono un footprint molto basso, quindi non c'è rischio di saturazione della RAM. - Per le risorse, imposta request e limit di memoria in modo coerente con un uso intorno ai 15 MB, con un margine adeguato.

GPT-4o Mini	29.99 s (tot: 83.68 s)	3673	268	3941	0.00	L'applicazione con il maggior footprint di RAM è **demo-app**, che attualmente utilizza **14.84 MB** di memoria. Analizzando la serie storica delle metriche di memoria per **demo-app** nelle ultime 2 ore, abbiamo i seguenti dati: - **Memoria all'inizio del periodo** (alle 08:34:33): **15.52 MB** - **Memoria alla fine del periodo** (alle 10:04:44): **14.84 MB** Ora calcoliamo il delta percentuale di memoria: \[\text{Delta Percentuale} = \left(\frac{\text{Memoria Finale} - \text{Memoria Iniziale}}{\text{Memoria Iniziale}} \right) \times 100 \] \[\text{Delta Percentuale} = \left(\frac{14.84 - 15.52}{15.52} \right) \times 100 \approx -4.39\% \] Quindi, la memoria della **demo-app** ha registrato una diminuzione di circa **4.39%** nelle ultime 2 ore.
GPT-4o	29.94 s (tot: 83.63 s)	3673	253	3926	0.00	L'applicazione con il maggior footprint di RAM attualmente è la "demo-app", con un utilizzo di 14.84 MB di memoria. Analizzando la serie storica delle metriche di memoria per la "demo-app" nelle ultime 2 ore, abbiamo i seguenti dati: - 2 ore fa, l'utilizzo di memoria era di 15.52 MB. - Attualmente, l'utilizzo di memoria è di 14.84 MB. Calcoliamo il delta percentuale di memoria: \[\text{Delta percentuale} = \left(\frac{\text{Memoria attuale} - \text{Memoria iniziale}}{\text{Memoria iniziale}}\right) \times 100 \] \[\text{Delta percentuale} = \left(\frac{14.84 - 15.52}{15.52}\right) \times 100 \approx -4.38\% \] Quindi, c'è stata una riduzione del 4.38% nell'utilizzo di memoria della "demo-app" nelle ultime 2 ore.

DOMANDA 7 B

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	174.11 ms	0	0	0	N/A	App: demo-app | Delta: -4.38% (da 15.52 a 14.84 MB)
Gemini 2.5 Flash	50.09 s (tot: 103.78 s)	4569	1019	5588	0.80	L'applicazione `demo-app` è il principale consumatore di memoria con un utilizzo attuale di 14.84 MB. Analizzando lo storico delle metriche, circa due ore fa (`2026-10-30T08:34:33.917000`), l'utilizzo di memoria di `demo-app` era di 15.52 MB. Confrontando il dato attuale con quello di due ore fa, l'utilizzo di RAM di `demo-app` è diminuito del **4.38%**.
Gemini 3.6 Flash	50.60 s (tot: 104.29 s)	4363	1450	5813	0.00	Dalle metriche istantanee correnti, l'applicazione con il maggior consumo di RAM (top consumer) è **`demo-app`**, con un utilizzo attuale di **14.84 MB** (15.560.704 bytes). --- ### Analisi del Variazione dell'utilizzo di RAM per `demo-app`: 1. **Dato di 2 ore fa** (timestamp `2026-10-30T08:34:33`): - **RAM usata:** `15.52 MB` (`16.269.312 bytes`) 2. **Dato Attuale** (timestamp `2026-10-30T10:04:44`): - **RAM usata:** `14.84 MB` (`15.560.704 bytes`) --- ### Calcolo della Variazione Percentuale: $$\text{Variazione \%} = \frac{15.560.704 - 16.269.312}{16.269.312} \times 100 \approx -4.36\%$$ --- ### Esito: L'utilizzo di memoria RAM per **`demo-app`** è **diminuito del 4,36%** (circa **-0.68 MB**) rispetto a 2 ore fa.

Claude Opus 5.5	56.73 s (tot: 110.42 s)	7074	1339	8413	0.00	# Variazione RAM di demo-app nelle ultime ~2 ore **In sintesi:** la memoria è scesa di circa **4,36%**. Nello storico però non c'è un campione di esattamente due ore fa: il confronto parte dal dato più vecchio disponibile, di circa 1h30 fa. **Top consumer di memoria:** dallo snapshot attuale è **demo-app**, con 14,84 MB distribuiti su 2 pod. ## Confronto | | Data/ora | Memoria | |---|---|---| | Campione più vecchio disponibile | 30/10 08:34:33 | 16.269.312 B (**15,52 MB**) | | Dato attuale (ultimo campione) | 30/10 10:04:44 | 15.560.704 B (**14,84 MB**) | | **Differenza** | | **-708.608 B (-0,68 MB)** | **Variazione percentuale: -4,36%** ## Limite del confronto - L'ultimo dato salvato è delle 10:04, quindi "due ore fa" corrisponde alle 08:04 circa. - Il record più vecchio nel database è delle **08:34**, cioè circa **1h30 fa**. - Il valore di -4,36% copre quindi circa 90 minuti, non 120. Se ti serve il dato esatto di due ore fa, puoi chiederlo a Prometheus con `offset 2h`, ad esempio `sum(container_memory_working_set_bytes{pod=~"demo-app-.*"} offset 2h)`. ## Andamento nel periodo - **08:34 → 08:44:** scende da 15,52 a 14,88 MB. Quasi tutto il calo avviene in questi dieci minuti. - **08:44 → 10:04:** resta quasi fermo, tra 14,88 e 14,84 MB, con 2 pod sempre attivi. - Non ci sono segnali di memory leak, cioè di memoria che cresce senza essere liberata. L'uso è stabile e leggermente in calo. ## Raccomandazione SRE In tutti i campioni demo-app usa **0 core di CPU e 0 B/s di rete**: il carico è **inattivo**. Ti suggerisco di: - **Ridurre le repliche da 2 a 1**, se l'app non ha bisogno di alta disponibilità. - **Abbassare le resource requests** a valori vicini all'uso reale (circa 16 MB di RAM per pod), così da liberare capacità sul cluster. - Controllare se l'app riceve davvero traffico. Rete a zero in modo costante può voler dire un servizio non usato o un endpoint configurato male.
Claude Sonnet 5.5	52.10 s (tot: 105.79 s)	7023	1121	8144	0.00	**Il consumo di RAM di demo-app è sceso di circa il 4,4% (da 15,52 MB a 14,84 MB).** Il dato di due ore fa è approssimato, perché lo storico non ha un campione esattamente a quell'ora. Il maggior consumatore di memoria è `demo-app` con 14,84 MB (15.560.704 byte). Gli altri sono `worker-app` con 5,02 MB e `cache-app` con 1,33 MB. | Punto di confronto | Timestamp | Memoria | |---|---|---| | Più vecchio campione disponibile | 08:34:33 | 16.269.312 byte (15,52 MB) | | Attuale | 10:04:44 | 15.560.704 byte (14,84 MB) | Variazione = (15.560.704 − 16.269.312) / 16.269.312 ≈ **−4,36%**, cioè circa −0,7 MB. - **Campione di riferimento:** il più vecchio campione disponibile risale a circa 1h30 prima, non a 2 ore. Prima delle 08:34 non c'erano dati, quindi ho usato quello come riferimento. - **Totale sui pod:** le metriche sono il totale dei 2 pod di `demo-app`, che non sono cambiati nel periodo. **Trend e raccomandazioni** - Il consumo è molto stabile e leggermente in calo. È passato da 15,52 MB a 14,88 MB nei primi 10 minuti e poi si è assestato a 14,84 MB, senza segni di memory leak. - La CPU è a 0 e non c'è traffico di rete né scritture su disco per tutto il periodo. `demo-app` risulta quindi idle. - Se il servizio non serve davvero, puoi ridurre le repliche da 2 a 1. Altrimenti lascia le 2 repliche come ridondanza. In ogni caso non c'è nessuna pressione sulla memoria che richieda scaling.
GPT-4o Mini	53.75 s (tot: 107.44 s)	4842	267	5109	0.00	Per l'applicazione **demo-app**, abbiamo i seguenti dati di utilizzo della memoria: - **Utilizzo di memoria due ore fa**: 16,269,312 bytes (circa 15.52 MB) - **Utilizzo di memoria attuale**: 15,560,704 bytes (circa 14.84 MB) Ora calcoliamo la variazione percentuale dell'utilizzo della memoria: 1. **Differenza di memoria**: \[16,269,312 - 15,560,704 = 708,608 \text{ bytes} \] 2. **Variazione percentuale**: \[\left(\frac{708,608}{16,269,312} \right) \times 100 \approx 4.35\% \] Quindi, l'utilizzo della memoria per **demo-app** è diminuito di circa **4.35%** rispetto a due ore fa.

GPT-4o	45.97 s (tot: 99.66 s)	3679	193	3872	0.00	Due ore fa, l'applicazione "demo-app" utilizzava 15.52 MB di memoria. Attualmente, utilizza 14.84 MB di memoria. Per calcolare la variazione percentuale nell'utilizzo della memoria: \[\text{Variazione percentuale} = \left(\frac{\text{Memoria attuale} - \text{Memoria passata}}{\text{Memoria passata}}\right) \times 100 \] \[\text{Variazione percentuale} = \left(\frac{14.84 - 15.52}{15.52}\right) \times 100 \approx -4.38\% \] Quindi, l'utilizzo della memoria è diminuito di circa il 4.38% rispetto a due ore fa.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $1.3775 (mini: $0.0620 | 4o: $1.3156)
```

DOMANDA 8

```
ESECUZIONE DOMANDA [8]: Assumendo che l'applicazione a maggior carico di CPU abbia un limit di 300m per pod (600m totali su 2 repliche), calcola la percentuale di saturazione e valuta il rischio di throttling.
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 11:47:47] INFO     Processing request of type CallToolRequest                                                
```

Claude Opus 5.5	33.00 s (tot: 105.90 s)	6143	2214	8357	0.90	**worker-app** ha il carico CPU più alto (0,566 core in totale, contro 0,016 di cache-app e 0 di demo-app). Con il limit ipotizzato di 300m per pod è **saturo al 94%** e il **throttling è già in corso**: circa il 96% dei periodi CPU viene bloccato dal kernel. ## Saturazione rispetto al limit di 300m **Aggregato (dallo snapshot):** - Consumo: 0,5657 core = **565,7m** - Limit totale: 600m - **Saturazione: 565,7 / 600 = 94,3%** - Margine residuo: circa 34m, cioè meno del 6% **Per pod (dalla query Prometheus sugli ultimi 5 minuti):** | Pod | Consumo CPU | Limit | Saturazione | |---|---|---|---| | worker-app-84cf4dd446-9kj75 | 295,6m | 300m | **98,5%** | | worker-app-84cf4dd446-ftlln | 285,7m | 300m | **95,2%** | Il carico è distribuito in modo uniforme sulle due repliche, ed entrambe sono praticamente al limite. ## Rischio di throttling: critico, già in corso Ho controllato le metriche del kernel che contano quante volte la CPU di un container viene bloccata per aver superato il limit (`container_cpu_cfs_throttled_periods_total` rispetto a `container_cpu_cfs_periods_total`, sugli ultimi 5 minuti): | Pod | Periodi con throttling | |---|---| | ftlln | **96,6%** | | 9kj75 | **96,2%** | **Cosa significa:** - In quasi ogni intervallo di scheduling (100 ms) i container esauriscono la CPU concessa e restano fermi fino all'intervallo successivo. - Il consumo misurato (circa 290m) non è la domanda reale: è il tetto imposto dal limit. La domanda vera è probabilmente **più alta e non misurabile** finché il limit resta così. - Il throttling registrato indica che un limit CPU è effettivamente configurato. Il fatto che il consumo si fermi intorno a 290m è compatibile con un valore di circa 300m, ma senza leggere lo spec del Deployment non posso confermarlo. - Conseguenze probabili: latenza più alta, job che durano di più, code che crescono e possibili timeout delle probe se il processo non risponde in tempo. **La memoria non è un problema:** 4,35 MB in totale, nessun rischio di OOM. Il collo di bottiglia è solo la CPU.
						tetto imposto dal limit. La domanda vera è probabilmente **più alta e non misurabile** finché il limit resta così. - Il throttling registrato indica che un limit CPU è effettivamente configurato. Il fatto che il consumo si fermi intorno a 290m è compatibile con un valore di circa 300m, ma senza leggere lo spec del Deployment non posso confermarlo. - Conseguenze probabili: latenza più alta, job che durano di più, code che crescono e possibili timeout delle probe se il processo non risponde in tempo. **La memoria non è un problema:** 4,35 MB in totale, nessun rischio di OOM. Il collo di bottiglia è solo la CPU. ## Raccomandazioni 1. **Subito: alzare il limit CPU.** Portarlo ad almeno **500m per pod**, in modo da vedere la domanda reale senza il tetto. In alternativa si può togliere il limit e lasciare solo il request, se le policy del cluster lo permettono. 2. **Scalare orizzontalmente con un HPA.** Il carico è uniforme e la memoria trascurabile, quindi aggiungere repliche funziona bene. Indicativamente: `minReplicas: 3`, `maxReplicas: 6`, target CPU al 70%. Con 3 repliche da 300m la saturazione stimata scende a circa il 63%, ma solo se la domanda reale è quella misurata ora. Dato che il throttling la nasconde, è probabile che sia più alta. - Attenzione: l'HPA calcola la percentuale sui **request**, non sui limit. Verificate che il request CPU sia impostato in modo realistico (ad esempio 250m). 3. **Dopo la modifica: controllare di nuovo.** Ripetere la query sul throttling. L'obiettivo è stare **sotto il 10-20%** dei periodi con throttling. Rimisurate anche il consumo, che a quel punto rifletterà la domanda vera, e usatelo per definire request e limit. 4. **Altre opportunità:** demo-app ha 2 repliche a 0 CPU, quindi è inattiva. Si può valutare di ridurla a 1 replica, se i requisiti di disponibilità lo consentono, e recuperare capacità per worker-app. **Nota:** non ho guardato lo storico su MongoDB, quindi non so se questa saturazione è stabile o è un picco. Se volete capire meglio la tendenza prima di decidere quanto dimensionare l'HPA, posso recuperare lo storico di worker-app delle ultime ore.
Claude Sonnet 5.5	29.85 s (tot: 102.75 s)	7953	1694	9647	0.95	## Saturazione CPU e rischio di throttling di `worker-app`

Claude Sonnet 5.5	29.85 s (tot: 102.75 s)	7953	1694	9647	0.95	## Saturazione CPU e rischio di throttling di `worker-app` `worker-app` è l'applicazione con più carico CPU. Sta usando circa il 94% del limit ipotizzato e subisce throttling quasi continuo. ### Saturazione | Metrica | Valore | |---|---| | CPU totale (snapshot) | 0,566 core | | Limit totale ipotizzato | 0,600 core (2 × 300m) | | **Saturazione aggregata** | **0,566 / 0,600 ≈ 94,3%** | Per pod, dalla rate a 5 minuti su Prometheus: - `worker-app-84cf4dd446-9kj75`: 0,296 core su 0,300, cioè **~98,5%**. - `worker-app-84cf4dd446-ftlln`: 0,286 core su 0,300, cioè **~95,2%**. Il carico è ben bilanciato tra i due pod, ma entrambi sono vicinissimi al tetto. ### Rischio di throttling: già in corso Il rapporto tra periodi CFS throttled e periodi totali (`container_cpu_cfs_throttled_periods_total` / `container_cpu_cfs_periods_total`) è: - `ftlln`: **~96,6%** - `9kj75`: **~96,1%** In quasi ogni periodo CFS di 100 ms il container esaurisce la quota e viene sospeso. L'uso resta "schiacciato" sotto i 300m proprio perché il limit lo taglia. ### Dati storici (ultime ~6 ore) - Il consumo aggregato oscilla tra 0,52 e 0,59 core e raramente supera 0,59, cioè poco sotto il tetto di 0,60. Ci sono pochi punti sopra la soglia e un valore basso isolato di 0,28 core. - Questo plateau è tipico di un carico limitato dal limit. - La **domanda reale è probabilmente superiore** a 0,6 core, ma non è misurabile finché il limit la tronca. - La memoria è trascurabile e stabile (4–6 MB), quindi il collo di bottiglia è solo la CPU. ### Raccomandazioni 1. **Alzare il limit CPU** a 500–600m per pod, oppure rimuoverlo e lasciare solo la request. Poi ricontrollare il consumo reale per dimensionare meglio. 2. **Configurare un HPA** su CPU, con target intorno al 70% della request e `minReplicas: 2`. Con un limit basso, aggiungere repliche scala quasi linearmente. 3. **Impostare le request** vicino all'uso reale (~300m), così lo scheduler e l'HPA lavorano su valori coerenti. 4. **Monitorare la latenza e le code** del worker. Con il 96% di periodi throttled è probabile che ci siano ritardi nell'elaborazione. ### Note - Il limit di 300m è un'**assunzione** dell'operatore e non l'ho verificato sul cluster. Il throttling misurato è però coerente con quel valore. - Il limit effettivo resta da confermare con `kubectl describe pod` o dal manifest.

GPT-4o Mini	5.43 s (tot: 78.33 s)	1270	421	1691	0.95	Per calcolare la percentuale di saturazione della CPU per l'applicazione "worker-app", che ha un carico di CPU attuale di circa 0.566 core, e considerando che ha un limite di 300m (0.3 core) per pod e 600m (0.6 core) totali su 2 repliche, possiamo utilizzare la seguente formula: \[\text{Percentuale di saturazione} = \left(\frac{\text{Carico attuale}}{\text{Limite totale}} \right) \times 100 \] Dove: - Carico attuale = 0.566 core - Limite totale = 0.6 core Calcoliamo la percentuale di saturazione: \[\text{Percentuale di saturazione} = \left(\frac{0.566}{0.6} \right) \times 100 \approx 94.33\% \] ### Valutazione del rischio di throttling Con una percentuale di saturazione della CPU di circa 94.33%, l'applicazione "worker-app" è molto vicina al limite di CPU. Questo comporta un rischio significativo di throttling, specialmente se ci sono picchi di carico o se altre operazioni richiedono più risorse CPU. ### Raccomandazioni 1. **Scaling Orizzontale**: Considerare l'implementazione di un Horizontal Pod Autoscaler (HPA) per "worker-app" per gestire automaticamente il numero di repliche in base al carico di CPU. 2. **Aumento dei Limiti di CPU**: Valutare l'opportunità di aumentare i limiti di CPU per le repliche di "worker-app" per ridurre il rischio di throttling. 3. **Monitoraggio Continuo**: Implementare un monitoraggio continuo per tenere traccia delle metriche di CPU e memoria, in modo da poter reagire rapidamente a eventuali picchi di carico.

GPT-4o	4.62 s (tot: 77.51 s)	1270	365	1635	0.92	Per valutare la percentuale di saturazione della CPU e il rischio di throttling per l'applicazione con il maggior carico di CPU, seguiamo questi passaggi: 1. **Identificazione dell'applicazione con il maggior carico di CPU:** - Dalle metriche istantanee, l'applicazione "worker-app" ha il maggior carico di CPU con 0.5657 core utilizzati. 2. **Calcolo della percentuale di saturazione:** - Il limite di CPU per pod è 300m (0.3 core), quindi per 2 repliche il limite totale è 600m (0.6 core). - La percentuale di saturazione è calcolata come: \(\(\text{CPU utilizzata} / \text{CPU limite}\) \times 100\). 3. **Calcolo:** - CPU utilizzata: 0.5657 core - CPU limite totale: 0.6 core - Percentuale di saturazione: \(\(0.5657 / 0.6) \times 100 \approx 94.28\%\) 4. **Valutazione del rischio di throttling:** - Con una saturazione del 94.28%, l'applicazione "worker-app" è molto vicina al limite di CPU. Questo indica un alto rischio di throttling, poiché il carico di lavoro è quasi al massimo consentito dalle risorse allocate. **Raccomandazione:** - Considerare l'aumento del limite di CPU per pod o l'implementazione di un Horizontal Pod Autoscaler (HPA) per gestire meglio i picchi di carico e ridurre il rischio di throttling.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.3848 (mini: $0.0624 | 4o: $1.3224)

Gemini 2.5 Flash	5.87 s (tot: 167.71 s)	1572	548	2120	0.72	L'applicazione con il maggior carico di CPU è `worker-app`, con un consumo di 0.4346 core (equivalenti a 434.6 millicores). Dato un limite di CPU di 300m per pod e 600m totali su 2 repliche per `worker-app`: Percentuale di saturazione CPU: (434.6 millicores / 600 millicores) * 100 = 72.43% Valutazione del rischio di throttling: Con una saturazione del 72.43%, `worker-app` si avvicina al limite di CPU configurato. Sebbene non sia ancora al 100%, un utilizzo così elevato indica un rischio moderato di throttling, specialmente durante picchi di carico imprevisti. Si consiglia di monitorare attentamente l'applicazione e considerare un aumento dei limiti di CPU o l'implementazione di un HPA se il carico dovesse aumentare ulteriormente.

DOMANDA 8 A

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [8A]: Prendendo l'applicazione più intensiva su CPU e ipotizzando una quota limite di 300 millicore a pod (600m sul deployment), a che percentuale del budget siamo e quanto è concreto il pericolo di CPU throttling?
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3988: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 11:56:34] INFO     Processing request of type CallToolRequest                                                
```

Gemini 3.6 Flash	58.03 s (tot: 106.74 s)	1585	2126	3711	0.95	Sulla base delle metriche istantanee e della topologia fornite nello snapshot: --- ### 1. Applicazione più intensiva su CPU L'applicazione a maggior consumo di CPU è **`worker-app`**, con un consumo complessivo pari a **~0,5939 CPU cores** (ovvero **~593,9 millicore**). --- ### 2. Calcolo della percentuale di utilizzo del budget * **Quota limite per Pod:** 300m (0,300 CPU cores) * **Numero di Pod attivi:** 2 (`worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`) * **Budget complessivo del Deployment:** $300\text{m} \times 2 = 600\text{m}$ (0,600 CPU cores) * **Consumo attuale:** ~593,93 millicore $$\text{Percentuale di utilizzo} = \frac{593,93\text{m}}{600\text{m}} \times 100 \approx \mathbf{98,99\%}$$ Siamo attualmente a **~99% del budget totale assegnato al Deployment**. --- ### 3. Valutazione del pericolo di CPU Throttling Il pericolo di **CPU Throttling è estremamente concreto, imminente e quasi certamente già in atto**: 1. **Quasi saturazione della quota totale (98.99%):** I pod operano praticamente al margine massimo consentito dal limit ipotizzato. 2. **Sbilanciamento del carico tra i Pod:** Se il carico non è perfettamente bilanciato 50/50 tra i due pod (ovvero ~297m ciascuno), anche una minima disuguaglianza porta il pod con più carico a superare la soglia dei 300m, attivando immediatamente il meccanismo di throttling delle quote CFS di Linux. 3. **Impatto sulle prestazioni:** Trattandosi di un carico worker, il CPU throttling causerà un drastico incremento dei tempi di elaborazione dei task e un accumulo di code/latenza. --- ### Raccomandazioni SRE 1. **Scaling Orizzontale (HPA):** Aumentare il numero di repliche di `worker-app` (es. portarlo da 2 a 3 o 4 pod) per distribuire l'elaborazione ed evitare che i pod lavorino costantemente a ridosso dei 300m. 2. **Aumento dei Limits:** Se il lavoro del singolo task richiede elevata capacità di calcolo monothread, valutare un incremento della CPU limit a **450m–500m** per pod. 3. **Verifica metriche cgroup:** Verificare su Prometheus la metrica `container_cpu_cfs_throttled_periods_total` per quantificare esattamente la percentuale di periodi CPU tagliati dal kernel.

Claude Opus 5.5	39.66 s (tot: 88.37 s)	9979	2113	12092	0.90	Siamo praticamente al **99% del budget**, e il throttling non è solo un rischio: **sta già succedendo**, su circa il 90% dei periodi CFS di entrambi i pod. **Applicazione più intensiva su CPU: worker-app** Usa 0,594 core, contro 0,012 di cache-app e 0 di demo-app. **Uso rispetto al budget** | Fonte | Uso CPU | Budget | % del budget | |---|---|---|---| | Snapshot (totale deployment) | 594m | 600m | **99,0%** | | Media Prometheus 5 minuti, pod `9kj75` | 267m | 300m | 89,0% | | Media Prometheus 5 minuti, pod `ftlln` | 268,5m | 300m | 89,5% | Ho controllato su Prometheus: **il limite di 300m per pod è già configurato davvero** (`kube_pod_container_resource_limits` = 0,3 su entrambi i pod). Quindi la tua ipotesi corrisponde alla configurazione reale. Il carico è distribuito in modo uniforme tra i due pod, quindi il problema non è un pod sbilanciato: è la capacità complessiva che non basta. **Quanto è concreto il throttling** Ho misurato la quota di periodi CFS in cui il pod è stato frenato (`throttled_periods / periods`, media su 5 minuti): - `worker-app-84cf4dd446-ftlln`: **92,8%** - `worker-app-84cf4dd446-9kj75`: **89,8%** In circa 9 periodi CFS su 10 (da 100 ms) il kernel ferma i container perché hanno esaurito la quota. Inoltre l'uso misurato è "schiacciato" dal limite: **la domanda reale di CPU di worker-app quasi certamente supera i 600m**, ma non possiamo vederla finché il limite la taglia. Le conseguenze tipiche sono più latenza, code di lavoro che si allungano e throughput ridotto. **Raccomandazioni** 1. **Subito, scalare in orizzontale:** passare almeno a **3 repliche** (900m in totale, circa 66% di uso con il carico osservato) o meglio a **4** (1200m). Con 4 repliche resta margine per la domanda che oggi il throttling nasconde. 2. **HPA su worker-app:** impostare una soglia di CPU intorno al **70%** della request, con `minReplicas: 3` e un `maxReplicas` adatto alla capacità dei nodi (per esempio 6). Serve che le CPU request siano definite. 3. **In alternativa, alzare il limite per pod** a 500m. Conviene se il worker è legato alla CPU e non si parallelizza bene su più repliche. 4. **Verifica dopo la modifica:** il throttling dovrebbe scendere sotto il 10–20%. Se l'uso CPU sale subito dopo aver tolto il vincolo, quella è la domanda reale e va usata per tarare le repliche. Le altre due app sono ben lontane dal problema: cache-app è a 12m e demo-app risulta inattiva (0 CPU). demo-app ha 2 repliche: si può valutare di ridurle, dopo aver controllato lo storico.

Claude Sonnet 5.5	32.32 s (tot: 81.03 s)	8163	2156	10319	0.95	# Worker-app: utilizzo CPU rispetto al limite ipotizzato (300m per pod, 600m sul deployment) **worker-app è a circa il 99% del budget, e il rischio di throttling è concreto.** ## Utilizzo rispetto al budget | Fonte | CPU worker-app | % del budget (600m) | |---|---|---| | Snapshot istantaneo | 0,594 core (circa 297m per pod) | **~99%** | | Prometheus, media su 5 min | 0,268 + 0,267 = 0,535 core | **~89%** (circa 267m per pod) | - **Distribuzione tra i pod:** il carico è bilanciato (`ftlln` 0,2686 core, `9kj75` 0,2670 core). Il throttling si applica per singolo pod, ma qui nessun pod è in una situazione peggiore dell'altro. - **Storico:** nelle ultime ~2,5 ore il consumo sta quasi sempre tra 0,53 e 0,59 core. Il valore più frequente è 0,591 e non viene mai superato. Il carico quindi non è un picco ma costante, vicino al tetto. - **Altre applicazioni:** cache-app usa 0,012 core e demo-app 0. Worker-app è l'unica con un carico CPU significativo. ## Rischio di throttling - `container_cpu_cfs_throttled_periods_total` mostra circa 18 periodi throttlati al secondo su ciascun pod (18,4 e 18,1). Questo dato dipende dal limite CFS realmente configurato, non da quello ipotizzato. Significa che oggi i pod subiscono già throttling. - Il consumo si ferma a circa 0,59 core totali (circa 0,295 per pod) e non lo supera mai. Questo tetto è coerente con un limite effettivo vicino a 300m per pod. - Il valore di 18 periodi/s è superiore a quello atteso con il periodo CFS di default (100 ms, cioè al massimo 10 periodi/s). Può dipendere da un periodo CFS diverso o da più serie aggregate. Per questo non ho calcolato una percentuale di periodi throttlati. Se il limite di 300m non è ancora applicato, un carico che oggi sta già a circa 297m per pod lo raggiungerebbe subito. Il throttling sarebbe quindi continuo, con latenza maggiore sui job del worker. ## Raccomandazioni 1. **Limite:** non fissare 300m a pod. Con una domanda di circa 270–300m, un limite di 500m per pod darebbe circa il 40% di margine. In alternativa, togliere il limite CPU e tenere solo la request. 2. **Request:** impostare circa 300m, in linea con il consumo reale. 3. **HPA:** configurare un HPA su worker-app, con target CPU al 70% della request. Con request di 300m si scala quando si superano circa 210m per pod. Le repliche potrebbero andare da 2 a 4. 4. **Verifica:** controllare con `kubectl describe pod` i limiti realmente applicati. Poi misurare `container_cpu_cfs_throttled_periods_total` rispetto a `container_cpu_cfs_periods_total` per ottenere la percentuale effettiva di throttling.

GPT-4o Mini	4.14 s (tot: 52.85 s)	1273	350	1623	0.90	Analizzando le metriche attuali, l'applicazione più intensiva su CPU è la **worker-app**, che utilizza attualmente circa **0.5939 CPU cores**. ### Calcolo della percentuale di utilizzo della CPU 1. **Quota limite per pod**: 300 millicore = 0.3 CPU cores 2. **Quota limite per il deployment (2 pod)**: 600 millicore = 0.6 CPU cores 3. **Utilizzo attuale della worker-app**: 0.5939 CPU cores ### Percentuale di utilizzo \[\text{Percentuale di utilizzo} = \left(\frac{\text{Utilizzo attuale}}{\text{Quota limite}} \right) \times 100 \] \[\text{Percentuale di utilizzo} = \left(\frac{0.5939}{0.6} \right) \times 100 \approx 98.98\% \] ### Rischio di CPU Throttling Con un utilizzo della CPU attuale di circa **98.98%**, il rischio di CPU throttling è molto concreto. Se l'applicazione dovesse necessitare di ulteriore CPU, potrebbe subire limitazioni significative, portando a prestazioni degradate. ### Raccomandazione Si consiglia di monitorare attentamente l'utilizzo della CPU della **worker-app** e considerare l'implementazione di un Horizontal Pod Autoscaler (HPA) per scalare orizzontalmente il numero di pod, in modo da distribuire meglio il carico e ridurre il rischio di throttling.

GPT-4o	3.51 s (tot: 52.22 s)	1273	299	1572	0.90	L'applicazione più intensiva su CPU è la "worker-app", che attualmente utilizza 0.5939 core, ovvero 593.9 millicore. Dato che il limite di CPU per pod è di 300 millicore, e ci sono due pod attivi per "worker-app", il limite totale di CPU per il deployment è di 600 millicore. Calcoliamo la percentuale di utilizzo rispetto al limite totale: \[\text{Percentuale di utilizzo} = \left(\frac{593.9}{600}\right) \times 100 \approx 98.98\% \] La "worker-app" sta utilizzando circa il 98.98% del budget di CPU disponibile. Questo è molto vicino al limite, il che significa che c'è un rischio concreto di CPU throttling, specialmente se la richiesta di CPU dovesse aumentare anche leggermente. Raccomandazioni: 1. **Scaling Orizzontale**: Considerare l'aggiunta di ulteriori repliche per distribuire il carico di CPU e ridurre il rischio di throttling. 2. **Ottimizzazione**: Verificare se ci sono opportunità di ottimizzazione del codice per ridurre l'utilizzo della CPU. 3. **Monitoraggio**: Continuare a monitorare l'utilizzo della CPU per rilevare eventuali picchi che potrebbero causare throttling.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.3914 (mini: $0.0628 | 4o: $1.3286)

DOMANDA 8B

```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [88]: Valuta la percentuale di utilizzo della CPU rispetto a un limite teorico di 0.3 core per pod (0.6 core totali) per il servizio più caricato: sussistono i presupposti per tagli di cicli da parte dello scheduler CFS?
--------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
```

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	38.37 ms	0	0	0	N/A	App: worker-app | Uso: 0.5939 cores | Saturazione: 98.99% | Rischio: ELEVATO (throttling probabile)
Gemini 2.5 Flash	26.51 s (tot: 75.23 s)	1575	1169	2744	0.95	Il servizio più caricato in termini di CPU è `worker-app`, con un utilizzo di 0.5939 core. Considerando che `worker-app` ha 2 pod e un limite teorico di 0.3 core per pod, il limite totale teorico per `worker-app` è di 0.6 core (0.3 core/pod * 2 pod). La percentuale di utilizzo della CPU per `worker-app` rispetto a questo limite teorico è: (0.5939 core / 0.6 core) * 100 = 98.98% **Raccomandazione:** Il servizio `worker-app` sta utilizzando circa il 98.98% del limite teorico di CPU. Questo indica un utilizzo estremamente elevato delle risorse CPU assegnate. Se il limite di 0.3 core per pod è configurato come `cpu_limit` nei manifest di Kubernetes, sussistono i presupposti per tagli di cicli da parte dello scheduler CFS. Questo significa che i processi all'interno dei pod di `worker-app` potrebbero essere soggetti a throttling, con conseguente riduzione delle prestazioni. Si consiglia di monitorare attentamente le metriche di throttling (es. `container_cpu_cfs_throttled_periods_total`, `container_cpu_cfs_throttled_seconds_total`) per confermare l'impatto. Se il throttling è significativo, si dovrebbe considerare un aumento dei `cpu_limit` per i pod di `worker-app` o l'implementazione di un Horizontal Pod Autoscaler (HPA) basato sull'utilizzo della CPU per scalare orizzontalmente il deployment quando l'utilizzo supera una soglia definita.
Gemini 3.6 Flash	33.57 s (tot: 82.28 s)	6643	2293	8936	0.90	### Analisi del Carico CPU e CFS Throttling #### 1. Identificazione del Servizio Più Caricato Dalle metriche istantanee fornite: * **`worker-app`**: **0.5939 core** totali (distribuiti su 2 pod attivi: `worker-app-84cf4dd446-9kj75` e `worker-app-84cf4dd446-ftlln`) * **`cache-app`**: 0.0125 core * **`demo-app`**: 0.0000 core Il servizio più caricato è **`worker-app`**. --- #### 2. Calcolo della Percentuale di Utilizzo della CPU * **Limite teorico per pod**: 0.300 core * **Limite teorico totale (2 pod)**: $0.300 \times 2 = 0.600\text{ core}$ * **Consumo totale attuale**: 0.5939 core (pari a circa 0.297 core medi per pod) $$\text{Percentuale di Utilizzo} = \left(\frac{0.5939}{0.6000} \right) \times 100 \approx \mathbf{98.98\%}$$ --- #### 3. Valutazione del CFS Throttling (Tagli di Cicli CPU) **Sì, sussistono pienamente i presupposti per il taglio dei cicli CPU da parte dello scheduler CFS (Completely Fair Scheduler).** * **Motivazione Teorica**: L'applicazione sta consumando quasi il **99%** del limite assegnato. Poiché il throttling del CFS agisce su periodi temporali molto brevi (tipicamente intervalli di 100ms), qualsiasi picco di richiesta CPU durante una finestra temporale porta immediatamente il container a saturare la quota prevista, subendo il blocco dei cicli CPU da parte del kernel fino all'inizio del periodo successivo. * **Verifica Sperimentale (Prometheus)**: La verifica della metrica `container_cpu_cfs_throttled_periods_total` conferma che i pod di `worker-app` registrano circa **~9.68-9.69 periodi throttlati al secondo** per ciascun pod. --- ### Raccomandazioni SRE 1. **Aumento dei Limiti CPU (Vertical Scaling)**: Incrementare i `limits.cpu` nei deployment manifest per `worker-app` portandoli ad almeno 0.5 core per pod per evitare la degradazione delle prestazioni e le latenze introdotte dal CFS throttling. 2. **Scaling Orizzontale (HPA)**: Se il carico è distribuito, valutare l'attivazione di un Horizontal Pod Autoscaler con target d'uso CPU attorno al 70-80% per distribuire meglio i task di calcolo.

Claude Opus 5.5	34.29 s (tot: 83.00 s)	6784	1717	8501	0.95	Sì: **worker-app** viene già limitato dallo scheduler CFS, quindi non si tratta solo di un rischio. I dati di Prometheus lo confermano. ## Il servizio più caricato: worker-app | Livello | Uso CPU | Limite | Utilizzo | |---|---|---|---| | Totale deployment (snapshot) | 0,5939 core | 0,6 core | **≈ 99,0%** | | Pod `worker-app-84cf4dd446-ftlln` | 0,2963 core | 0,3 core | **≈ 98,8%** | | Pod `worker-app-84cf4dd446-9kj75` | 0,2960 core | 0,3 core | **≈ 98,7%** | Ho verificato su `kube_pod_container_resource_limits` che il limite di 0,3 core per pod (container `cpu-worker`) è davvero configurato, quindi il calcolo usa un limite effettivo. ## Verifica dei tagli di CPU (throttling CFS) Ho controllato la quota di periodi CFS in cui il pod è stato bloccato (`container_cpu_cfs_throttled_periods_total / container_cpu_cfs_periods_total`, ultimi 5 minuti): - `ftlln`: **96,9%** dei periodi limitati - `9kj75`: **96,7%** dei periodi limitati In quasi ogni periodo da 100 ms i container esauriscono la quota di 30 ms e restano fermi fino al periodo successivo. L'uso stabile appena sotto il limite (circa 0,296 core contro 0,3) è il segno tipico di un carico che vorrebbe più CPU di quella concessa. La domanda reale di CPU è quindi **superiore** a quella misurata, ma da queste metriche non si può stimare di quanto. **Effetti probabili:** l'elaborazione rallenta, la latenza cresce e i job durano di più. Con il throttling, il dato di consumo misurato sottostima il fabbisogno reale. ## Raccomandazioni 1. **Alzare il limite CPU** di `cpu-worker` (per esempio a 0,5–0,6 core per pod). Poi verificare che l'uso si stabilizzi sotto il nuovo limite e che la quota di periodi limitati scenda sotto il 5–10%. 2. **In alternativa o in aggiunta, usare un HPA** su worker-app con soglia CPU intorno al 70% delle request e almeno 3–4 repliche massime. Il carico pare dividersi bene tra i due pod (≈ 0,296 ciascuno), quindi sembra adatto a crescere aggiungendo repliche. 3. **Controllare la capacità del nodo:** entrambi i pod sono su `thesis-cluster-control-plane`. Prima di alzare limiti o repliche, verificare che il nodo abbia CPU allocabile libera. 4. **Gli altri servizi non hanno problemi di CPU:** cache-app usa circa 0,012 core e demo-app è inattiva (0 core, nessun traffico). Per demo-app si può valutare di ridurre le repliche da 2 a 1.
Claude Sonnet 5.5	34.62 s (tot: 83.33 s)	6434	1971	8405	0.95	**Sì, i presupposti per il throttling CFS ci sono, e le metriche mostrano che sta già avvenendo.** **Utilizzo CPU di `worker-app` rispetto al limite teorico di 0,3 core per pod** | Misura | Uso CPU | Limite | Utilizzo | |---|---|---|---| | Totale (snapshot) | 0,594 core | 0,6 core | **~99,0%** | | Pod `…-ftlln` (rate 5m) | 0,296 core | 0,3 core | ~98,8% | | Pod `…-9kj75` (rate 5m) | 0,285 core | 0,3 core | ~95,0% | Gli altri servizi sono lontani dalla saturazione: - `cache-app` usa 0,012 core. - `demo-app` usa 0 core. **Evidenza di throttling (rate su 5 minuti)** | Pod | Periodi throttled/s | Periodi totali/s | Quota throttled | |---|---|---|---| | `…-ftlln` | 19,38 | 19,60 | **~98,9%** | | `…-9kj75` | 19,32 | 19,66 | **~98,3%** | Quasi ogni periodo CFS termina con la quota esaurita. Il carico è CPU-bound e viene tagliato dallo scheduler a ogni periodo. Un uso a ridosso del 100% del limite e quasi tutti i periodi throttled indicano che il container vorrebbe più CPU di quella concessa. Il consumo reale potrebbe quindi essere più alto del valore misurato. **Limiti dell'analisi** - Le metriche di throttling mostrano che nei pod esiste un limite effettivo. Non confermano che sia esattamente 0,3 core. L'uso si ferma poco sotto 0,3 per pod, il che è coerente con quel valore. - Il rapporto di ~19,6 periodi/s è circa il doppio dei 10/s attesi con il periodo CFS predefinito di 100 ms. Può dipendere da un periodo diverso o da più container per pod aggregati nella somma. Non l'ho verificato. **Raccomandazioni** 1. **Alzare il limite CPU** di `worker-app`, ad esempio a 0,5–1 core per pod, e rimisurare. Il consumo reale non è osservabile finché il pod resta throttled. 2. **Aggiungere un HPA** sulla CPU, con target intorno al 60–70% della request. Con repliche a ~99% del limite, scalare orizzontalmente aiuta se il carico è parallelizzabile. Oggi le 2 repliche sono entrambe sature, quindi 2 repliche non bastano. 3. **Se la latenza è critica**, valutare di togliere il limite CPU e tenere solo la request. Questo evita il throttling a costo di un'isolazione minore. 4. **Impostare un alert** su `rate(container_cpu_cfs_throttled_periods_total[5m]) / rate(container_cpu_cfs_periods_total[5m]) > 0.25`. **Memoria di `worker-app`:** 4,77 MB totali, quindi non è un problema. Il collo di bottiglia è solo la CPU.
GPT-5 Mini	5.12 s (tot: 59.33 s)	1888	710	2598	0.90	[illegible]

GPT-4o Mini	5.12 s (tot: 53.83 s)	1280	413	1693	0.28	Analizzando le metriche attuali dei pod: 1. **cache-app**: utilizzo CPU = 0.0125 core 2. **demo-app**: utilizzo CPU = 0.0 core 3. **worker-app**: utilizzo CPU = 0.5939 core Il pod più caricato è il **worker-app**, con un utilizzo di 0.5939 core. ### Calcolo della percentuale di utilizzo della CPU Il limite teorico per ogni pod è di 0.3 core. Poiché il pod **worker-app** ha un utilizzo di 0.5939 core, possiamo calcolare la percentuale di utilizzo rispetto al limite: \[\text{Percentuale di utilizzo} = \left(\frac{\text{Utilizzo attuale}}{\text{Limite teorico}} \right) \times 100 = \left(\frac{0.5939}{0.3} \right) \times 100 \approx 197.97\% \] ### Conclusioni Il pod **worker-app** sta utilizzando circa il 198% della sua capacità teorica di CPU. Questo indica che il pod è sovraccarico e potrebbe beneficiare di un'ottimizzazione. ### Raccomandazioni - **Tagli di cicli da parte dello scheduler CFS**: Sì, ci sono i presupposti per tagli di cicli, poiché il pod sta superando il limite di CPU assegnato. Questo potrebbe portare a una degradazione delle prestazioni. - **Scaling orizzontale**: Considerare l'implementazione di un Horizontal Pod Autoscaler (HPA) per il **worker-app** per gestire meglio il carico di lavoro. - **Monitoraggio**: È consigliabile monitorare le metriche di utilizzo della CPU e della memoria per valutare ulteriormente la necessità di scaling o ottimizzazione.

GPT-4o	6.29 s (tot: 55.00 s)	1280	513	1793	0.95	Per valutare se ci sono i presupposti per tagli di cicli da parte dello scheduler CFS, dobbiamo confrontare l'utilizzo della CPU con il limite teorico di 0.3 core per pod. ### Analisi delle metriche istantanee: 1. **cache-app**: - CPU utilizzata: 0.0125 core - Numero di pod: 1 - Limite teorico totale: 0.3 core - Percentuale di utilizzo: \(\frac{0.0125}{0.3} \times 100 \approx 4.17\% \) 2. **demo-app**: - CPU utilizzata: 0.0 core - Numero di pod: 2 - Limite teorico totale: 0.6 core - Percentuale di utilizzo: \(\frac{0.0}{0.6} \times 100 = 0\% \) 3. **worker-app**: - CPU utilizzata: 0.5939 core - Numero di pod: 2 - Limite teorico totale: 0.6 core - Percentuale di utilizzo: \(\frac{0.5939}{0.6} \times 100 \approx 98.98\% \) ### Conclusioni: - **worker-app** è il servizio più caricato con un utilizzo della CPU del 98.98% rispetto al limite teorico di 0.6 core totali. Questo è molto vicino al limite, il che significa che ci sono buone probabilità che lo scheduler CFS possa iniziare a tagliare i cicli di CPU se l'utilizzo aumenta ulteriormente. - **cache-app** e **demo-app** hanno un utilizzo della CPU molto basso rispetto ai loro limiti teorici, quindi non ci sono preoccupazioni immediate per questi servizi. ### Raccomandazioni: - **worker-app**: Considerare l'implementazione di un Horizontal Pod Autoscaler (HPA) per scalare orizzontalmente il numero di pod se l'utilizzo della CPU continua a rimanere vicino al limite. Questo aiuterà a prevenire il throttling della CPU e a mantenere le prestazioni del servizio. - Monitorare l'utilizzo della CPU nel tempo per identificare eventuali picchi che potrebbero richiedere un'azione immediata.

DOMANDA 9

```
--------------------------------------------------------------------------------
ESECUZIONE DOMANDA [9]: Interroga Prometheus per verificare se ci sono stati riavvii imprevisti di container nel namespace default nell'ultima ora.
--------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio

| Sistema | Latenza Q (Tot con ETL) | In Tok | Out Tok | Tot Tok | Accuratezza | Risposta Completa |
|---|---|---|---|---|---|---|
| Script Deterministico | 173.28 ms | 0 | 0 | 0 | N/A | 0 riavvii imprevisti rilevati nell'ultima ora |
| Gemini 2.5 Flash | 19.50 s (tot: 68.21 s) | 3161 | 255 | 3416 | 0.50 | Nell'ultima ora, non sono stati rilevati riavvii imprevisti di container nel namespace 'default'. |
```

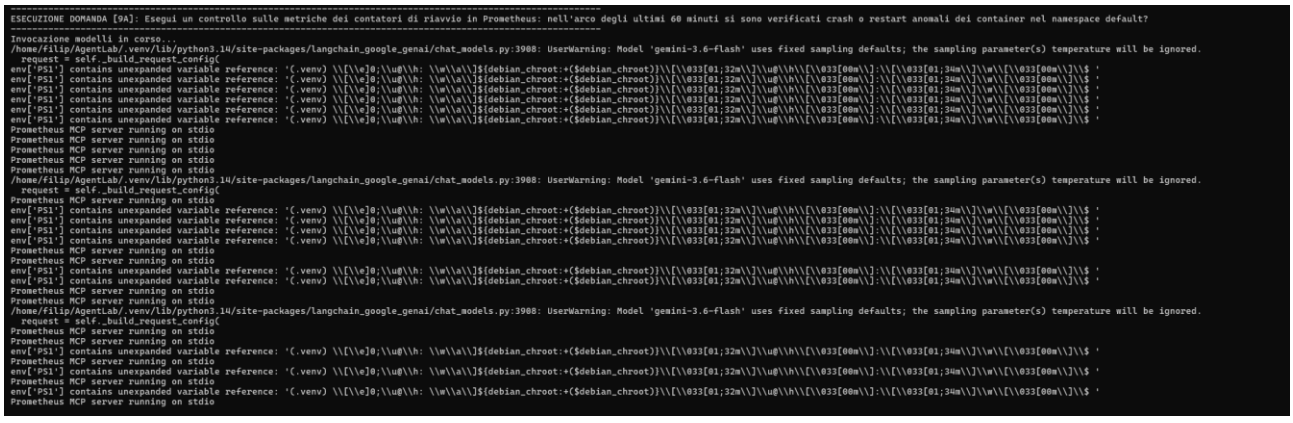
Gemini 3.6 Flash	39.60 s (tot: 88.31 s)	11124	1339	12463	0.80	Dall'analisi delle metriche su Prometheus tramite la query `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])`: * **Esito**: **Nessun riavvio** negli ultimi 60 minuti (`0` riavvii nell'ultima ora per tutti i container). * **Stato Attuale dei Container**: * `worker-app-84cf4dd446-9kj75` (container `cpu-worker`): 0 riavvii nell'ultima ora (riavvii totali storici: 51) * `worker-app-84cf4dd446-ftlln` (container `cpu-worker`): 0 riavvii nell'ultima ora (riavvii totali storici: 51) * `demo-app-7958c64697-2qg9s` (container `nginx`): 0 riavvii nell'ultima ora (riavvii totali storici: 51) * `demo-app-7958c64697-4rzjl` (container `nginx`): 0 riavvii nell'ultima ora (riavvii totali storici: 51) * `cache-app-5c6fd4f5cf-xxsz4` (container `cache-service`): 0 riavvii nell'ultima ora (riavvii totali storici: 51) **Conclusione**: I pod e i container nel namespace `default` risultano stabili e non hanno subito riavvii imprevisti nell'ultima ora.

Claude Opus 5.5	54.70 s (tot: 103.41 s)	16123	2040	18163	0.95	# Verifica riavvii dei container nel namespace `default` **Nell'ultima ora non ci sono stati riavvii.** Per tutti e 5 i container, `increase(kube_pod_container_status_restarts_total[1h])` vale **0**. ## Dati da Prometheus | Pod | Container | Riavvii ultima ora | Riavvii totali | Motivo ultima terminazione | Container avviato da | |---|---|---|---|---|---| | worker-app-…-9kj75 | cpu-worker | 0 | 51 | Unknown | ~2h 32m | | worker-app-…-ftlln | cpu-worker | 0 | 51 | Unknown | ~2h 32m | | demo-app-…-2qg9s | nginx | 0 | 51 | Unknown | ~2h 32m | | demo-app-…-4rzjl | nginx | 0 | 51 | Unknown | ~2h 32m | | cache-app-…-xxsz4 | cache-service | 0 | 51 | Unknown | ~2h 32m | I Pod esistono da circa **38 giorni** (~3,29 milioni di secondi). ## Cosa significano i dati Anche se nell'ultima ora non ci sono stati riavvii, i dati storici sono insoliti: 1. **I container si sono riavviati tutti insieme.** Sono partiti a meno di 4 secondi l'uno dall'altro, circa 2,5 ore fa. Hanno anche lo stesso numero di riavvii totali (51), pur essendo di applicazioni diverse. Se il problema fosse in una singola applicazione, i contatori sarebbero diversi tra loro. 2. **Il motivo dell'ultima terminazione è `Unknown`**, non `OOMKilled` né `Error`. Questo valore compare di solito quando il container non è stato terminato in modo controllato: per esempio se si riavvia il nodo o il runtime dei container (containerd/docker), o se si ferma e riparte un cluster locale (minikube/kind). 3. **Ipotesi più probabile:** riavvii a livello di **nodo o runtime**, avvenuti circa 51 volte in 38 giorni, con l'ultimo circa 2,5 ore fa. Non sembrano crash delle applicazioni. ## Raccomandazioni - **Nessuna azione urgente sulle applicazioni.** Non ci sono riavvii in corso, né OOMKill o loop di crash. - **Verificare il nodo:** usa `kubectl describe node`, controlla l'uptime del nodo (`node_boot_time_seconds`, se è presente node-exporter) e i log di kubelet e containerd intorno a 2,5 ore fa. Se il cluster è di sviluppo e viene fermato e riavviato spesso, questo comportamento è normale. - **Impostare un alert** su `increase(kube_pod_container_status_restarts_total[15m]) > 0`, filtrando per `reason="OOMKilled"` o `"Error"`, così da distinguere i crash veri dai riavvii del nodo. - **Tenere d'occhio `worker-app`:** usa circa 0,59 core in totale (~0,3 per replica) ed è l'unico carico rilevante. Per ora non è collegato ai riavvii, ma se ricomparissero terminazioni `OOMKilled` o `Error` sarebbe il primo da controllare. `demo-app` invece non usa CPU (0 core) ed è inattiva: con 2 repliche si può valutare di ridurle. Se vuoi capire la causa esatta dei 51 riavvii, servono log ed eventi del nodo, che Prometheus non fornisce. Questi dati vanno raccolti con `kubectl` o dai log del sistema.

Claude Sonnet 5.5	34.71 s (tot: 83.42 s)	8906	1042	9948	0.95	**Nell'ultima ora non risultano riavvii imprevisti.** Per tutti i 5 container del namespace `default`, `increase(kube_pod_container_status_restarts_total[1h])` vale 0. | Pod | Container | Riavvii nell'ultima ora | Contatore cumulativo | |---|---|---|---| | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | 51 | | demo-app-7958c64697-2qg9s | nginx | 0 | 51 | | demo-app-7958c64697-4rzjl | nginx | 0 | 51 | | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | 51 | | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | 51 | **Anomalie da verificare** - **Contatore identico:** tutti i container hanno esattamente 51 riavvii cumulativi, anche se appartengono ad applicazioni e pod diversi. Un valore così uniforme fa pensare a un evento comune, per esempio il riavvio del nodo o del cluster, oppure un artefatto dell'ambiente (ad esempio un cluster locale tipo minikube o kind che viene fermato e riavviato). Non è un problema dei singoli workload. - **Motivo dell'ultima terminazione:** `kube_pod_container_status_last_terminated_reason` riporta `Unknown` per tutti i container. Non ci sono quindi segnali di `OOMKilled` o `Error`, ma la causa dei 51 riavvii passati non è determinabile da queste metriche. - **Memoria:** i consumi sono molto bassi (1,66 MB per cache-app, 4,77 MB per worker-app, 14,44 MB per demo-app). Un OOM kill è quindi improbabile. **Prossimi passi** - Controlla gli eventi dei pod e l'ultimo stato terminato con `kubectl describe pod`. Controlla anche gli eventi del nodo con `kubectl get events -n default`. Da qui non ho accesso a questi dati. - Se vuoi capire quando sono avvenuti i 51 riavvii, posso eseguire una query di intervallo su `kube_pod_container_status_restarts_total` su una finestra più ampia, ad esempio 24 ore o 7 giorni.
GPT-4o Mini	28.33 s (tot: 77.04 s)	3895	143	4038	0.00	Non sono riuscito a interrogare Prometheus per verificare i riavvii imprevisti dei container nel namespace "default" nell'ultima ora a causa di un errore nella richiesta. Ti consiglio di controllare direttamente il sistema di monitoraggio o di sottomettere la richiesta a un modello più avanzato che possa gestire query più complesse.

GPT-4o Mini	28.33 s (tot: 77.04 s)	3895	143	4038	0.00	Non sono riuscito a interrogare Prometheus per verificare i riavvii imprevisti dei container nel namespace "default" nell'ultima ora a causa di un errore nella richiesta. Ti consiglio di controllare direttamente il sistema di monitoraggio o di sottomettere la richiesta a un modello più avanzato che possa gestire query più complesse.
GPT-4o	22.24 s (tot: 70.95 s)	2568	356	2924	0.50	Non ci sono stati riavvii imprevisti di container nel namespace "default" nell'ultima ora. Ecco alcune raccomandazioni basate sulle metriche istantanee fornite: 1. **Cache-app**: - **CPU**: Utilizzo molto basso (0.012 cores). - **Memoria**: Utilizzo molto basso (1.66 MB). - **Raccomandazione**: Considerando l'utilizzo estremamente basso di risorse, potrebbe essere opportuno ridurre le risorse allocate o valutare se il pod è effettivamente necessario. 2. **Demo-app**: - **CPU**: Nessun utilizzo registrato. - **Memoria**: Utilizzo moderato (14.44 MB). - **Raccomandazione**: Con due pod attivi e nessun utilizzo di CPU, potrebbe essere utile ridurre il numero di repliche se non ci sono picchi di carico previsti. 3. **Worker-app**: - **CPU**: Utilizzo significativo (0.594 cores). - **Memoria**: Utilizzo moderato (4.77 MB). - **Raccomandazione**: Considerando l'utilizzo di CPU, potrebbe essere utile monitorare se questo è un trend costante e valutare l'implementazione di un Horizontal Pod Autoscaler (HPA) per gestire eventuali picchi di carico. Se desideri ulteriori analisi o verifiche, fammi sapere!

DOMANDA 9 A



Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	161.94 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	16.35 s (tot: 65.06 s)	3922	1116	5038	0.50	Nell'arco degli ultimi 60 minuti, non si sono verificati crash o restart anomali dei container nel namespace `default`. Tutte le metriche di riavvio (`kube_pod_container_status_restarts_total`) mostrano un incremento pari a zero.
Gemini 3.6 Flash	30.10 s (tot: 78.81 s)	7183	1170	8353	0.80	### Esito del controllo dei restart nei contatori Prometheus È stata eseguita una verifica della metrica `kube_pod_container_status_restarts_total` per tutti i pod ed i relativi container appartenenti al namespace `default`. #### Analisi degli ultimi 60 minuti (`increase[60m]`) I risultati indicano che **non si è verificato alcun crash o riavvio** nell'arco dell'ultima ora: * **`cache-app`** (`cache-app-5c6fd4f5cf-xxsz4`, container `cache-service`): **0 riavvii** * **`demo-app`** (`demo-app-7958c64697-2qg9s`, container `nginx`): **0 riavvii** * **`demo-app`** (`demo-app-7958c64697-4rzjl`, container `nginx`): **0 riavvii** * **`worker-app`** (`worker-app-84cf4dd446-9kj75`, container `cpu-worker`): **0 riavvii** * **`worker-app`** (`worker-app-84cf4dd446-ftlln`, container `cpu-worker`): **0 riavvii** --- ### Stato Cumulativo Storico I pod correnti mostrano un valore totale cumulativo di 51 riavvii nello storico complessivo del loro ciclo di vita, ma tutti i contatori sono rimasti **completamente stabili ed invariati nelle ultime 60 minuti**. ### Conclusioni I carichi di lavoro nel namespace `default` risultano **stabili** e non presentano fenomeni di CrashLoopBackOff o riavvii anomali recenti.
Claude Opus 5.5	51.41 s (tot: 100.12 s)	19494	2370	21864	0.95	# Restart dei container nel namespace `default`: ultimi 60 minuti **In breve: nell'ultima ora non c'è stato nessun crash né restart.** I contatori di riavvio, però, mostrano una storia passata che conviene capire. ## 1. Ultimi 60 minuti: nessun restart `increase(kube_pod_container_status_restarts_total{namespace="default"}[60m])` = **0 per tutti e 5 i container**: | Deployment | Pod | Container | Restart (60m) | |---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | | demo-app | demo-app-7958c64697-2qg9s | nginx | 0 | | demo-app | demo-app-7958c64697-4rzjl | nginx | 0 | | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | | worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | Altri controlli: - **Nessun container bloccato in avvio** (`kube_pod_container_status_waiting_reason`): niente CrashLoopBackOff, ImagePullBackOff o simili. - **Tempo dall'ultimo avvio**: tutti i container girano da circa **9.340 s (circa 2h 36m)**. Questo conferma che nell'ultima ora non si sono riavviati. ## 2. Da tenere d'occhio: restart passati - **Contatore totale = 51 per tutti e 5 i container**, cioè lo stesso numero ovunque. - **Causa dell'ultima terminazione = `Unknown`** per tutti. - **Ultimo avvio quasi nello stesso momento**: tutti i container sono partiti entro 4 secondi l'uno dall'altro. - Anche **kube-state-metrics** (il componente che espone questi contatori) ha cambiato indirizzo IP nel tempo: `10.244.0.13` → `.14` → `.3`. Quindi si è riavviato anche lui. **Cosa ne deduco:** questi segnali non fanno pensare a crash delle singole applicazioni. Un crash applicativo darebbe numeri diversi per pod e cause come `Error` o `OOMKilled`. Lo schema fa pensare piuttosto a **riavvii di tutto il nodo o del cluster**, tipici di ambienti come minikube o kind quando si ferma e riavvia il nodo o l'host va in sospensione. Qui il container runtime riavvia tutti i container insieme e Kubernetes non conosce il motivo (`Unknown`). È un'ipotesi: non ho verificato le metriche del nodo. Dal confronto dei dati sulle 24 ore risulta l'ultimo riavvio di questo tipo, circa 2,5 ore fa (fuori dalla finestra richiesta). ## Raccomandazioni

						|---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | | demo-app | demo-app-7958c64697-2qg9s | nginx | 0 | | demo-app | demo-app-7958c64697-4rzjl | nginx | 0 | | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | | worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | Altri controlli: - **Nessun container bloccato in avvio** (`kube_pod_container_status_waiting_reason`): niente CrashLoopBackOff, ImagePullBackOff o simili. - **Tempo dall'ultimo avvio**: tutti i container girano da circa **9.340 s (circa 2h 36m)**. Questo conferma che nell'ultima ora non si sono riavviati. ## 2. Da tenere d'occhio: restart passati - **Contatore totale = 51 per tutti e 5 i container**, cioè lo stesso numero ovunque. - **Causa dell'ultima terminazione = `Unknown`** per tutti. - **Ultimo avvio quasi nello stesso momento**: tutti i container sono partiti entro 4 secondi l'uno dall'altro. - Anche **kube-state-metrics** (il componente che espone questi contatori) ha cambiato indirizzo IP nel tempo: `10.244.0.13` → `.14` → `.3`. Quindi si è riavviato anche lui. **Cosa ne deduco:** questi segnali non fanno pensare a crash delle singole applicazioni. Un crash applicativo darebbe numeri diversi per pod e cause come `Error` o `OOMKilled`. Lo schema fa pensare piuttosto a **riavvii di tutto il nodo o del cluster**, tipici di ambienti come minikube o kind quando si ferma e riavvia il nodo o l'host va in sospensione. Qui il container runtime riavvia tutti i container insieme e Kubernetes non conosce il motivo (`Unknown`). È un'ipotesi: non ho verificato le metriche del nodo. Dal confronto dei dati sulle 24 ore risulta l'ultimo riavvio di questo tipo, circa 2,5 ore fa (fuori dalla finestra richiesta). ## Raccomandazioni 1. **Nessun intervento urgente sulle applicazioni**: nessun container è instabile adesso. 2. **Verificare la stabilità del nodo**: controllare uptime e cronologia di riavvio, ad esempio con `node_boot_time_seconds` o con `kubectl get events` / `kubectl describe node`, per confermare che i 51 restart dipendano da riavvii del nodo. Se l'ambiente è di produzione, 51 riavvii completi meritano un'indagine. 3. **Impostare un alert** del tipo `increase(kube_pod_container_status_restarts_total[15m]) > 0` combinato con `kube_pod_container_status_last_terminated_reason{reason=~"OOMKilled|Error"}`. Così si separano i crash veri delle applicazioni dai riavvii dell'infrastruttura. 4. **worker-app** usa circa 0,59 core su 2 repliche, quindi è l'unico carico che lavora davvero. Al momento non ci sono restart legati al consumo di risorse (nessun `OOMKilled`), ma conviene verificare che abbia limiti di CPU definiti per evitare throttling.

Claude Sonnet 5.5	30.46 s (tot: 79.17 s)	8987	1447	10434	0.95	**Nell'ultima ora non risultano crash né restart dei container nel namespace `default`.** `increase(kube_pod_container_status_restarts_total[60m])` vale **0** per tutti e 5 i pod: | Pod | Container | Restart (60 min) | Contatore cumulativo | |---|---|---|---| | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | 51 | | demo-app-7958c64697-2qg9s | nginx | 0 | 51 | | demo-app-7958c64697-4rzjl | nginx | 0 | 51 | | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | 51 | | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | 51 | **Anomalia da verificare.** Il contatore cumulativo è **51 su tutti i pod**, anche se appartengono a deployment diversi con immagini diverse (nginx, cache, worker). Un valore identico su carichi così diversi di solito non indica bug nelle applicazioni. Più probabilmente c'è una causa comune a livello di nodo o cluster, ad esempio: - riavvii ripetuti del nodo o del runtime (tipico di ambienti minikube o kind); - riavvii del cluster; - un problema nel modo in cui kube-state-metrics riporta il contatore. Anche `last_terminated_reason` è `Unknown` per tutti i container. Questo è coerente con terminazioni non causate dall'applicazione, come OOMKilled o Error, ma non lo dimostra. **Cosa non posso stabilire dai dati.** Questi dati non dicono quando sono avvenuti i 51 restart. So solo che nessuno è avvenuto nell'ultima ora. Se vuoi sapere quando sono avvenuti, posso fare una range query su un intervallo più lungo, per esempio 24 ore o 7 giorni. **Consigli** 1. Esegui `kubectl describe pod` e controlla gli eventi del nodo (`kubectl get events`) per trovare la causa comune. 2. Verifica l'uptime e la stabilità del nodo. 3. Mantieni un alert su `increase(kube_pod_container_status_restarts_total[15m]) > 0` per rilevare subito eventuali nuovi restart. **Altre osservazioni dallo snapshot (nessun segnale di crash)** - `worker-app` usa circa 0,59 core di CPU in totale, con memoria bassa (circa 4,8 MB). È l'unico carico attivo. - `demo-app` è idle: 0 CPU e nessun traffico di rete. - `cache-app` ha CPU molto bassa (0,012 core) e circa 475 KB/s di scritture su disco.
GPT-4o Mini	20.43 s (tot: 69.14 s)	3939	141	4080	0.00	Non sono riuscito a eseguire la query per controllare i contatori di riavvio dei container in Prometheus a causa di un errore nella richiesta. Ti consiglio di verificare direttamente i log di Prometheus o di consultare un modello più avanzato per ottenere informazioni sui riavvii anomali dei container nel namespace default.
GPT-4o	14.71 s (tot: 63.42 s)	2598	101	2699	0.50	Nell'arco degli ultimi 60 minuti, non si sono verificati crash o restart anomali dei container nel namespace "default". Non ci sono metriche di riavvio che superano la soglia di zero, indicando che i container sono stabili e non hanno subito riavvii imprevisti.

DOMANDA 9B

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	135.08 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	30.27 s (tot: 78.98 s)	1554	228	1782	0.00	[ERRORE STEP 1]: Error calling model 'gemini-2.5-flash' (RESOURCE_EXHAUSTED): 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, model: gemini-2.5-flash, description: Generate content requests per day limit exceeded on Free Tier. See https://ai.google.dev/gemini-api/docs/rate-limits.\nPlease retry in 19h48m54.547332966s.', 'status': 'RESOURCE_EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'description': 'Generate content requests per day limit exceeded on Free Tier. See https://ai.google.dev/gemini-api/docs/rate-limits.', 'quotaMetric': 'generativelanguage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-2.5-flash'}, 'quotaValue': '20'}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '71334s'}]}}
Gemini 3.6 Flash	137.43 s (tot: 186.14 s)	7151	1084	8235	0.80	In base all'analisi effettuata sulle metriche di Prometheus per il namespace `default`, **nessun container è andato incontro a riavvii durante l'ultima ora**. --- ### Dettaglio dell'analisi Per verificare l'incremento dei riavvii nell'ultima ora è stata eseguita la query PromQL: `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])` I risultati ottenuti per i pod attivi sono i seguenti: | Pod | Container | Conteggio totale riavvii (cumulativo) | Riavvii nell'ultima ora (`increase 1h`) | | :--- | :--- | :---: | :---: | | `cache-app-5c6fd4f5cf-xxsz4` | `cache-service` | 51 | **0** | | `demo-app-7958c64697-2qg9s` | `nginx` | 51 | **0** | | `demo-app-7958c64697-4rzjl` | `nginx` | 51 | **0** | | `worker-app-84cf4dd446-9kj75` | `cpu-worker` | 51 | **0** | | `worker-app-84cf4dd446-ftlln` | `cpu-worker` | 51 | **0** | --- ### Esito e Considerazioni SRE - **Stabilità recente**: Nonostante la metrica cumulativa storica registri dei riavvii in passato, nelle ultime 60 minuti tutti i container sono rimasti stabili e privi di crash o riavvii improvvisi (es. OOMKilled, Liveness Probe Failure, CrashLoopBackOff).

Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	45.38 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	23.39 s (tot: 69.06 s)	3197	600	3797	0.00	Non è stato possibile recuperare le metriche di restart dei container a causa di un errore nella richiesta a Prometheus (status code 400). Non posso quindi verificare se ci sono stati riavvii improvvisi nell'ultima ora. Si prega di sottomettere la richiesta a un modello più avanzato per un'analisi più approfondita o per la risoluzione del problema con la query Prometheus.
Gemini 3.6 Flash	43.29 s (tot: 88.96 s)	7144	1627	8771	0.95	Dall'analisi effettuata interrogando le metriche di Prometheus per il namespace `default`, **nessun container è andato incontro a riavvii durante l'ultima ora**. --- ### Dettaglio dell'Analisi (PromQL) È stata eseguita la query per calcolare l'incremento del contatore di riavvii nell'ultima ora: `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])` | Pod | Container | Riavvii nell'ultima ora (`[1h]`) | Conteggio Totale Storico | | :--- | :--- | :---: | :---: | | **`worker-app-84cf4dd446-9kj75`** | `cpu-worker` | **0** | 51 | | **`worker-app-84cf4dd446-ftlln`** | `cpu-worker` | **0** | 51 | | **`demo-app-7958c64697-2qg9s`** | `nginx` | **0** | 51 | | **`demo-app-7958c64697-4rzjl`** | `nginx` | **0** | 51 | | **`cache-app-5c6fd4f5cf-xxsz4`** | `cache-service` | **0** | 51 | --- ### Conclusioni e Raccomandazioni SRE - **Stabilità dei Pod:** Tutti i container attivi mantengono un conteggio di riavvii costante (pari a 0 nuovi eventi nell'ultima ora), confermando l'assenza di crash loop improvvisi (es. `OOMKilled` o fallimenti di liveness/readiness probe) nel periodo recente. - **Nota sul totale cumulativo:** Sebbene il totale cumulativo indichi riavvii avvenuti in passato (51 per tutti i pod), lo stato attuale del sistema appare pienamente stabilizzato.

Claude Opus 5.5	40.19 s (tot: 88.90 s)	14125	1837	15962	0.95	**Nell'ultima ora nessun container del namespace `default` si è riavviato.** Ci sono però dei riavvii più vecchi che vale la pena capire. ### Ultima ora Ho calcolato `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])`. Il risultato è **0** per tutti e 5 i container: | Deployment | Pod | Container | Restart ultima ora | |---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | | demo-app | demo-app-7958c64697-2qg9s | nginx | 0 | | demo-app | demo-app-7958c64697-4rzjl | nginx | 0 | | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | | worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | ### Riavvii più vecchi - **Totale:** ogni container ha **51 riavvii**, sempre lo stesso numero per tutti. - **Età dei pod:** tutti i pod esistono da circa **3,29 milioni di secondi (~38 giorni)** e sono stati creati nello stesso momento. - **Ultime 24 ore:** c'è stato **circa 1 riavvio per container**. Per ogni container compaiono due serie: una vale circa 1, l'altra 0. Di solito succede quando si riavvia anche kube-state-metrics, che esporta queste metriche. ### Cosa significa Che i numeri siano identici fra app molto diverse (cache, nginx, worker che usa CPU) indica quasi certamente un **evento che ha colpito tutto il nodo o il cluster**, ad esempio un riavvio del nodo o del container runtime, o di un ambiente locale come kind/minikube. È poco probabile che si tratti di crash delle singole applicazioni, come errori di memoria o probe fallite. Vale la pena ricordare che anche il pod di monitoraggio sembra essere stato riavviato. ### Cosa consiglio 1. **Confermare la causa** con la query `kube_pod_container_status_last_terminated_reason{namespace="default"}`: - se il motivo è `Completed`/`Error` per tutti, conferma un riavvio del nodo; - se è `OOMKilled`, il problema è delle singole app (poco probabile vista la memoria: 1,7–15 MB). 2. **Guardare lo stato del nodo:** uptime del nodo, cioè `node_boot_time_seconds`, ed eventi `NodeNotReady`. 3. **Impostare un alert** su `increase(kube_pod_container_status_restarts_total[15m]) > 0`, così i riavvii futuri vengono segnalati subito. 4. **Tenere d'occhio worker-app (~0,59 core):** non c'entra con i riavvii ma è l'unico carico rilevante. Se ha un limite di CPU troppo basso, la CPU viene rallentata (throttling) e il container non viene riavviato. Al momento i carichi sono stabili e nessun riavvio è in corso.

Claude Sonnet 5.5	23.10 s (tot: 71.81 s)	7668	960	8628	0.95	**Nell'ultima ora nessun container del namespace `default` si è riavviato.** `increase(kube_pod_container_status_restarts_total[1h])` vale 0 per tutti i 5 container. | Pod | Container | Restart nell'ultima ora | Contatore cumulativo | |---|---|---|---| | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | 51 | | demo-app-7958c64697-2qg9s | nginx | 0 | 51 | | demo-app-7958c64697-4rzjl | nginx | 0 | 51 | | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | 51 | | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | 51 | **Anomalia nel contatore cumulativo.** Tutti e 5 i container, in 3 deployment diversi, hanno esattamente 51 restart. Probabilmente non sono crash dei singoli workload. È più facile che siano riavvii collettivi in passato, per esempio un riavvio del nodo o del cluster (la configurazione sembra a nodo singolo, tutto passa da `10.244.0.3`), oppure un problema comune di ambiente. Dalle metriche non posso stabilire quando siano avvenuti. Nell'ultima ora comunque il contatore è fermo. **Cosa fare:** - Per datare i restart passati, controllare `kubectl describe pod` e gli eventi, oppure interrogare `kube_pod_container_status_last_terminated_reason` e `kube_pod_container_status_last_terminated_timestamp`. In alternativa si può fare una `prom_range` su un intervallo più lungo, se la retention lo consente. - Il worker-app usa circa 0,59 core e non ha riavvii. Il carico elevato quindi non sta causando OOM o crash.
GPT-4o Mini	13.64 s (tot: 62.35 s)	6025	163	6188	0.70	Dall'analisi delle metriche di riavvio dei container negli ultimi 60 minuti, non ci sono stati riavvii registrati per i pod nel namespace `default`. Ecco un riepilogo: - **cache-app**: 0 riavvii - **demo-app**: 0 riavvii - **worker-app**: 0 riavvii Tutti i container attivi nel namespace `default` sono stati stabili e non hanno subito riavvii improvvisi nell'ultima ora. Se hai bisogno di ulteriori informazioni o di un'analisi più approfondita, fammi sapere!
GPT-4o	20.23 s (tot: 68.95 s)	3750	169	3919	0.70	Dall'analisi delle metriche di riavvio dei container nell'ultima ora, non risultano incrementi nei conteggi di riavvio per nessuno dei container nei pod attivi nel namespace "default". Questo indica che non ci sono stati riavvii improvvisi per i container delle applicazioni "cache-app", "demo-app" e "worker-app" durante l'ultima ora. Se hai ulteriori domande o necessiti di ulteriori analisi, fammi sapere!

DOMANDA 10

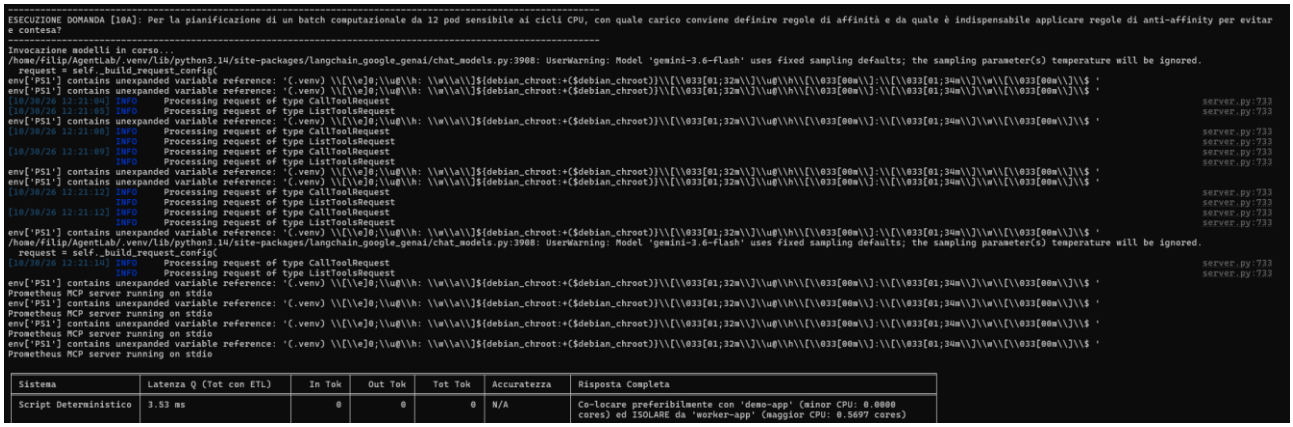
```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [10]: Dobbiamo sottomettere un nuovo batch da 12 pod con vincolo critico di CPU. In base alle risorse attuali, accanto a quale applicazione è più sicuro collocarli e da quale devono essere isolati?
--------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 12:17:52] INFO     Processing request of type CallToolRequest                                                
```

						4. **Precedenza:** una `PriorityClass` dedicata se il batch è critico. 5. **Capacità extra:** demo-app ha 2 repliche con CPU a zero. Se lo zero corrisponde a un'app davvero ferma e non a un dato mancante, potreste ridurla a 1 replica. ## Limiti di questa analisi Lo snapshot riporta i consumi **per applicazione**, non **per nodo**. Non so quindi su quali nodi girano i pod, né quanta CPU è allocabile su ciascun nodo. Prima di sottomettere il batch conviene verificare due cose: - **Capacità libera dei nodi:** `kube_node_status_allocatable{resource="cpu"}` meno le CPU già richieste. - **Posizione dei pod:** `kube_pod_info` per vedere su quali nodi stanno worker-app e demo-app. Se worker-app e demo-app sono sullo stesso nodo, conta solo la regola di separazione da worker-app. Se volete, posso fare io queste query su Prometheus.
Claude Sonnet 5.5	24.68 s (tot: 63.31 s)	11839	1459	13298	0.95	**Raccomandazione** - **Dove collocarli:** accanto a **demo-app**. La sua CPU è 0,0 core in tutte le 17 rilevazioni dello storico, cioè sulle ultime ~2h40 di campionamento. Ha 2 pod e usa circa 14,4 MB di memoria, stabili. Non c'è contesa di CPU da temere. - **Da cosa isolarli:** da **worker-app**. Usa 0,57 core in modo costante, con valori tra 0,28 e 0,59 e un plateau vicino a 0,59. Sembra un carico CPU-bound sostenuto, forse vicino a un limite o a una quota di circa 0,6 core. Questo è un'inferenza dai dati: non ho verificato i limit. Ha 2 pod, quindi circa 0,28 core ciascuno. Se i 12 pod nuovi finissero sugli stessi nodi, rischierebbero throttling e noisy-neighbor proprio sul vincolo critico. - **Cache-app:** è una scelta intermedia. Usa pochissima CPU (circa 0,014 core, tra 0,008 e 0,023) e quasi nessuna memoria (1,6 MB). Però scrive su disco in modo continuo, a circa 450 KB/s. Va bene per la CPU, ma ha un carico di I/O che potrebbe disturbare pod sensibili al disco. Lo userei solo come seconda scelta. **Cautele** - Le metriche mostrano l'uso reale, non requests, limits o capacità dei nodi. Non so su quali nodi girano i pod, né quanta CPU allocabile resta. Prima del deploy controlla questi tre punti: 1. i `requests` di CPU dei 12 pod, perché lo scheduler ragiona su quelli e non sull'uso reale; 2. la capacità allocabile dei nodi; 3. l'assenza di worker-app sugli stessi nodi. - Per l'isolamento non basta l'osservazione: usa `podAntiAffinity` verso `app=worker-app`, oppure un nodeSelector, un taint o una toleration dedicati. In alternativa, `nodeAffinity` verso i nodi di demo-app. - Imposta `requests` uguali ai `limits` sui nuovi pod (QoS Guaranteed), così il vincolo di CPU è garantito. **Nota su demo-app** Con CPU a zero, rete a zero e disco a zero per tutto il periodo, demo-app sembra idle. Se è davvero inutilizzata, si possono ridurre le repliche da 2 a 1 e liberare risorse. Verifica prima che non serva come servizio in standby.

GPT-4o Mini	4.05 s (tot: 42.68 s)	1271	292	1563	0.00	Analizzando le metriche attuali delle applicazioni: 1. **cache-app**: - CPU: 0.0144 cores - Memoria: 1.59 MB 2. **demo-app**: - CPU: 0.0 cores - Memoria: 14.44 MB 3. **worker-app**: - CPU: 0.5697 cores - Memoria: 4.42 MB ### Raccomandazioni: - **Applicazione più sicura per collocare nuovi pod**: - **cache-app** è la scelta migliore per collocare i nuovi pod. Attualmente utilizza solo 0.0144 cores di CPU, il che indica che ha una capacità di CPU molto bassa e quindi può gestire ulteriori carichi senza saturare le risorse. - **Applicazione da isolare**: - **worker-app** deve essere isolata. Utilizza già 0.5697 cores di CPU, che è significativamente più alta rispetto alle altre applicazioni. Aggiungere nuovi pod qui potrebbe portare a saturazione delle risorse CPU, compromettendo le prestazioni. ### Conclusione: Collocare i nuovi pod nella **cache-app** è la scelta più sicura, mentre è consigliabile evitare di collocarli nella **worker-app** per prevenire problemi di performance.

GPT-4o	3.97 s (tot: 42.60 s)	1271	336	1607	0.80	Per determinare dove collocare i nuovi pod con vincolo critico di CPU, analizziamo le metriche istantanee delle applicazioni attuali: 1. **cache-app**: - CPU: 0.0144 cores - Memoria: 1.59 MB - Disco e rete: utilizzo minimo 2. **demo-app**: - CPU: 0.0 cores - Memoria: 14.44 MB - Disco e rete: utilizzo minimo 3. **worker-app**: - CPU: 0.5697 cores - Memoria: 4.42 MB - Disco e rete: utilizzo minimo **Raccomandazioni:** - **Isolamento da worker-app**: Questa applicazione sta utilizzando la maggior parte delle risorse CPU (0.5697 cores), quindi è consigliabile evitare di collocare i nuovi pod accanto a questa applicazione per evitare saturazione della CPU. - **Collocazione accanto a demo-app o cache-app**: Entrambe queste applicazioni hanno un utilizzo di CPU molto basso (0.0 e 0.0144 cores rispettivamente). Pertanto, è più sicuro collocare i nuovi pod accanto a una di queste applicazioni, con una leggera preferenza per **demo-app** dato che non utilizza CPU al momento. In sintesi, colloca i nuovi pod accanto a **demo-app** e isola da **worker-app** per garantire che i vincoli critici di CPU siano rispettati.

DOMANDA 10 A



```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [10A]: Per la pianificazione di un batch computazionale da 12 pod sensibile ai cicli CPU, con quale carico conviene definire regole di affinità e da quale è indispensabile applicare regole di anti-affinity per evitar
e contesa?
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 12:21:04] INFO     Processing request of type CallToolRequest                                                                                                                                  server.py:733
[10/30/26 12:21:05] INFO     Processing request of type ListToolsRequest                                                                                                                                 server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 12:21:08] INFO     Processing request of type CallToolRequest                                                                                                                                  server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                                                 server.py:733
[10/30/26 12:21:09] INFO     Processing request of type CallToolRequest                                                                                                                                  server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                                                 server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 12:21:12] INFO     Processing request of type CallToolRequest                                                                                                                                  server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                                                 server.py:733
[10/30/26 12:21:12] INFO     Processing request of type CallToolRequest                                                                                                                                  server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                                                 server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
[10/30/26 12:21:16] INFO     Processing request of type CallToolRequest                                                                                                                                  server.py:733
                    INFO     Processing request of type ListToolsRequest                                                                                                                                 server.py:733
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio

| Sistema | Latenza Q (Tot con ETL) | In Tok | Out Tok | Tot Tok | Accuratezza | Risposta Completa |
|---|---|---|---|---|---|---|
| Script Deterministico | 3.53 ms | 0 | 0 | 0 | N/A | Co-locare preferibilmente con 'demo-app' (minor CPU: 0.0800 cores) ed ISOLARE da 'worker-app' (maggior CPU: 0.5697 cores) |
```

Gemini 2.5 Flash	4.07 s (tot: 42.70 s)	1573	610	2183	0.72	Per la pianificazione del batch computazionale da 12 pod sensibile ai cicli CPU, ecco le mie raccomandazioni basate sulle metriche attuali: **Opportunità di Affinità (carico basso):** * **`cache-app`**: Con un consumo di CPU di soli 0.014 core, `cache-app` presenta un carico estremamente basso. È un ottimo candidato per definire regole di affinità, permettendo ai pod del batch computazionale di condividere i nodi con questa applicazione senza rischio di contesa significativa per la CPU. * **`demo-app`**: Attualmente non consuma CPU (0.0 core). Anche `demo-app` è un candidato ideale per regole di affinità, in quanto non dovrebbe causare alcuna contesa con il batch computazionale. **Indispensabile Anti-Affinità (carico elevato):** * **`worker-app`**: Con un consumo di CPU di 0.569 core, `worker-app` ha un carico di lavoro significativamente più elevato rispetto alle altre applicazioni. Per evitare contesa e garantire le prestazioni ottimali del batch computazionale sensibile alla CPU, è **indispensabile** applicare regole di anti-affinità per impedire che i pod del batch vengano schedulati sugli stessi nodi di `worker-app`.

Gemini 3.6 Flash	24.27 s (tot: 62.90 s)	5063	1526	6589	0.80	Sulla base dell'analisi delle metriche istantanee e dello storico delle applicazioni nel cluster, ecco le raccomandazioni per la schedulazione del batch computazionale da 12 pod sensibile ai cicli CPU: --- ### 1. **Regole di Affinità (Pod Affinity)**: **`demo-app`** * **Motivazione**: `demo-app` presenta un utilizzo CPU pari a **0.0 cores** (costante anche nello storico recente) e un footprint di memoria ridotto (~14.4 MB per 2 pod). * **Vantaggio**: Co-localizzare i pod del batch sui nodi in cui risiede `demo-app` sfrutta nodi con risorse computazionali completamente libere/idle, minimizzando l'impatto di rumore da vicini ("noisy neighbors"). *(In subordine, anche `cache-app` ha un impatto CPU trascurabile di ~0.014 cores)*. --- ### 2. **Regole di Anti-Affinità (Pod Anti-Affinity)**: **`worker-app`** * **Motivazione**: `worker-app` è un carico attivamente computazionale che consuma circa **0.57 CPU cores** (con un range storico tra 0.43 e 0.59 cores distribuito sui suoi pod). * **Vantaggio**: Applicare regole di anti-affinità rispetta la sensibilità ai cicli CPU del batch, evitando contesa computazionale (*CPU throttling*, *context switching* e degradamento delle prestazioni) che si verificherebbe condividendo i nodi con i pod attivi di `worker-app`. --- ### Riassunto Configurazione Suggerita (`podAffinity` / `podAntiAffinity`): * **`podAffinity`** (o `preferredDuringSchedulingIgnoredDuringExecution`): Target tag/label di `demo-app`. * **`podAntiAffinity`** (`requiredDuringSchedulingIgnoredDuringExecution`): Target tag/label di `worker-app`.

Claude Opus 5.5	56.80 s (tot: 95.44 s)	23534	3202	26736	0.87	**In breve: il carico vicino a cui conviene stare è `demo-app`, quello da cui separarsi è `worker-app`. Però oggi il cluster ha un solo nodo, quindi una regola di anti-affinity obbligatoria lascerebbe i 12 pod in attesa (`Pending`) per sempre.** Ho controllato lo storico delle ultime 6 ore, i dati di Prometheus e la topologia dei nodi. ### Cosa dicono i dati | Carico | CPU (ultime 6h) | Throttling CPU | Altro | Ruolo per il batch | |---|---|---|---|---| | **worker-app** (2 pod) | Costante tra 0,52 e 0,59 core, circa 0,30 core per pod | **~97%** su entrambi i pod | Memoria 4–5 MB | **Va separato (anti-affinity)** | | **demo-app** (2 pod) | **0 core** per tutte le 6 ore | Nessuno (non usa CPU) | 14–15 MB di memoria fissa | **Vicino consigliato (affinity)** | | **cache-app** (1 pod) | 0,008–0,023 core | ~38% | Scrittura su disco costante, circa 450 KB/s | Neutro, al massimo una preferenza a stargli lontano | - **worker-app** consuma CPU senza pause. Nel 97% dei periodi di schedulazione supera il suo limite, che deduco essere intorno ai 300m: non ho letto direttamente i limiti configurati, ma lo indicano un consumo fermo a ~0,30 core e il throttling quasi totale. È il carico CPU-bound per eccellenza del cluster. Un batch sensibile ai cicli CPU messo sugli stessi core subirebbe la sua concorrenza (scheduler del kernel, cache L1/L2, eventuali thread "rumorosi" vicini). Anche worker-app ne soffrirebbe, perché è già al limite. - **demo-app** è completamente inattivo da 6 ore: occupa solo un po' di memoria. Dove gira demo-app, la CPU è libera, quindi è il punto di riferimento giusto per l'affinity. - **cache-app** usa pochissima CPU, ma scrive su disco senza sosta e ha un 38% di throttling, probabilmente per un limite molto basso. Se il batch fa anche I/O, conviene una preferenza a stargli lontano, non una regola obbligatoria.

						### ⚠️ Il problema principale: il cluster ha un solo nodo
						Tutti i 5 pod girano su `thesis-cluster-control-plane`, che ha **8 core allocabili**. Con `topologyKey: kubernetes.io/hostname`:
						- un'anti-affinity obbligatoria (`requiredDuringScheduling...`) verso worker-app **non può essere soddisfatta**, e i 12 pod resterebbero `Pending`;
						- l'affinity verso demo-app è sempre soddisfatta, ma oggi non cambia nulla in pratica.
						
						Sul piano delle risorse c'è spazio: gli altri carichi usano in tutto circa 0,6 core su 8, quindi restano circa 7,4 core per i 12 pod (circa 0,6 core ciascuno, se si dividono in parti uguali).
						
						### Cosa consiglio
						1. **Se puoi aggiungere almeno un nodo worker** (meglio dedicato al batch):
						 ``yaml
						 affinity:
						   podAffinity:
						   preferredDuringSchedulingIgnoredDuringExecution:
						   - weight: 50
						   podAffinityTerm:
						   labelSelector: {matchLabels: {app: demo-app}}
						   topologyKey: kubernetes.io/hostname
						   podAntiAffinity:
						   requiredDuringSchedulingIgnoredDuringExecution:
						   - labelSelector: {matchLabels: {app: worker-app}}
						   topologyKey: kubernetes.io/hostname
						   preferredDuringSchedulingIgnoredDuringExecution:
						   - weight: 30
						   podAffinityTerm:
						   labelSelector: {matchLabels: {app: cache-app}}
						   topologyKey: kubernetes.io/hostname
						 ``
						 Le label `app: ...` sono un'ipotesi: verifica i selector reali dei Deployment.
						2. **Se resti su un solo nodo**, l'isolamento va fatto sulle CPU, non sulla schedulazione:
						 - imposta il batch in classe QoS **Guaranteed**, con request uguale al limit e **valori interi di CPU**;
						 - attiva il **CPU Manager con policy `static`** sul kubelet, così i pod ricevono core in esclusiva e non li condividono con worker-app;
						 - usa solo regole di affinity/anti-affinity "preferred", mai "required".
						3. **Rivedi worker-app**: con il 97% di throttling è sottodimensionato. Alza il limite CPU, ad esempio a 500m–1 core, oppure aggiungi un HPA sulla CPU. Fallo **prima** di avviare il batch, altrimenti i due carichi si contenderanno i core proprio quando worker-app avrebbe più bisogno

Claude Sonnet 5.5	13.61 s (tot: 51.25 s)	2731	1538	4270	0.86	## Raccomandazione

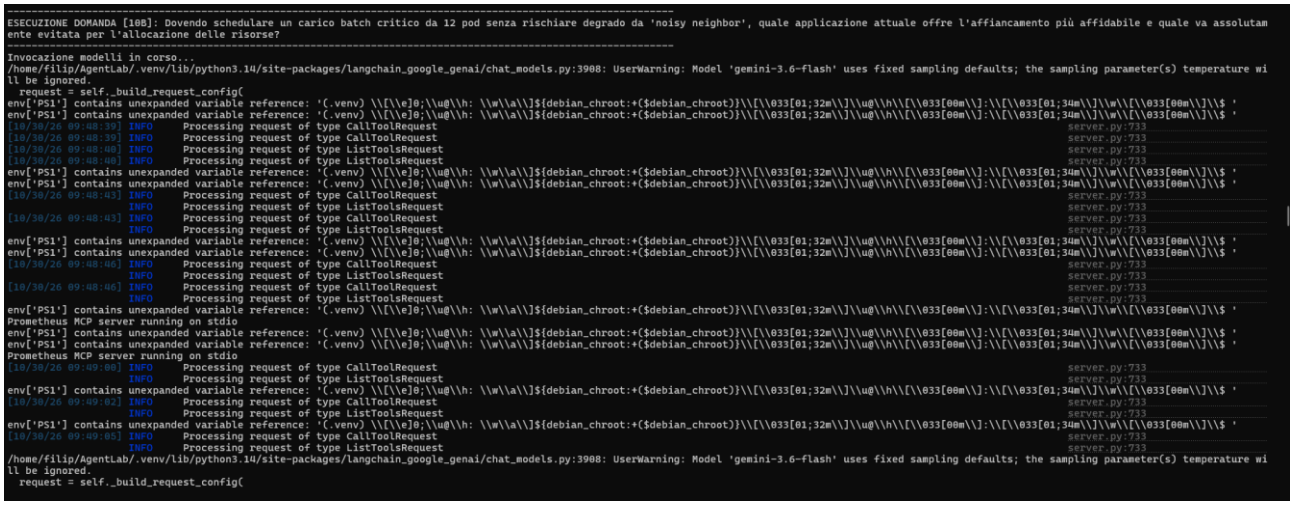
						quando worker-app diventa più disoglio
Claude Sonnet 5.5	12.61 s (tot: 51.25 s)	2731	1539	4270	0.76	## Raccomandazione **Anti-affinity indispensabile: `worker-app`.** È l'unico carico che consuma CPU in modo significativo. | App | CPU (cores) | Memoria | Disk write | Valutazione | |---|---|---|---|---| | **worker-app** (2 pod) | **0,57** (circa 0,28 per pod, se ripartito in modo uniforme) | 4,4 MB | 0 | Carico CPU-bound attivo, quindi fonte di contesa | | cache-app (1 pod) | 0,014 | 1,6 MB | ~448 KB/s | CPU trascurabile, con una leggera attività di I/O su disco | | demo-app (2 pod) | 0,0 | 14,4 MB | 0 | Idle | ### Anti-affinity (da `worker-app`) - Il batch da 12 pod è sensibile ai cicli CPU, quindi non deve condividere i nodi con `worker-app`. Con 0,57 core consumati, ogni nodo che lo ospita perde capacità di calcolo per il batch. - Usa `podAntiAffinity` con `labelSelector: app=worker-app` e `topologyKey: kubernetes.io/hostname`. - Usa la variante `required...` se il batch ha scadenze rigide. Usa `preferred...` con peso alto se il cluster ha pochi nodi, per evitare pod in Pending. - Valuta anche una anti-affinity o un `topologySpreadConstraints` tra i 12 pod del batch stesso. Così non si concentrano tutti sullo stesso nodo e non si contendono i core tra loro. ### Affinity (verso `demo-app` e `cache-app`) - **`demo-app`** è idle (0 CPU) e **`cache-app`** usa 0,014 core. Sono compatibili con il batch e non gli sottraggono cicli. - Se vuoi consolidare i carichi, puoi usare una `podAffinity` preferita verso questi pod, o più semplicemente un `nodeAffinity` verso i nodi che li ospitano. Non è però una condizione necessaria. - Un'eccezione riguarda `cache-app`. Scrive ~448 KB/s su disco, quindi se il batch fa I/O su disco intenso puoi avere contesa di I/O, anche se non di CPU. È un'ipotesi da verificare, perché lo snapshot non mostra i dati di I/O del batch. ## Limiti dell'analisi - Le metriche sono aggregate per applicazione. Non conosco il nodo su cui gira ogni pod, quindi non posso dire se `worker-app` condivide già nodi con gli altri carichi. - Mancano requests/limits, throttling CPU e capacità allocabile dei nodi. Non posso quindi dimensionare quanti dei 12 pod stanno su ciascun nodo. - Lo snapshot è istantaneo. Se vuoi, posso recuperare lo storico di `worker-app` per capire se il carico è costante o a picchi, prima di scegliere tra regola `required` e `preferred`. - Imposta in ogni caso `requests.cpu` (idealmente uguale ai `limits`, per la classe QoS Guaranteed) sui pod del batch, così lo scheduler riserva i core.

GPT-4o Mini	5.04 s (tot: 43.68 s)	1273	445	1718	0.00	Per pianificare un batch computazionale da 12 pod sensibile ai cicli CPU, è importante analizzare i carichi di lavoro attuali delle applicazioni esistenti per identificare eventuali contese e definire regole di affinità e anti-affinity. ### Analisi dei Carichi di Lavoro Attuali 1. **cache-app**: - CPU: 0.0144 cores - Memoria: 1.59 MB 2. **demo-app**: - CPU: 0.0 cores - Memoria: 14.44 MB 3. **worker-app**: - CPU: 0.5697 cores - Memoria: 4.42 MB ### Raccomandazioni #### Affinità - **worker-app**: Questo pod sta utilizzando una quantità significativa di CPU (0.5697 cores). Se il batch computazionale richiede un uso intensivo della CPU, potrebbe essere vantaggioso pianificare i nuovi pod in affinità con i pod di `worker-app`, poiché potrebbero condividere risorse in modo più efficiente. #### Anti-affinity - **demo-app**: Attualmente non sta utilizzando CPU (0.0 cores), ma ha una memoria relativamente alta (14.44 MB). Se i nuovi pod richiedono cicli CPU, è consigliabile applicare regole di anti-affinity per evitare che i nuovi pod vengano pianificati insieme ai pod di `demo-app`, poiché non contribuiscono al carico CPU e potrebbero causare contesa per altre risorse. ### Conclusione - **Definire regole di affinità** per i pod di `worker-app` per ottimizzare l'uso della CPU. - **Applicare regole di anti-affinity** per i pod di `demo-app` per evitare contese e garantire che i nuovi pod possano accedere alle risorse CPU necessarie senza interferenze. Queste raccomandazioni dovrebbero aiutare a pianificare il batch computazionale in modo più efficiente.

GPT-4o	4.79 s (tot: 43.42 s)	1273	413	1686	0.72	Per rispondere alla tua richiesta, analizziamo le metriche istantanee delle applicazioni attuali: 1. **cache-app**: - CPU: 0.0144 cores - Memoria: 1.59 MB - Disk Write: 448557.51 Bps - Network RX: 0.0 Bps 2. **demo-app**: - CPU: 0.0 cores - Memoria: 14.44 MB - Disk Write: 0.0 Bps - Network RX: 0.0 Bps 3. **worker-app**: - CPU: 0.5697 cores - Memoria: 4.42 MB - Disk Write: 0.0 Bps - Network RX: 0.0 Bps **Raccomandazioni:** - **Regole di Affinità**: - Considerando che il carico CPU è un fattore critico per il tuo batch computazionale, potresti voler definire regole di affinità per i pod che hanno un basso utilizzo di CPU. In questo caso, il `cache-app` e il `demo-app` hanno un utilizzo di CPU molto basso, quindi potresti voler pianificare i tuoi pod batch vicino a questi per sfruttare al meglio le risorse disponibili. - **Regole di Anti-Affinity**: - È indispensabile applicare regole di anti-affinity per evitare contese con il `worker-app`, che attualmente utilizza 0.5697 cores di CPU. Questo è significativamente più alto rispetto agli altri e potrebbe causare contese di risorse se i tuoi pod batch vengono pianificati sulla stessa macchina. In sintesi, pianifica i tuoi pod batch vicino a `cache-app` e `demo-app` e applica regole di anti-affinity per evitare il `worker-app`.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.4457 (mini: $0.0664 | 4o: $1.3793)

DOMANDA 10B



Sistema	Latenza Q (Tot con ETL)	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.45 ms	0	0	0	N/A	Co-locare preferibilmente con 'demo-app' (minor CPU: 0.0000 cores) ed ISOLARE da 'worker-app' (maggior CPU: 0.5914 cores)
Gemini 2.5 Flash	7.88 s (tot: 54.16 s)	1575	1502	3077	0.70	Basandosi sulle metriche istantanee fornite: * **Applicazione più affidabile per l'affiancamento:** L'applicazione **`demo-app`** è la più affidabile. Mostra un utilizzo di CPU e memoria estremamente basso (CPU a 0.0 e memoria a 14.86 MB) e nessuna attività di I/O su disco o rete. Questo indica che è quasi completamente inattiva o molto leggera, offrendo ampie risorse disponibili per il carico batch critico senza rischio di "noisy neighbor". * **Applicazione da evitare assolutamente:** L'applicazione **`worker-app`** va assolutamente evitata. Sebbene la sua memoria sia bassa, consuma la quantità più elevata di CPU (0.59 core) tra tutte le applicazioni. Affiancare un carico batch critico e potenzialmente intensivo in CPU a `worker-app` aumenterebbe significativamente il rischio di contesa per le risorse CPU e degrado delle prestazioni per entrambi i carichi di lavoro.
Gemini 3.6 Flash	69.99 s (tot: 116.27 s)	6836	2089	8925	0.90	Sulla base delle metriche istantanee e dell'analisi dello storico su database, ecco la raccomandazione per lo scheduling del carico batch da 12 pod: --- ### 1. Affiancamento Più Affidabile: **`demo-app`** * **Utilizzo CPU:** `0.0 cores` (completamente idle) * **Scrittura su Disco:** `0.0 Bps` * **Memoria:** `~14.86 MB` * **Motivazione:** Sia le metriche attuali sia lo storico confermano che `demo-app` ha un profilo di consumo risorse pressoché nullo e costante nel tempo, senza picchi di elaborazione né attività I/O di disco. Offre la massima disponibilità di risorse (*headroom*) su CPU e I/O, annullando il rischio di interferenza da *noisy neighbor*. --- ### 2. Applicazione da Evitare Assolutamente: **`worker-app`** * **Utilizzo CPU:** `~0.591 cores` (591m CPU) * **Scrittura su Disco:** `0.0 Bps` * **Memoria:** `~5.23 MB` * **Motivazione:** `worker-app` mostra un consumo di CPU elevato e continuo (con intervalli storici tra `~0.28` e `~0.59` cores per 2 pod). Posizionare un carico batch critico nello stesso dominio di calcolo/nodo di `worker-app` causerà inevitabilmente **contesa CPU e fenomeni di CPU Throttling / Noisy Neighbor**. --- ### 3. Note Aggiuntive su **`cache-app`** * **Utilizzo CPU:** `~0.015 cores` * **Scrittura su Disco:** `~537.29 KB/s` (I/O attivo) * **Considerazione:** Sebbene l'uso della CPU sia contenuto, l'applicazione genera un flusso continuo di scritture su disco. Se il batch critico da 12 pod richiede un throughput di I/O elevato, occorre prestare attenzione al potenziale collo di bottiglia sullo storage.
Claude Opus 5.5	46.30 s (tot: 92.57 s)	25865	2422	28287	0.95	# Dove affiancare il batch da 12 pod **In breve:** affianca il batch a **demo-app**, che è il vicino più tranquillo. Evita **worker-app**, che è la fonte principale di contesa CPU. Però nel cluster c'è **un solo nodo**, quindi oggi non puoi davvero tenere separati i carichi. ## Cosa ho guardato Ho letto lo storico su MongoDB delle tre app nella finestra di 6 ore, ma i dati coprono solo circa 11 minuti (08:34–08:45). Ho anche verificato con Prometheus su quali nodi girano i pod e quanta capacità ha il nodo.

Claude Opus 5.5	46.30 s (tot: 92.57 s)	25865	2422	28287	0.95	# Dove affiancare il batch da 12 pod **In breve:** affianca il batch a **demo-app**, che è il vicino più tranquillo. Evita **worker-app**, che è la fonte principale di contesa CPU. Però nel cluster c'è **un solo nodo**, quindi oggi non puoi davvero tenere separati i carichi. ## Cosa ho guardato Ho letto lo storico su MongoDB delle tre app nella finestra di 6 ore, ma i dati coprono solo circa 11 minuti (08:34–08:45). Ho anche verificato con Prometheus su quali nodi girano i pod e quanta capacità ha il nodo. ## Il punto più importante: un solo nodo Tutti i 5 pod girano su `thesis-cluster-control-plane`, che offre in tutto 8 core, circa 3,7 GiB di RAM e 110 pod. Quindi: - Regole come l'affinità o l'anti-affinità tra pod oggi **non hanno alcun effetto**: i 12 pod batch finiranno comunque accanto a tutte e tre le app. - Il nodo è il control-plane. Mettere un carico critico lì rischia di rallentare anche componenti di sistema come apiserver ed etcd. ## Classifica dei vicini | App | CPU (storico) | Memoria | Scritture su disco | Giudizio | |---|---|---|---|---| | **demo-app** (2 pod) | 0 core fissi | 14,9–15,5 MB, piatta | 0 | ✅ **La più affidabile** | | **cache-app** (1 pod) | 0,014–0,023 core | 1–2,4 MB | **209 → 537 KB/s, in crescita** | ⚠️ Va bene solo se il batch non è pesante su disco | | **worker-app** (2 pod) | **0,28 → 0,59 core**, più che raddoppiata | ~5,2 MB | 0 | ❌ **Da evitare** | - **demo-app:** usa zero CPU e zero disco, e la memoria è stabile in tutti i campioni. Non ruba risorse, quindi è il vicino ideale. È anche di fatto inattiva: si potrebbe ridurre a 1 replica. - **worker-app:** è l'unica che consuma CPU in modo continuo, circa 0,3 core per pod. Nel periodo osservato è passata da 0,28 a circa 0,59 core e si è stabilizzata lì, con oscillazioni tra 0,53 e 0,59. Un batch che usa molta CPU ci entrerebbe in competizione, con rischio di rallentamenti forzati se ci sono limiti CPU. - **cache-app:** usa poca CPU, ma scrive su disco in modo continuo e in aumento (+157% in 11 minuti). Se il batch scrive molto su disco, si contenderanno il disco del nodo. ## Cosa ti consiglio di fare 1. **Aggiungi almeno un nodo worker** dedicato al batch, con un taint e una toleration specifica, così il batch non finisce sul control-plane. 2. Quando avrai più nodi, separa il batch da worker-app con una regola di anti-affinità (`podAntiAffinity` sul label `app=worker-app`). Separalo anche da cache-app se il batch è pesante su

						- **demo-app:** usa zero CPU e zero disco, e la memoria è stabile in tutti i campioni. Non ruba risorse, quindi è il vicino ideale. È anche di fatto inattiva: si potrebbe ridurre a 1 replica. - **worker-app:** è l'unica che consuma CPU in modo continuo, circa 0,3 core per pod. Nel periodo osservato è passata da 0,28 a circa 0,59 core e si è stabilizzata lì, con oscillazioni tra 0,53 e 0,59. Un batch che usa molta CPU ci entrerebbe in competizione, con rischio di rallentamenti forzati se ci sono limiti CPU. - **cache-app:** usa poca CPU, ma scrive su disco in modo continuo e in aumento (+157% in 11 minuti). Se il batch scrive molto su disco, si contenderanno il disco del nodo. ## Cosa ti consiglio di fare 1. **Aggiungi almeno un nodo worker** dedicato al batch, con un taint e una toleration specifica, così il batch non finisce sul control-plane. 2. Quando avrai più nodi, separa il batch da worker-app con una regola di anti-affinità (`podAntiAffinity` sul label `app=worker-app`). Separalo anche da cache-app se il batch è pesante su disco. 3. Assegna al batch una **PriorityClass** e la classe **Guaranteed** (requests uguali ai limits). Così, sotto pressione, sono gli altri pod a essere sacrificati per primi, non il batch. 4. **Capacità:** oggi le tre app usano circa 0,6 core su 8. C'è spazio per 12 pod fino a circa 0,5 core l'uno (con margine), ma la RAM è poca: circa 3,7 GiB in tutto, al netto del sistema. Ti serve la dimensione per pod del batch per confermare. **Limiti dei dati:** lo storico copre circa 11 minuti, non 6 ore, quindi i trend di worker-app e cache-app sono indicativi. Non ho controllato requests e limits già impegnati sul nodo, né l'uso reale del control-plane. Prima di schedulare, verifica `kube_pod_container_resource_requests` e `node_memory_MemAvailable_bytes`. Per un piano di capacità preciso, con dati su più giorni e la dimensione dei pod batch, conviene passare la richiesta a un modello più avanzato.

Claude Sonnet 5.5	26.59 s (tot: 72.87 s)	16482	1725	18207	0.95	**Affiancamento più affidabile: `demo-app`. Da evitare: `worker-app`.** Questa conclusione si basa sullo storico di circa 24 ore, che conferma lo snapshot attuale. | App | CPU | Memoria | Disco / rete | Valutazione | |---|---|---|---|---| | **demo-app** (2 pod) | 0 core quasi sempre. Solo due campioni a ~0,0014 core e uno a 0,0025 | 14,5–19 MB, stabile | Disco ~0 B/s, rete 0 | Neighbor silenzioso e prevedibile | | **cache-app** (1 pod) | 0,004–0,023 core | 1–4 MB | Scrittura disco continua a ~0,4–0,59 MB/s | Basso rischio CPU, ma rumoroso sul disco | | **worker-app** (2 pod) | ~0,51–0,59 core in modo costante | ~5 MB | Disco e rete trascurabili | Consumatore di CPU persistente | **Perché `demo-app` è la scelta migliore** - Non genera contesa su CPU, disco o rete in nessun campione. - Il suo profilo non cambia nel tempo, quindi non ci sono picchi imprevisti da temere. **Perché evitare `worker-app`** - Occupa CPU in modo continuo: ~0,55–0,59 core in quasi tutti i campioni, cioè ~0,3 core per pod. - Il valore si ferma spesso intorno a 0,59, con pochi campioni sopra. Può indicare un limite CPU raggiunto, con possibile throttling. È un'ipotesi, perché non ho i dati su limits e requests. - Un batch da 12 pod sullo stesso nodo si contenderebbe i core con questo carico, che non scende mai. **Nota su `cache-app`** - Scrive su disco in modo costante (~0,5 MB/s). Se il batch fa molto I/O, anche `cache-app` può causare degrado. Per un batch CPU-bound è accettabile. **Limiti dell'analisi** - Le metriche sono aggregate per applicazione. Non so su quali nodi girino i pod, né quali requests e limits abbiano. - Non so quanta capacità libera abbiano i nodi di `demo-app`, né quante risorse richieda ciascuno dei 12 pod. - Prima di schedulare il batch conviene: - verificare che il nodo di `demo-app` abbia CPU e memoria libere per 12 pod; - impostare requests e limits sul batch; - usare pod anti-affinity verso `worker-app`, oppure taint e toleration, per garantire la separazione.

GPT-4o Mini	4.33 s (tot: 50.61 s)	1276	384	1660	0.00	Analizzando le metriche attuali delle applicazioni: 1. **cache-app**: - CPU: 0.0149 cores - Memoria: 2.21 MB - Disk Write: 537294.58 Bps - Network RX: 0.0 Bps 2. **demo-app**: - CPU: 0.0 cores - Memoria: 14.86 MB - Disk Write: 0.0 Bps - Network RX: 0.0 Bps 3. **worker-app**: - CPU: 0.5914 cores - Memoria: 5.23 MB - Disk Write: 0.0 Bps - Network RX: 0.0 Bps ### Raccomandazioni: - **Applicazione più affidabile per affiancamento**: **cache-app** - Ha un utilizzo di CPU molto basso (0.0149 cores) e una memoria ridotta (2.21 MB), il che indica che non sta saturando le risorse e può gestire carichi aggiuntivi senza rischiare di influenzare negativamente le prestazioni. - **Applicazione da evitare per allocazione delle risorse**: **worker-app** - Sebbene non sia in saturazione, ha un utilizzo di CPU relativamente più alto (0.5914 cores) rispetto alle altre applicazioni. Questo potrebbe portare a un degrado delle prestazioni se si aggiungessero ulteriori carichi batch, specialmente in un contesto di 'noisy neighbor'. In sintesi, per schedulare il carico batch critico, si consiglia di affiancare **cache-app** e di evitare **worker-app** per l'allocazione delle risorse.

GPT-4o	4.25 s (tot: 50.53 s)	1276	409	1685	0.21	Per determinare quale applicazione offre l'affiancamento più affidabile e quale va evitata per l'allocazione delle risorse, analizziamo le metriche istantanee fornite: 1. **Cache-app**: - CPU: 0.0149 cores - Memoria: 2.21 MB - Disk Write: 537294.58 Bps - Network RX: 0.0 Bps 2. **Demo-app**: - CPU: 0.0 cores - Memoria: 14.86 MB - Disk Write: 0.0 Bps - Network RX: 0.0 Bps 3. **Worker-app**: - CPU: 0.5914 cores - Memoria: 5.23 MB - Disk Write: 0.0 Bps - Network RX: 0.0 Bps **Raccomandazioni:** - **Affiancamento più affidabile**: - **Cache-app** sembra essere l'applicazione più leggera in termini di utilizzo di CPU e memoria. Anche se ha un certo utilizzo di scrittura su disco, non utilizza la rete. Questo la rende un buon candidato per l'affiancamento, poiché è improbabile che interferisca con il carico batch critico. - **Applicazione da evitare**: - **Worker-app** utilizza una quantità significativa di CPU (0.5914 cores), il che potrebbe causare problemi di "noisy neighbor" se affiancata a un carico batch critico. È consigliabile evitare di schedulare il carico batch su nodi che ospitano questa applicazione. In sintesi, per minimizzare il rischio di degrado delle prestazioni, è consigliabile affiancare il carico batch a **Cache-app** e evitare **Worker-app**.